Yozgat Bozok Üniversitesi

Sistem Yazılımı

GÖKALP ÇINARER

Sistem Programlamaya Giriş

Bilgisayar donanımıyla bir arayüz sunan, uygulama programlarına birçok yönden hizmet veren programlara "sistem programları", programlamanın bunlarla ilgili alanına da "sistem programlama" denilmektedir.

Basit olarak örnek verilebilecek sistem programlarından bazıları şunlardır:

- İşletim sistemleri, derleyiciler ve yorumlayıcılar, gömülü sistem programları, virüs ve antivirüs yazılımları, editörler, veri tabanı motorları, oyun motorları vb.

Sistem programlama kavramı, uygulama geliştiricisinin sistemle daha alt seviye bir katman üzerinden etkileşime geçtiğinden dolayı geliştirmelerin sisteme daha yakın bir noktada yapıldığı yazılım geliştirme modelini tarif eder.

Yazılım Geliştirme Süreci

- Herhangi bir yazılım geliştirme yalnızca kodlamadan oluşmamaktadır. Kodlama aşamasına gelene kadar geçen süreçler, uygulamanın gereksinimlerinin öğrenilmesinden başlayıp kullanıcının kullanımı ve bakımına kadar bir çok aşamayı barındırır. Yazılım talepleri ile alakalı gelişmeler sürekli olarak değiştiği ve büyüdüğünden, söz konusu aşamalar devamlı bir döngü biçiminde ele alınır. Döngü içinde herhangi bir aşamada geriye dönmek ve tekrar etmek söz konusudur. Bu döngünün temel bazı adımlarını analiz, tasarım ve kodlama, test ve bakım olarak sıralayabiliriz.

Analiz

- Yazılımın amacı, yani yazılımın ne yapması gerektiği iyi tespit edilmelidir. Bu aşamanın atlanması veya bu aşamada yapılan yanlışlar yazılımın ileriki aşamalarında birçok problemlere yol açmaktadır.
- Projede zaman kaybı yaratacak önemsiz veya etkisiz özellikler bir sonraki aşamaya aktarılabilmelidir. Daha sonra bu özelliklerin gerekli olup olmadığı tekrar analiz edilebilecektir. Böylece proje süreci minimuma indirgenmiş ve proje en etkili bir biçimde tamamlanmış olacaktır.
- Kullanılacak en doğru yazılım dili, yazılım mimarisi, sunucu gereksinimleri belirlenmelidir.

Tasarım ve Kodlama

- Bu aşamada yazılımın tasarımı yapılır. Projede bulunacak modüller ve bu modüllerin ne iş yapacağı belirlenir.
- Tasarımların sade ve kullanıcıya hitap eden tasarımlar olmasına özen gösterilmelidir.
- Modüllerin hızlı çalışması için sayfa içerisinde gereksiz sorgu olmamasına dikkat edilmelidir.
- Tasarımı yapılmış olan yazılımın gerçek kodlaması yapılır.

Test

- Yazılımın aşağı seviyeli ve yüksek seviyeli testleri kodlama aşamasıyla eş zamanlı olarak yürütülür.
- Bu testlerde yazılımın belirlenen gereksinimlerin karşılanıp karşılanmadığının ve doğru sonuçlar üretip üretmediğinin testleri yapılmalıdır.
- Güvenlik testleri yapılarak, sistemdeki açıklıklar kapatılmalıdır.
- Stres testleri ile sistemin dayanıklılığı test edilmeli, gerekli yerlerde sorgular optimize edilmelidir.

Bakım

- Yazılım kullanıcıya sunulduktan sonra çeşitli geliştirmeler, düzeltmeler yapılabilir ve kullanım hizmetleri verilebilir.
- Hedeflenen durum, üretilmiş yazılımın sorunsuz ve istenilen biçimde çalışmasını sağlamaktır.

Standartlardaki Önemli Kavramlar

Implementation-Defined Behavior (Uygulama Tanımlı Davranış)

- Bir durum standartlarda bu şekilde belirtilmişse, bu konu derleyicileri hazırlayanların inisiyatifine bırakılmıştır. Bu durumlar derleyiciyi yazanlar tarafından belgelenmek zorundadır. Örneğin “int” türünün uzunluğu implementation dependent biçimindedir. Minimum limitleri verilmiştir ama gerçek uzunluk derleyiciyi yazanlara bırakılmıştır. Derleyiciyi hazırlayanlar bu seçimi nasıl yaptıklarını belirtmek zorundadırlar.

Undefined Behavior (Tanımsız Davranış)

- Yorumlanması derleyiciden derleyiciye değişebilen ve genellikle anlamsız olan durumları belirtir. Bu durumlara karşı derleyicinin davranışı standartlarda undefined behavior olarak belirtilmiştir. Derleyiciyi hazırlayanlar bu tür işlemleri nasıl yorumladıklarını belgelemek zorunda değillerdir.
- Örneğin: result = ++b + ++b;
- Tipik bir undefined behavior durumudur.

Unspecified Behavior (Belirsiz Davranış)

- “Implementationdependent behavior” a benzer, birkaç olasılık söz konusuyken, derleyicinin hangi olasılığı tercih ettiğinin belirsiz olduğu durumdur. Olasılıklar sınırlıdır, seçeneklerin hepsi mantıklıdır, ancak derleyiciyi hazırlayanlar seçimlerini belgelemek zorunda değildirler.

Diagnostic Message (Tanımlayıcı Mesaj)

- Standartlara göre belirtilen sentaks ve semantik kurallarına uymayan durumlar derleyici tarafından tespit edilerek bir mesaj olarak gösterilmelidir. Bu mesaja diagnostic message denir. Standartlarda, mesajın ne olması gerektiği, yani uyarı ya error olması gerektiği belirtilmemiştir. Yalnızca bu tür durumlarda bir mesaj vermesi gerektiği belirtilmiştir.

Macar Notasyonu

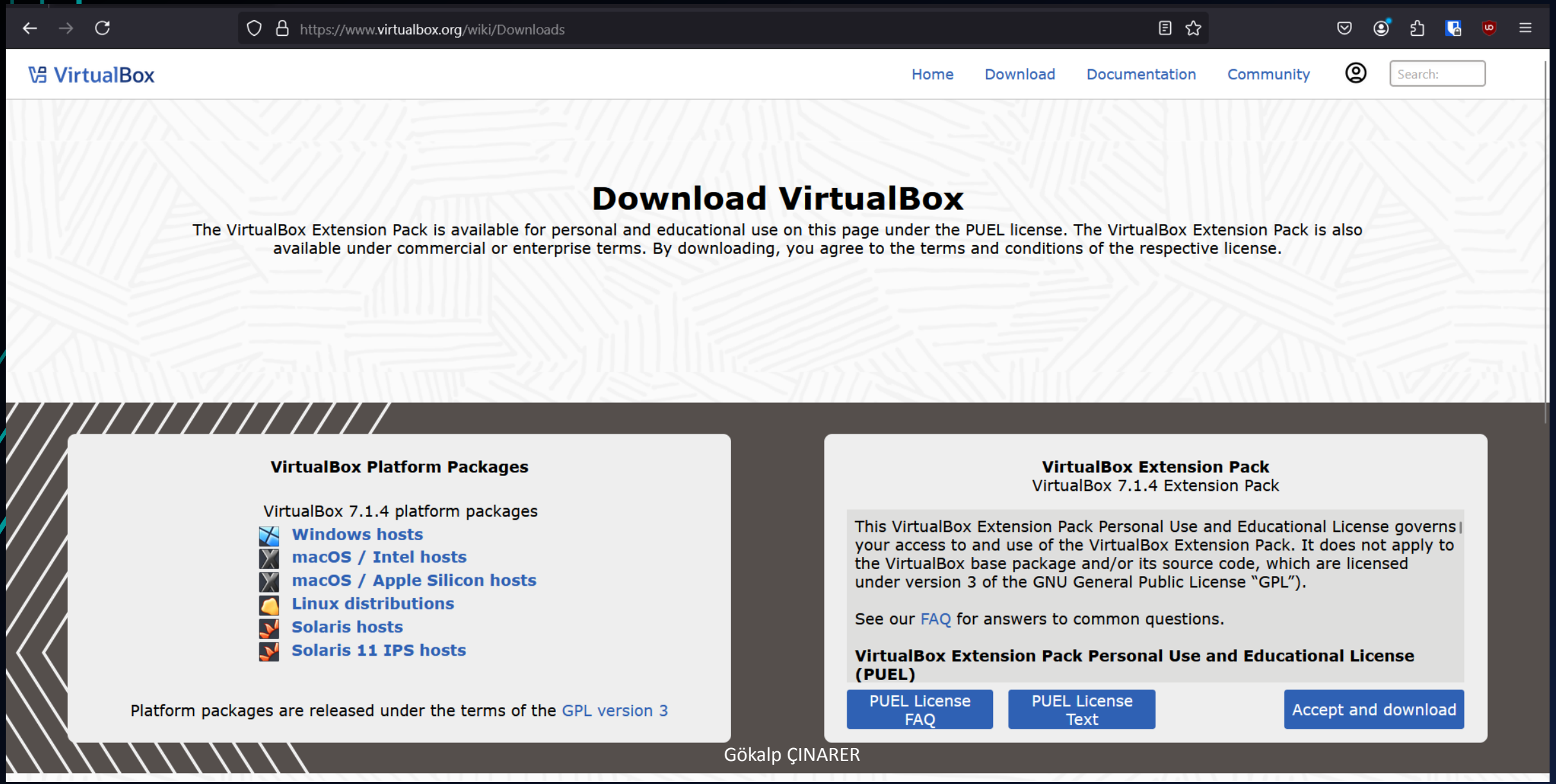
- Macar notasyonu, temel olarak bir değişkenin isimlendirilmesine ilişkin kurallar içeren bir belirlemeler topluluğudur. Charles Simone tarafından oluşturulmuştur. Microsoft API programlama sisteminde ve SDK (software development kit) içerisinde bu notasyonu kullanmaktadır. Macar notasyonunun en önemli özelliği değişken isimlerinin, değişkenin türünü belirten bir önekle başlatılmasıdır. Macar notasyonunun ana noktaları şunlardır. Fonksiyon isimleri, her sözcüğün ilk harfi büyük olacak şekilde önce bir fiil, sonra nesnesi gelecek şekilde isimlendirilir.

Linux Sanal Makine Ortamı Oluşturmak

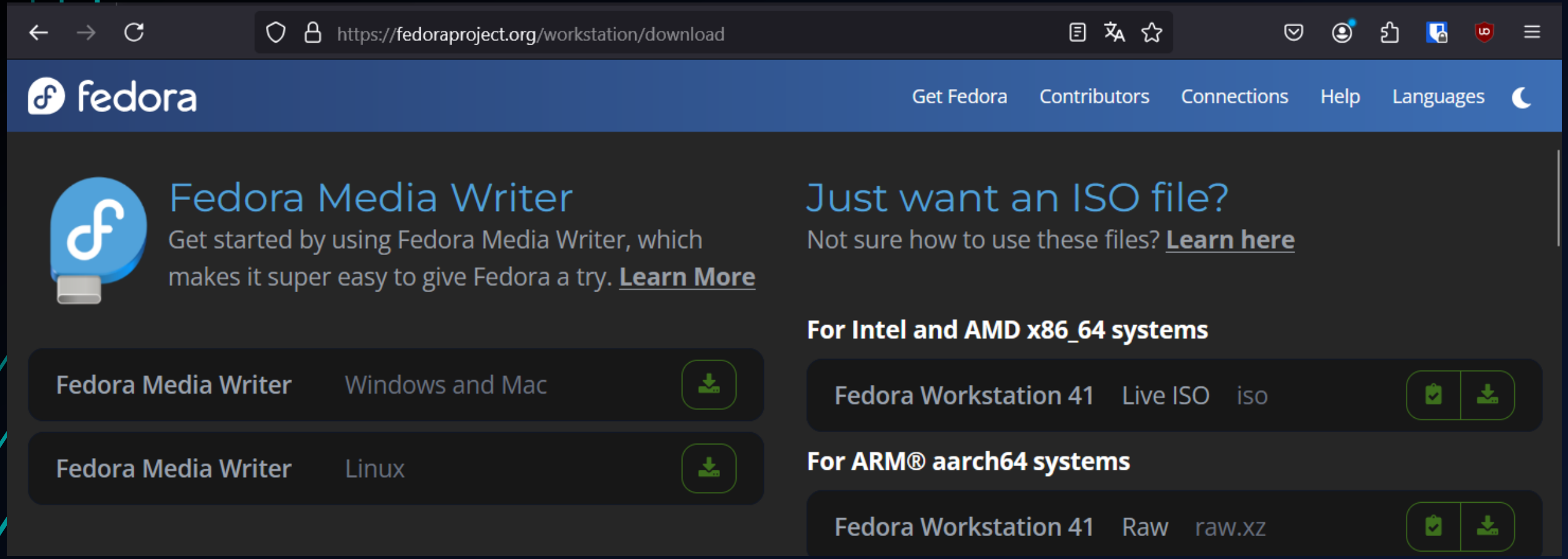
- VMware, VirtualBox, QEMU gibi sanallaştırma ve hypervisor yazılımlarla kuracağınız herhangi bir işletim sistemi tamamen sanallaştırma yoluyla çalıştırılabilmektedir.
- Bu programların çoğu sanallaştırmada "host" ve "guest" sistemler arasında kopyala-yapıştır işlemlerine de olanak tanır.
- Sanallaştırma yazılımları "host" olarak Windows, Linux ve macOS sistemlerinde bulunmaktadır.

- Dersimizde VirtualBox üzerinden gideceğiz. Uygulamayı www.virtualbox.org/wiki/Downloads adresinden işletim sisteminize uygun kurulum dosyasını seçin ya da eğer Linux kullanıyorsanız terminal üzerinden kurabilirsiniz.
- Linux dağıtımlarının neredeyse hepsi sunucularında VirtualBox için kurulum paketlerini paket yöneticileirne bağlı sunucularda barındırmaktadır.
- Sanal işletim sistemi olarak Fedora Linux kullanacağız. Kurulum ISO dosyasını ise fedoraproject.org/workstation/download adresinden işlemcinize uygun olanı seçerek indirin.
- Bu seçenek genellikle «For Intel and AMD x86_64 systems» olacaktır.

VirtualBox sayfasından indirme seçenekleri



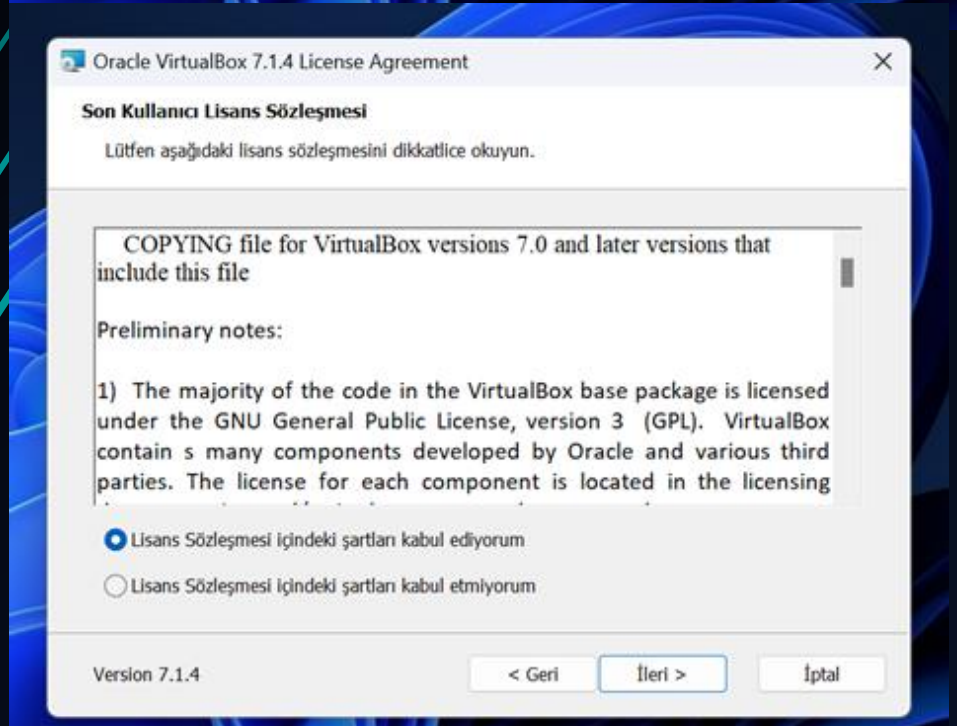
Fedora indirme sayfasından bir görüntü



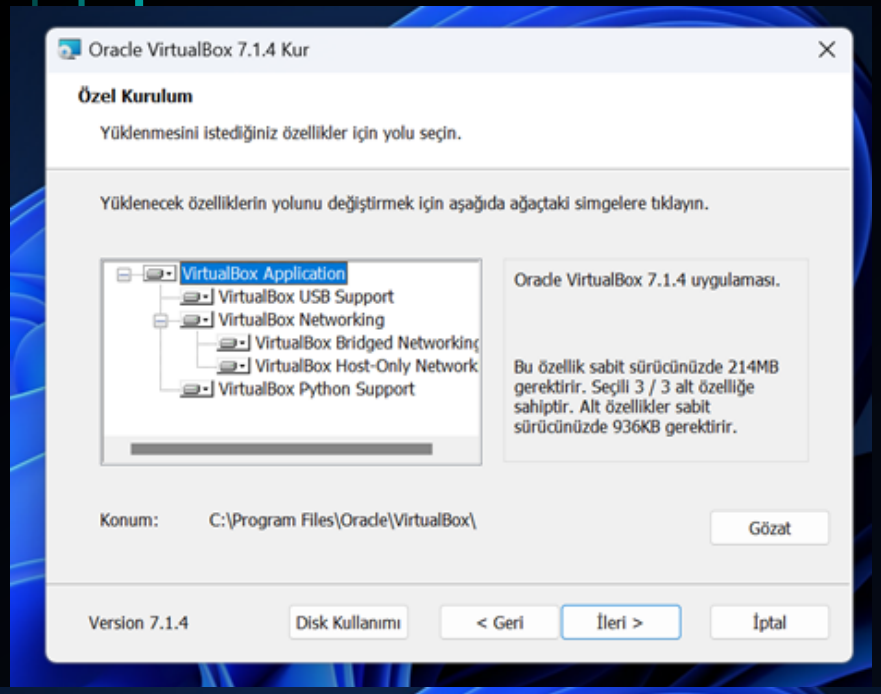
Sağ taraftaki ISO dosyası seçeneklerinden «For Intel and AMD x86_64 systems» olanı seçerek ve seçenek içindeki ikinci butona basarak indirebilirsiniz.

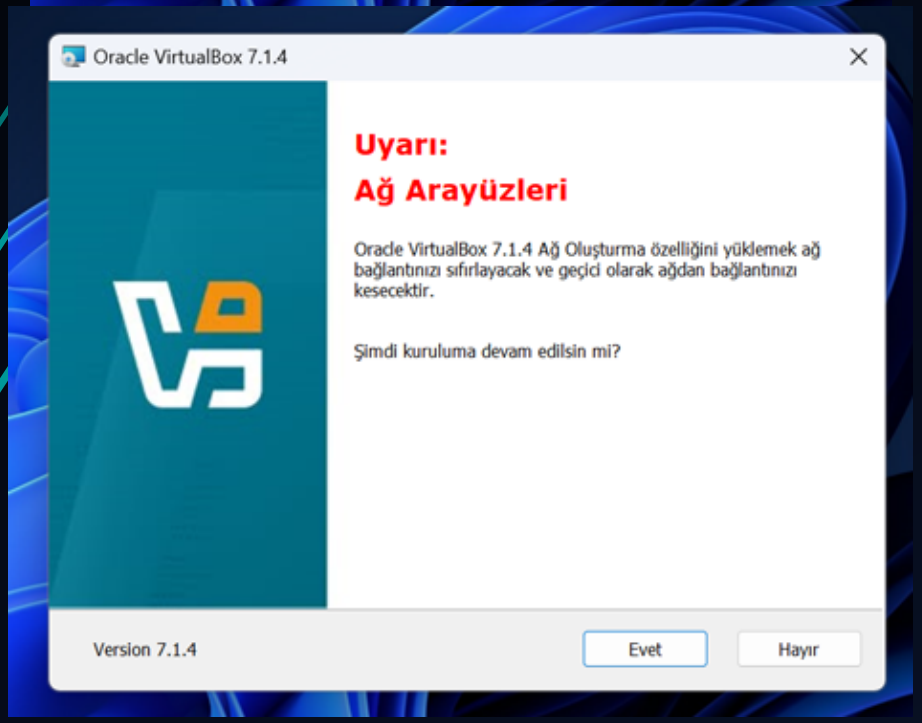
- Elde artık Fedora kurulum dosyası ve VirtualBox kurulum dosyası bulunduğuna göre başlayabiliriz.
- VirtualBox kurulum dosyasını açın ve kurulum ekranını harfiyen takip ederek kurulumunu gerçekleştirin.



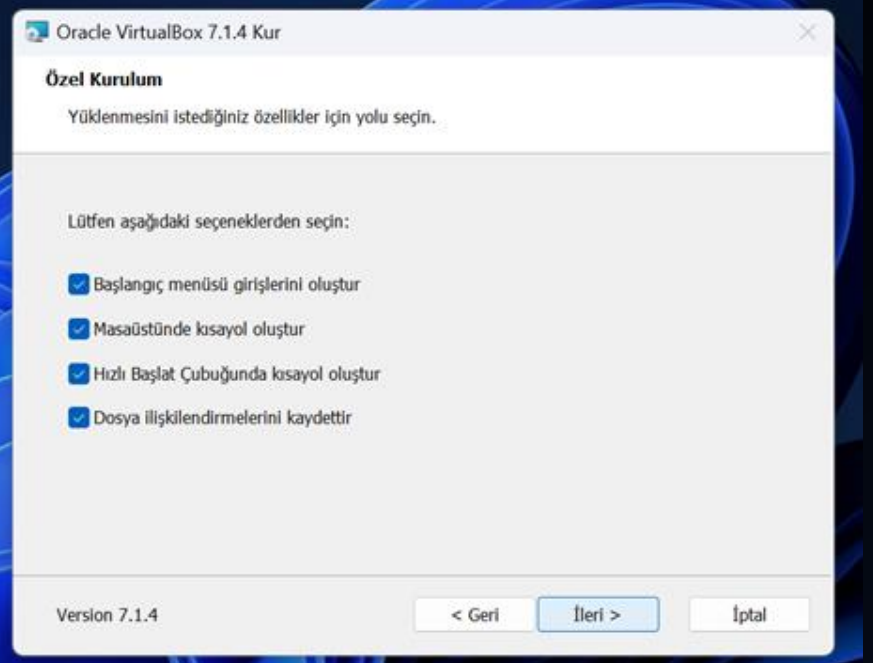


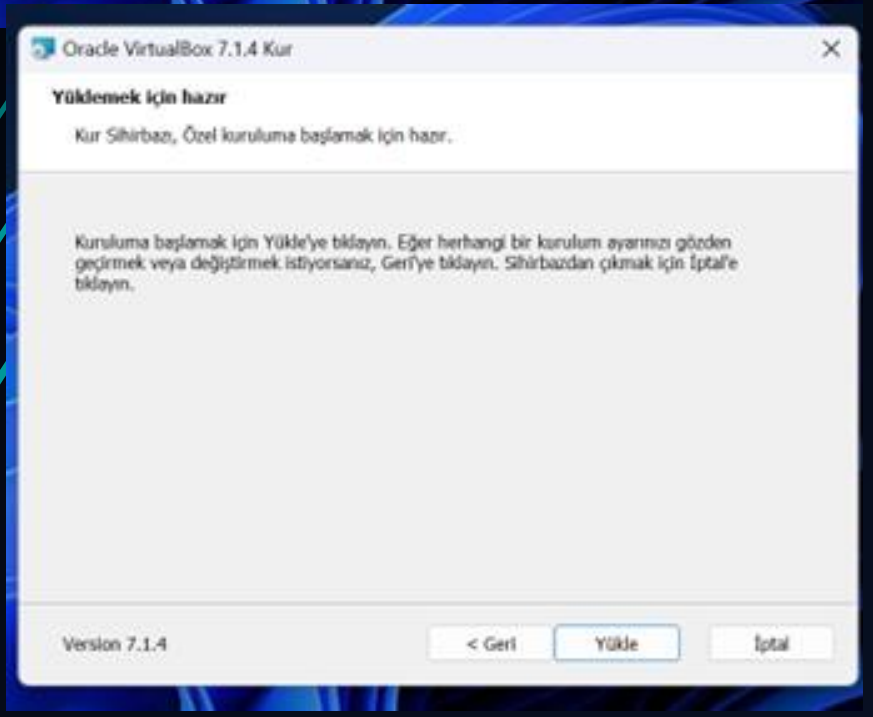
- Kurulum ekranı bizleri karşıladı.
- İleri diyerek kurulumu adım adım ilerleyerek tamamlayalım.
- Lisans sözleşmesini kabul ederek sonraki aşamaya geçelim.



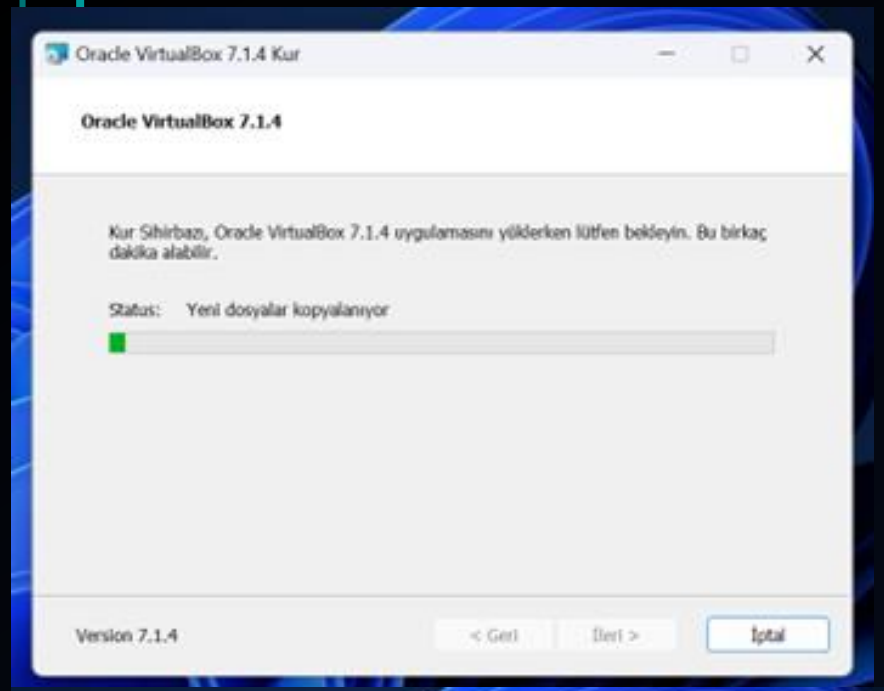


- Bir sonraki aşamada VirtualBox'ın kurulacak kısımlarının ayarlanması var.
- Özellikle bir şey değiştirmenize gerek kalmadan ekranda gösterildiği gibi kuruluma devam edebilirsiniz.
- Ağ arayüzlerinin anlık olarak yeniden başlatılacağı uyarısı gelecek, gayet normal. Onaylayarak devam ediyoruz.



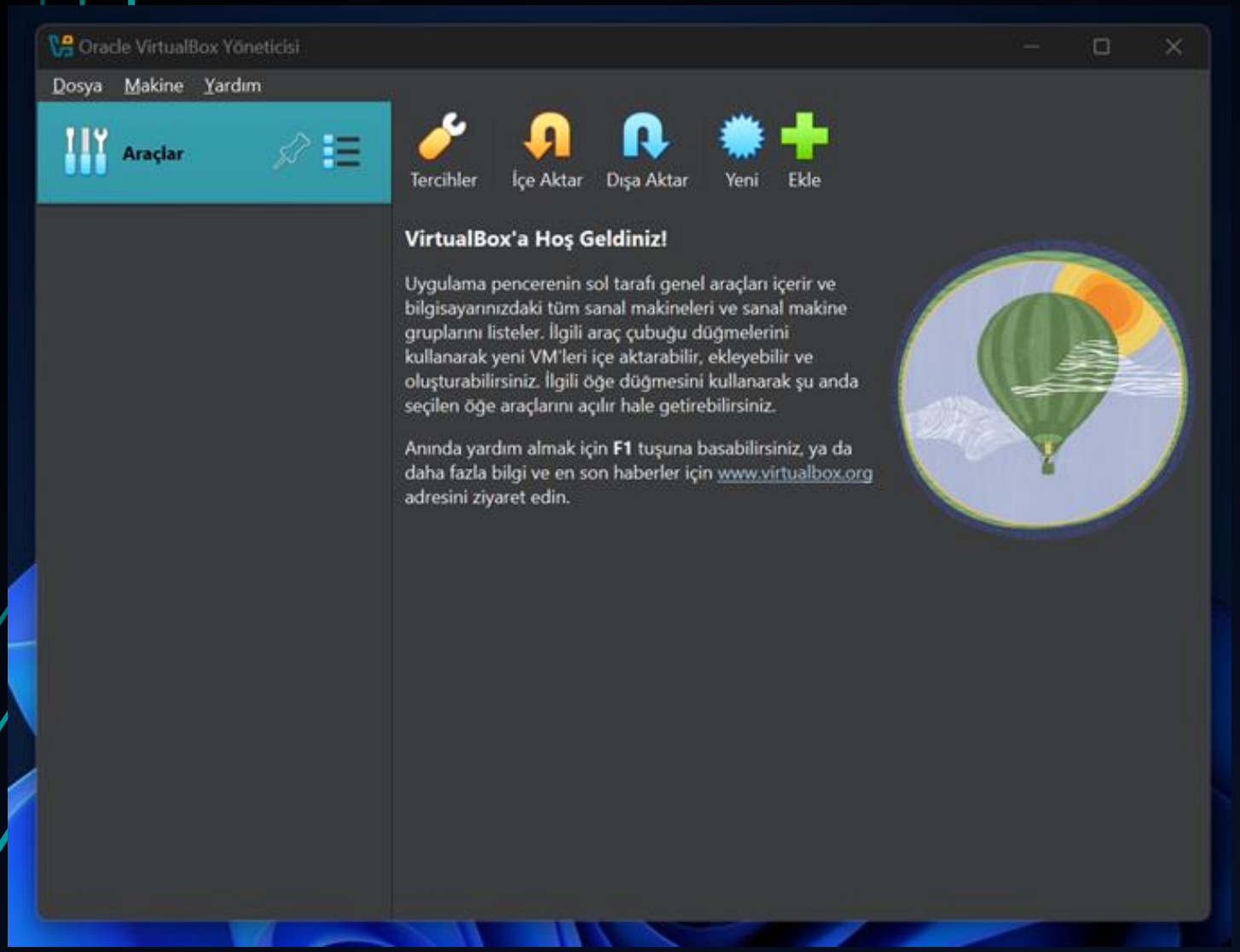


- Bir sonraki aşamada ise bilgisayara program ile ilgili eklenecek ufak değişiklikler soruluyor.
- Özellikle istemediklerinizi devre dışı bırakmak size kalmış.
- İleri diyerek devam ediyoruz ve ondan sonra gelen pencerede kurulumumuzu başlatıyoruz.
- Bu işlem 5-10 dk sürecektir.

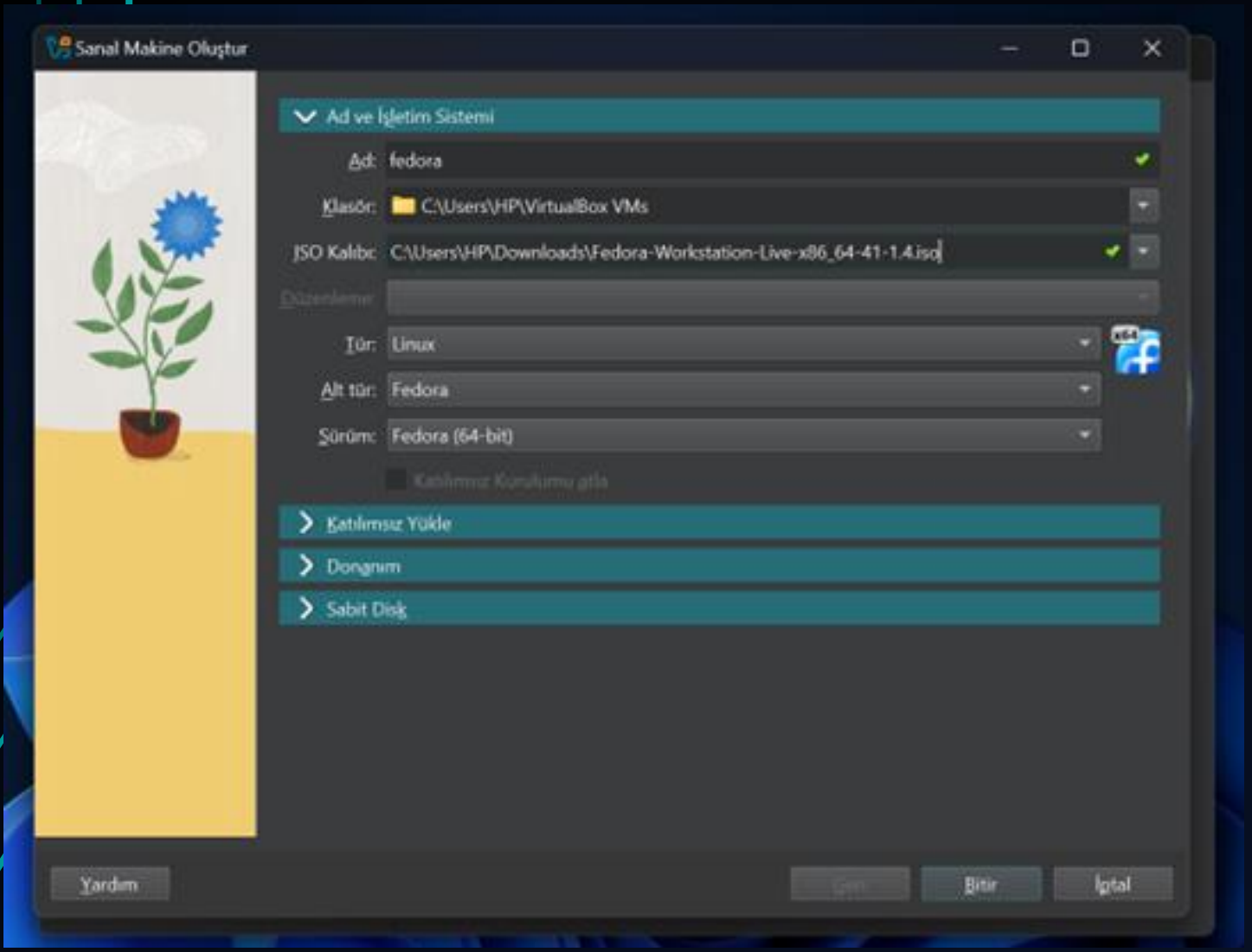




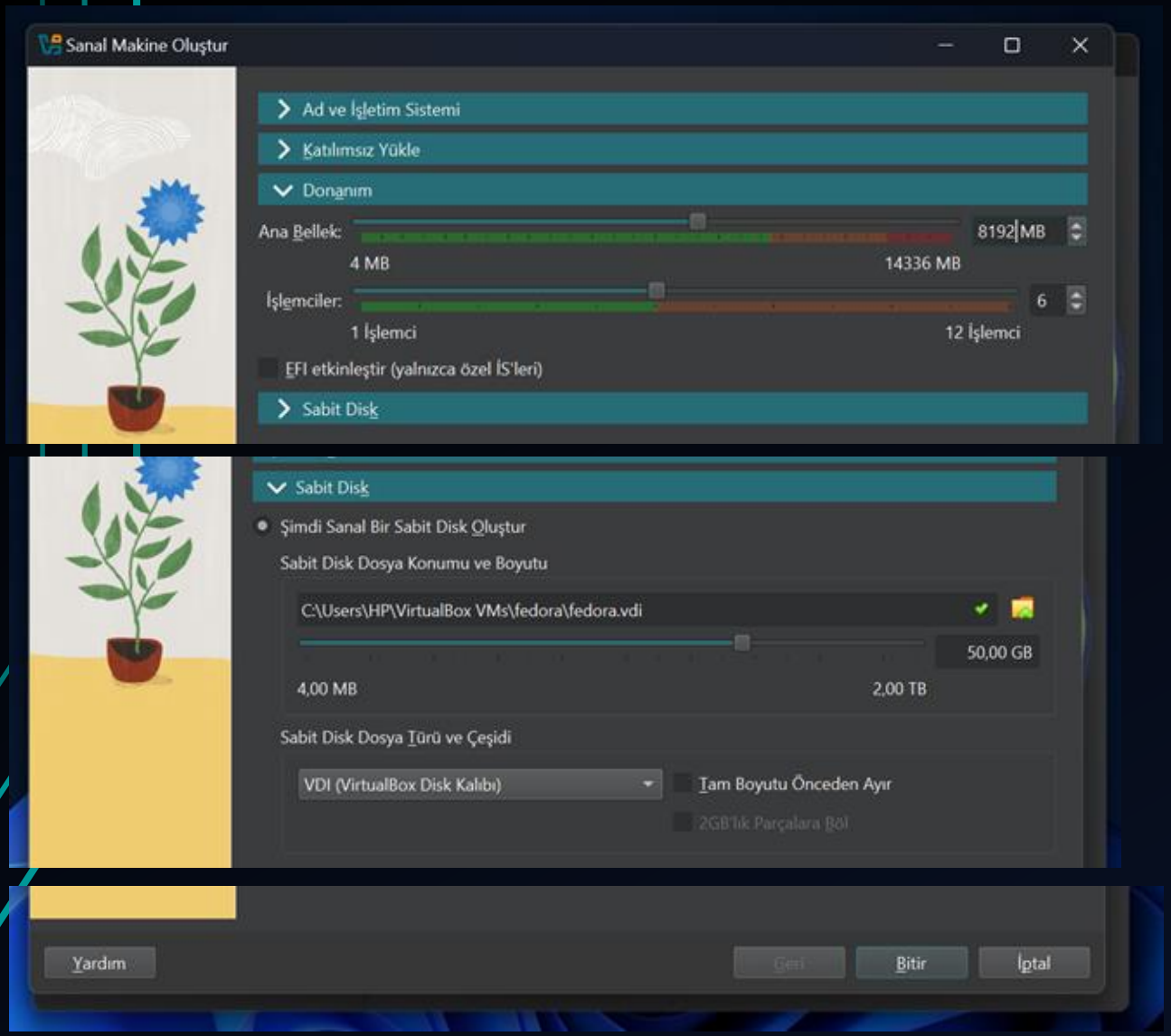
- Kurulumun devam ettiğine dair bir ekran görüntüsü görüyorsunuz.
- Kurulum bitttiğinde VirtualBox uygulamasını başlatarak kurulumu bitiriyoruz.



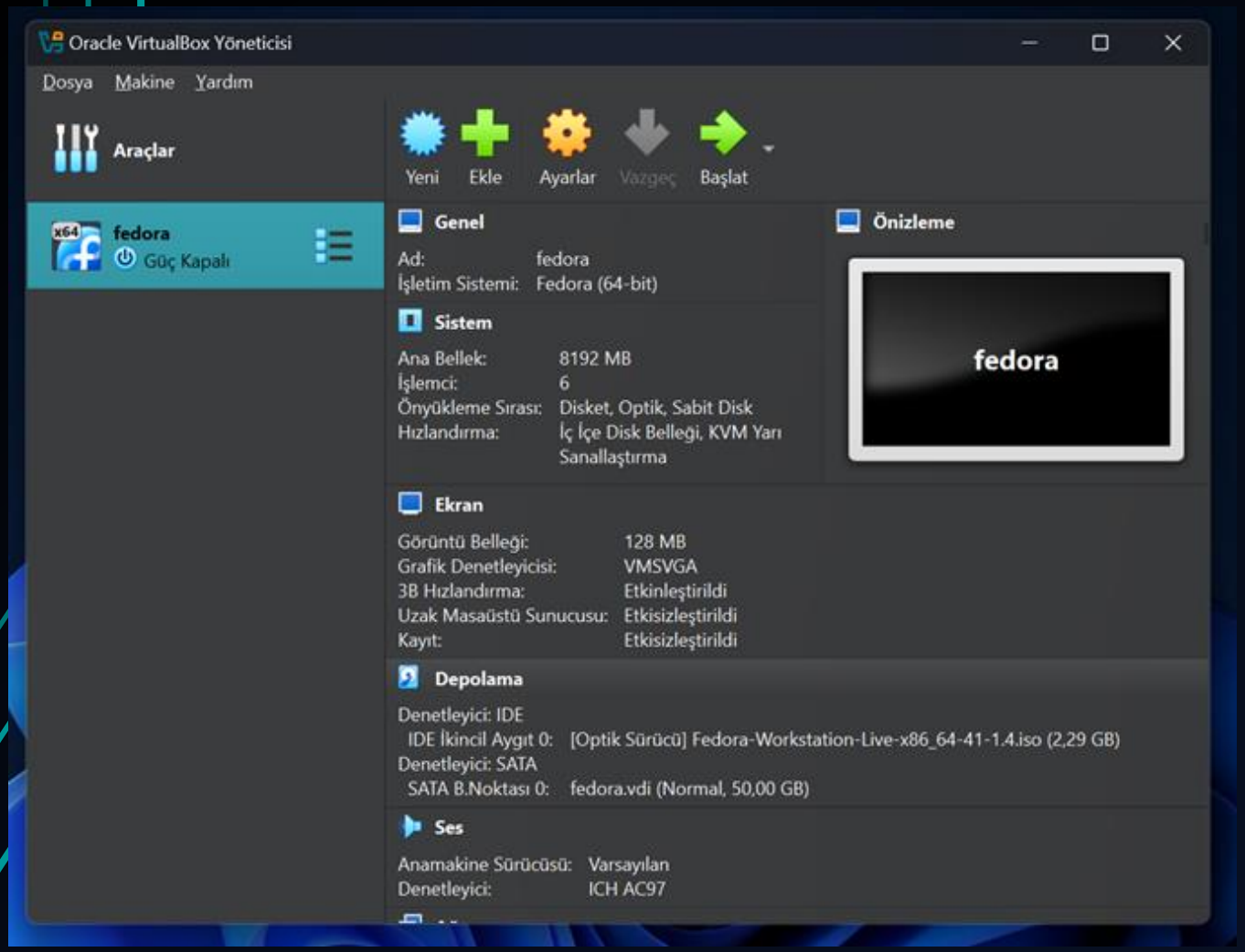
- VirtualBox’ın yönetim ekranı bizi karşılıyor.
- Yeni tuşuna basarak ilk sanal makinemizi oluşturmaya başlayalım.



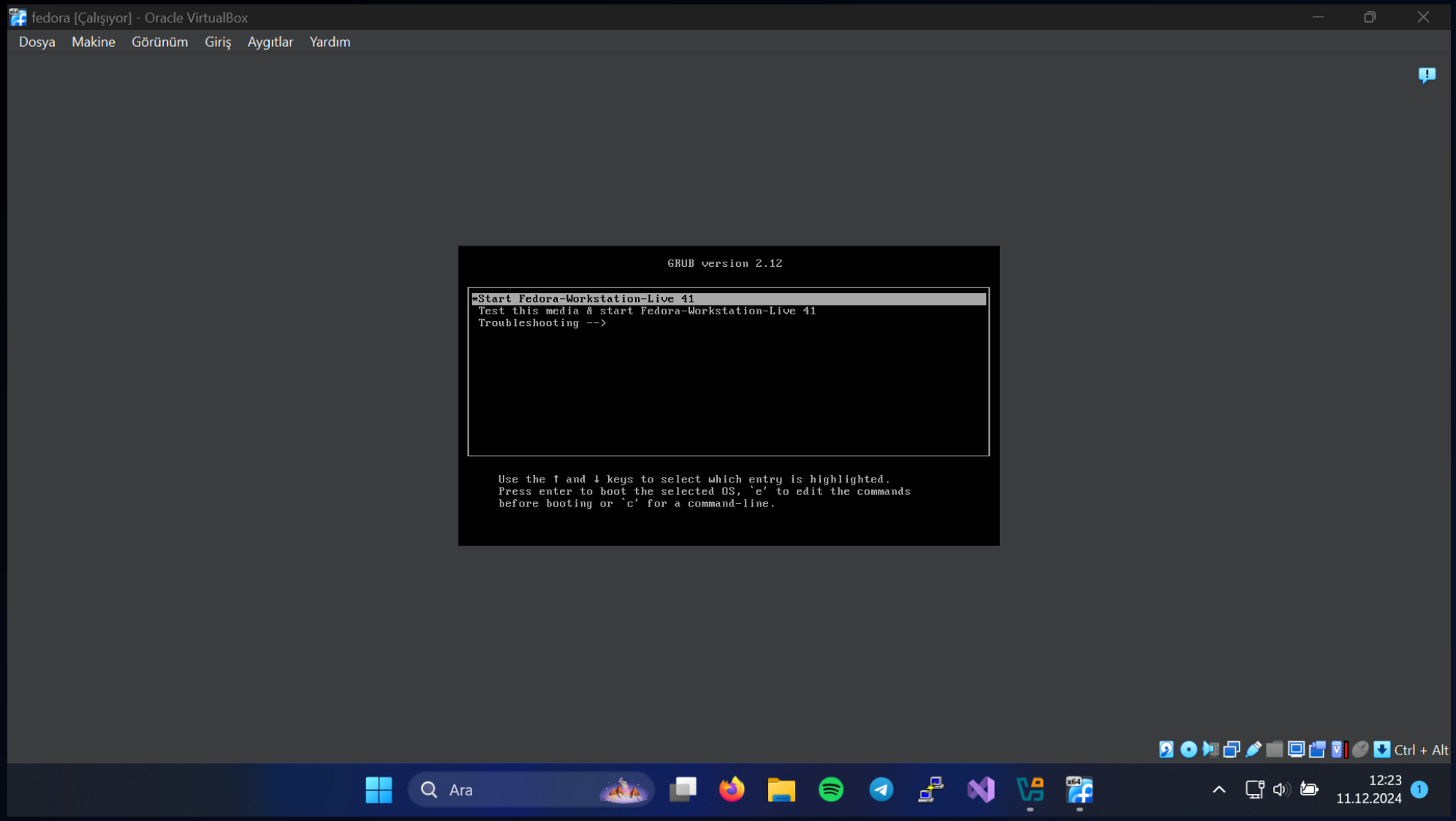
- İlk aşamada sanal makinemize ad verelim.
- Fedora yazdığımızda otomatik olarak Fedora kurulumu yapacağımızı varsayarak işletim sistemini seçecektir.
- ISO kalıbı kımsından ise indirdiğimiz Fedora ISO dosyasını seçiyoruz.
- Bir sonraki sayfada ise donanım detayları yer alıyor.



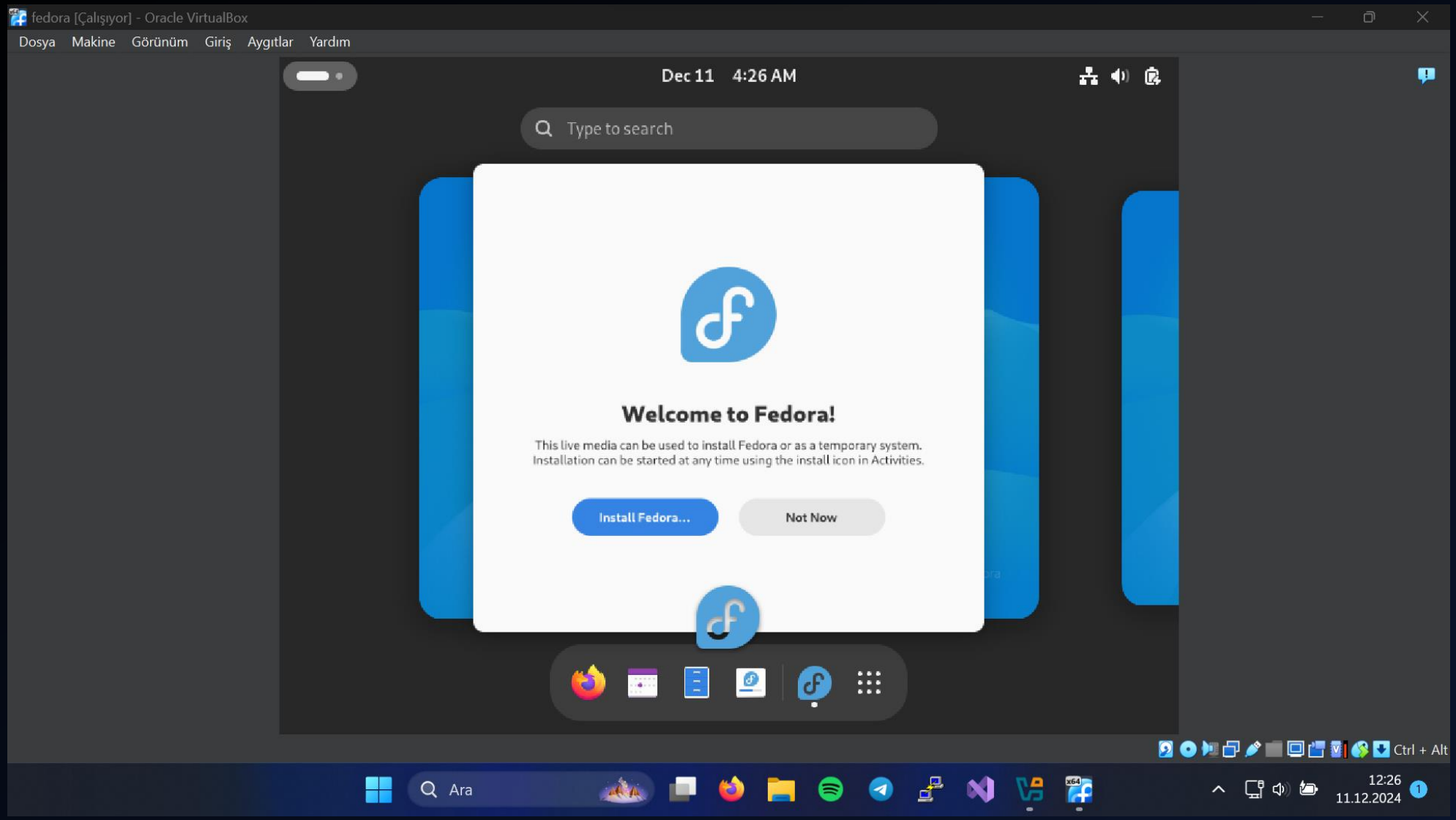
- Eklemek istediğiniz ana bellek, disk ve işlemci sayısını isteğinize ve iş planlarınıza göre ayarlayabilirsiniz.
- Ben kendim için 8 GB RAM, 6 işlemci ve 50 GB disk ayırdım.
- Diğer ayarları kendi isteğinize göre keşfedebilirsiniz.
- Bitir diyerek ilk sanal makinemizi oluşturalım.



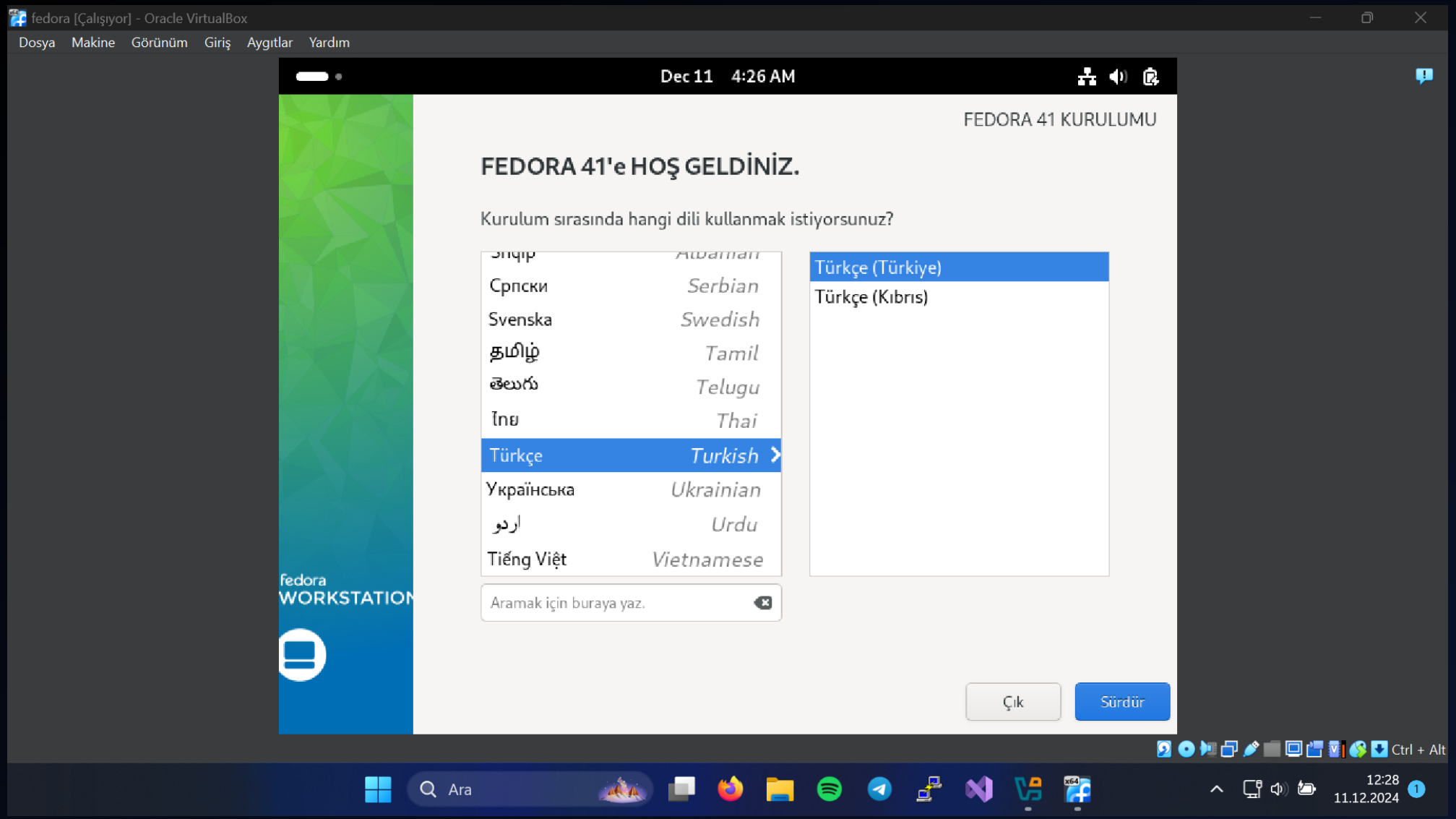
- Yönetim ekranına geri yönlendirdi ve sanal makinemizin yönetim ekranıyla karşı karşıyayız.
- Başlat tuşuna basarak ilk çalıştırmamızı yapalım.



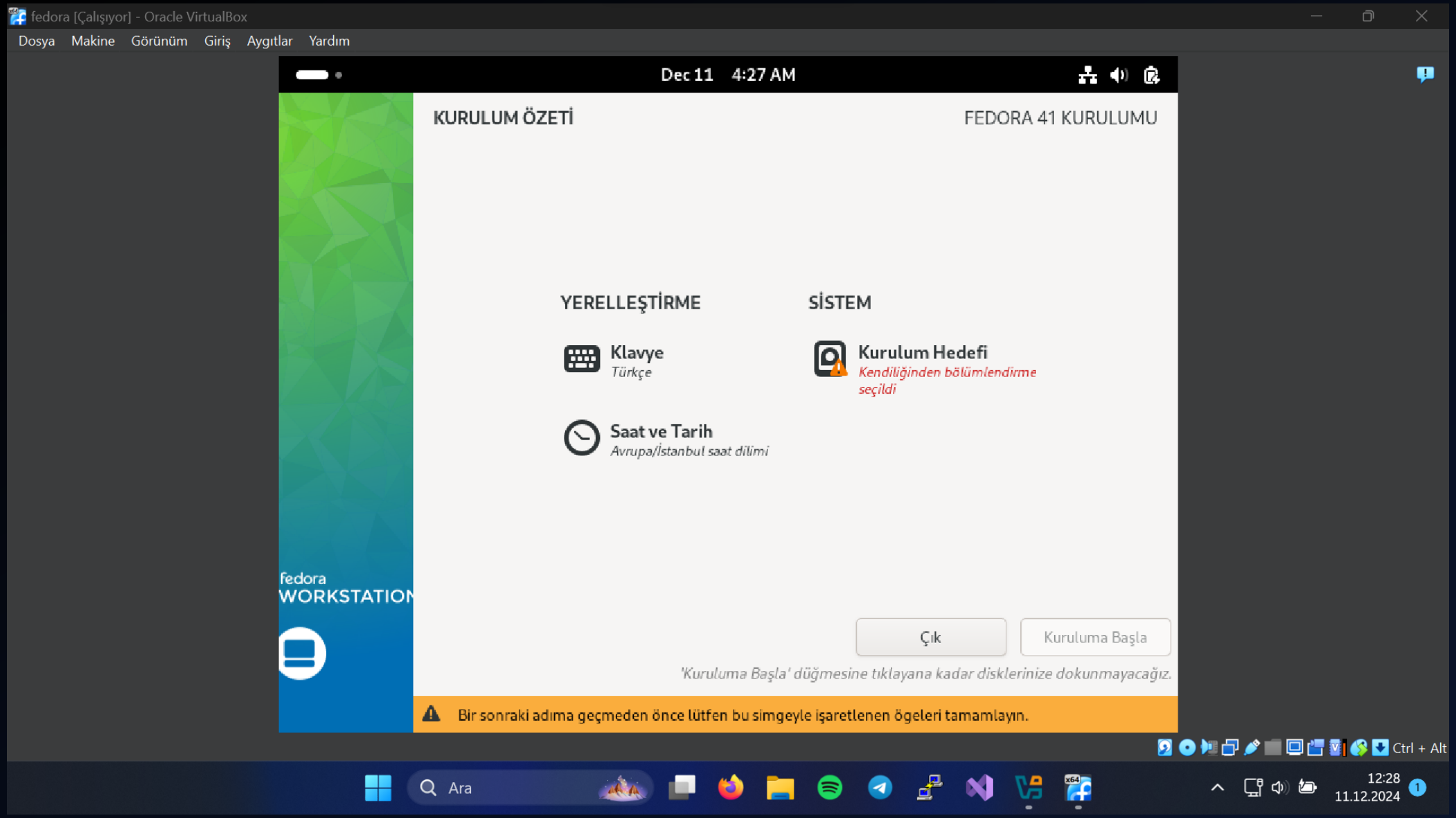
- Fedora işletim sisteminin kurulum seçenek ekranı bizi karşıladı. İlk seçeneğin üstüne ok tuşlarıyla gelerek Enter tuşuna basalım ve Feodra işletim sistmeinin yüklenmesini bekleyelim.



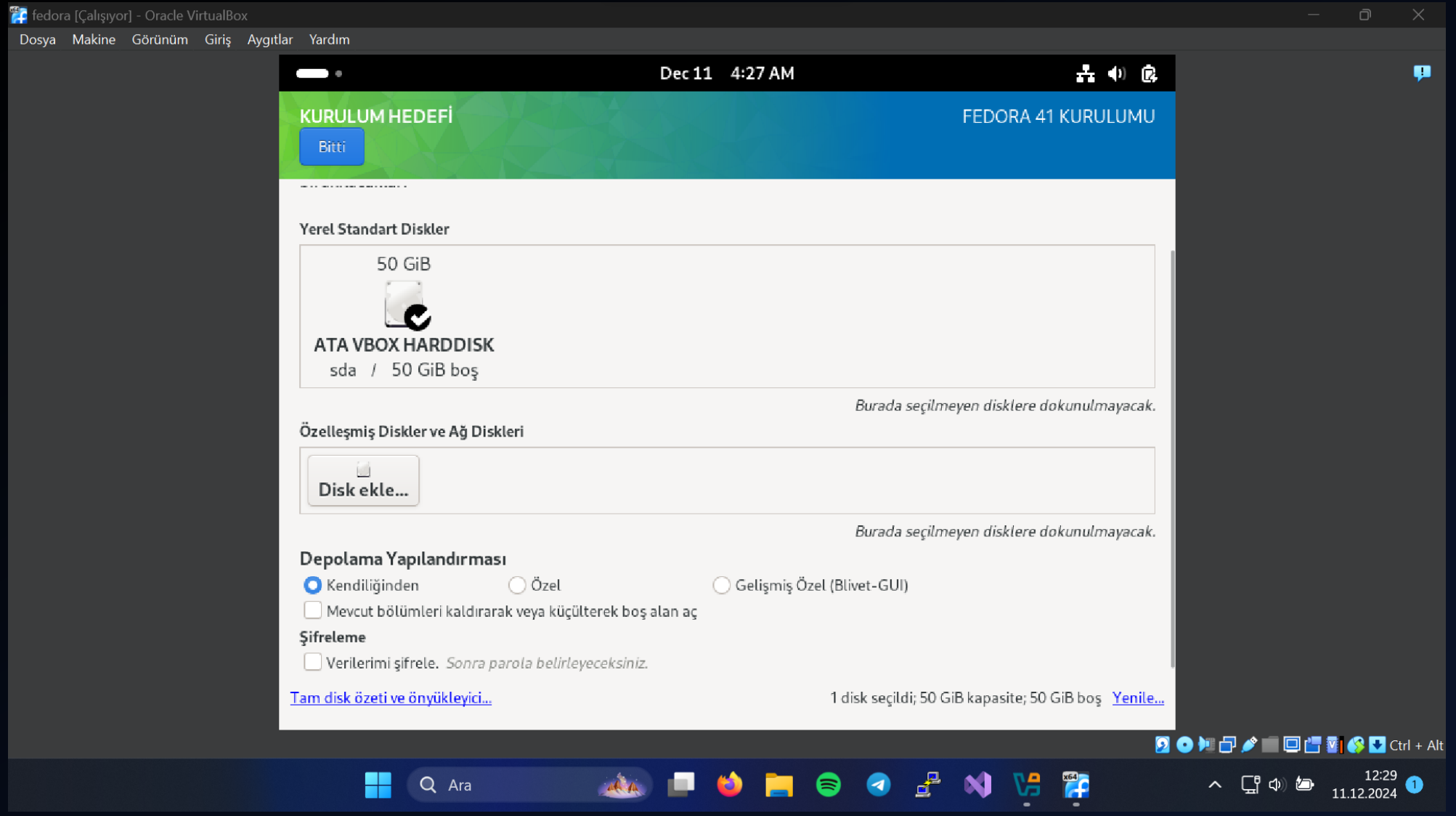
- Fedora’nın masaüstü ekranı bizi karşıladı. İlk gelen pencereyi seçelim ve sonrasında «Install Fedora…» tuşuna basarak kuruluma geçelim.



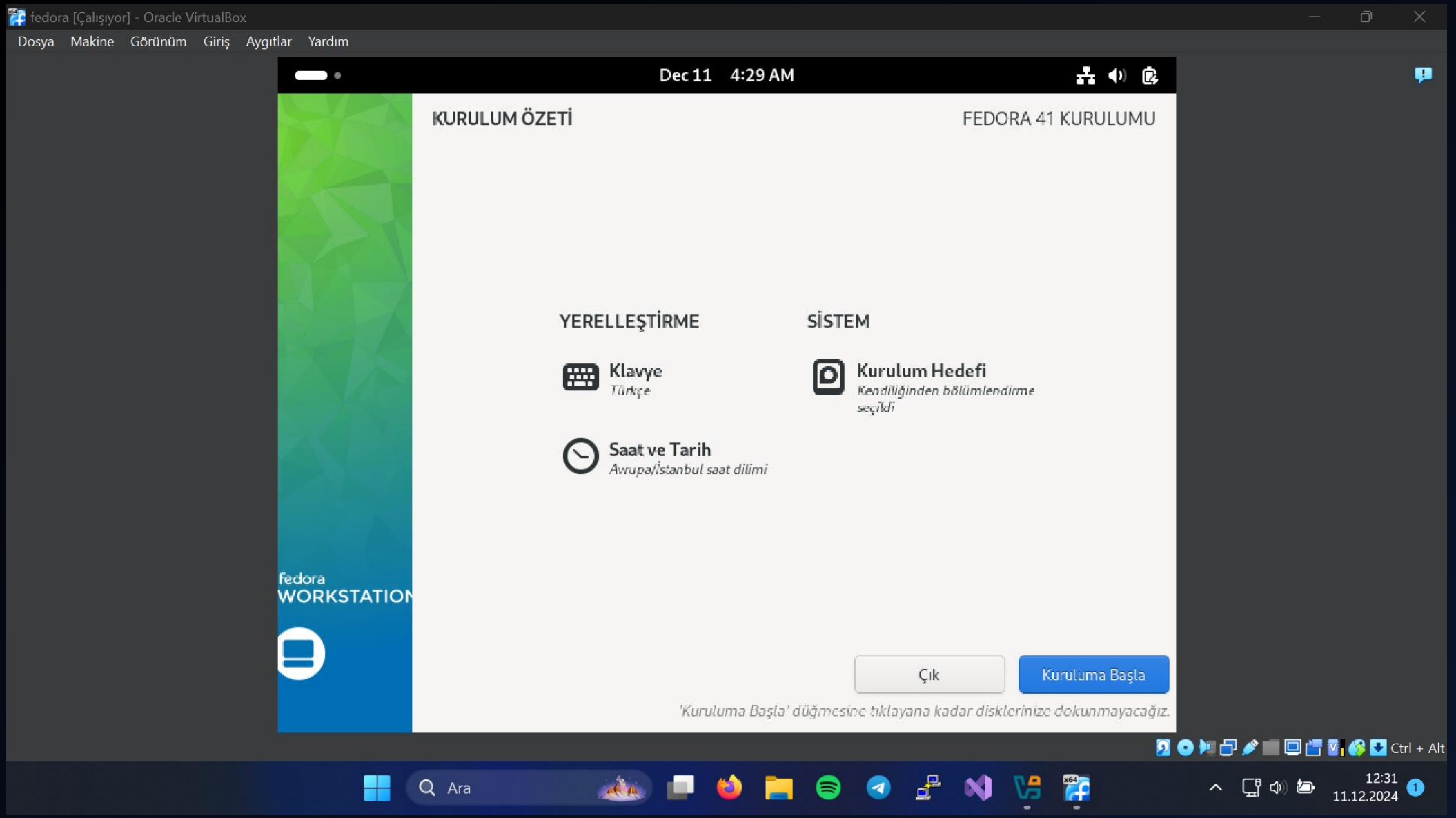
- İlk kurulum ekranında kurulum yapacğaımız dili seçerek devam edelim. Geliştirme programlarında Türkçe dilinde problemler yaşanabileceğinden İngilizce kurulmanız tavsiye edilir.



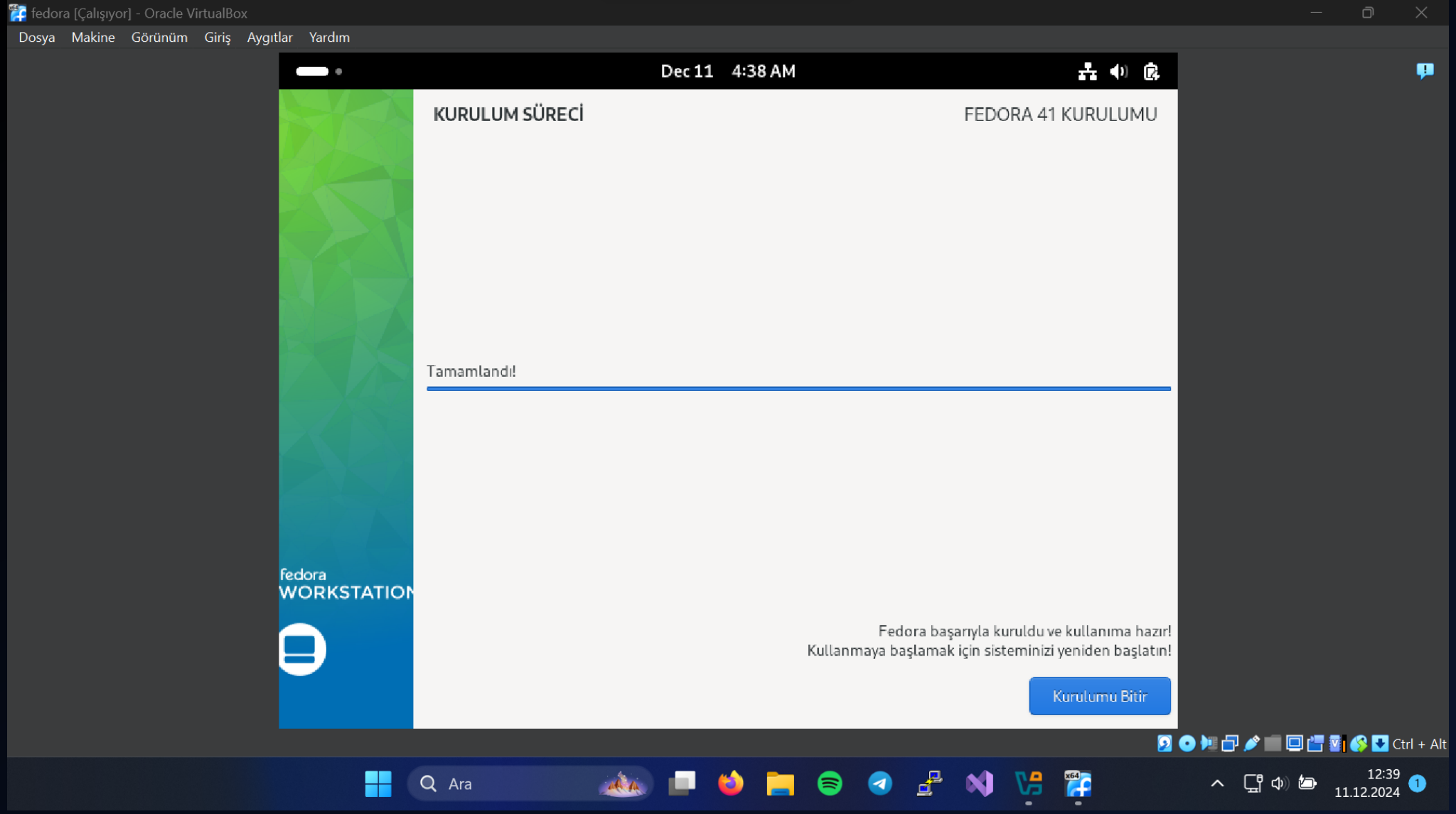
- Kurulum ekranının da altta verdiği uyarı üzere gerekli ayarlamalarımızı yapalım.



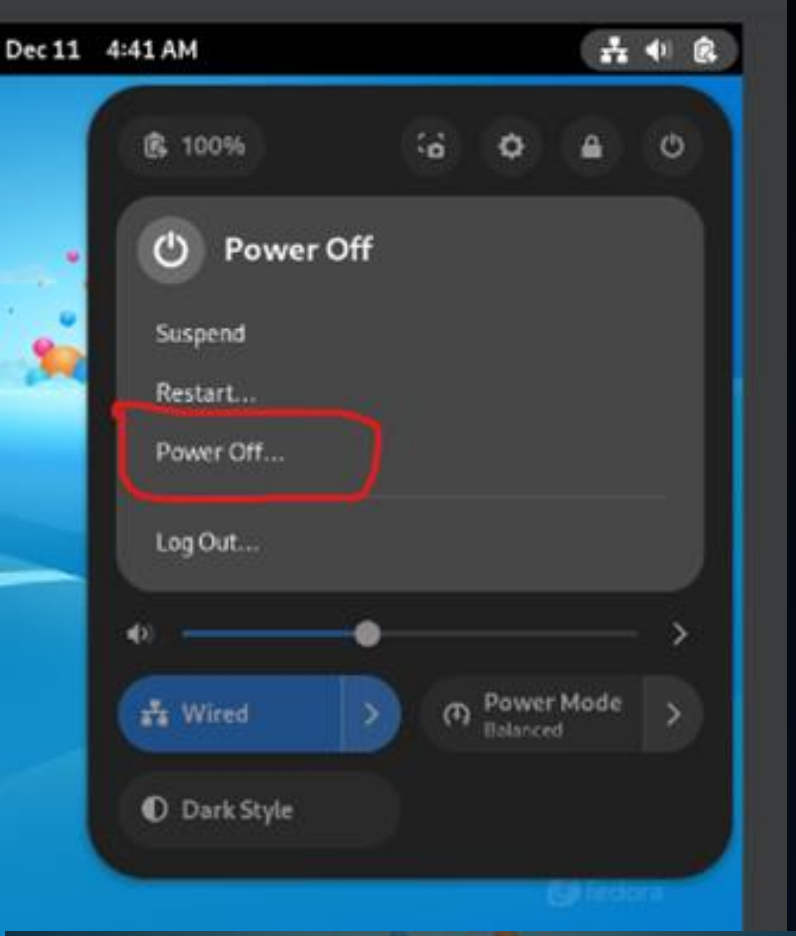
- Diskimizi seçtiğimizde üstünde siyah tik belirecek. Depolama yapılandırmasını «Kendiliğinden» seçeneğinde tutarak sol üstteki «Bitti» tuşuna basarak diskimizin seçimini gerçekleştirelim.

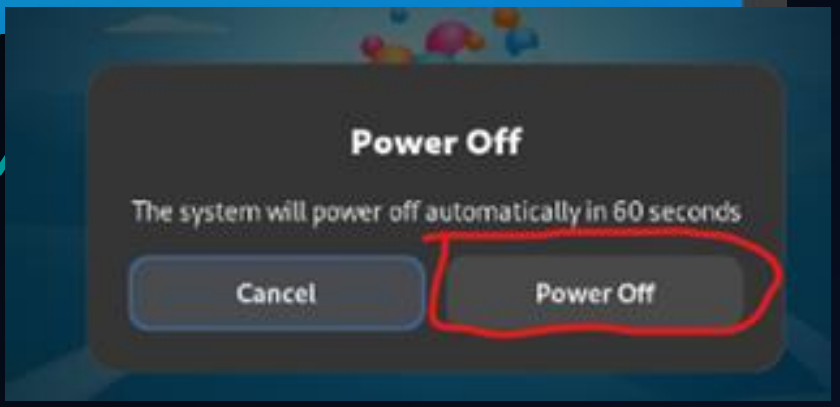


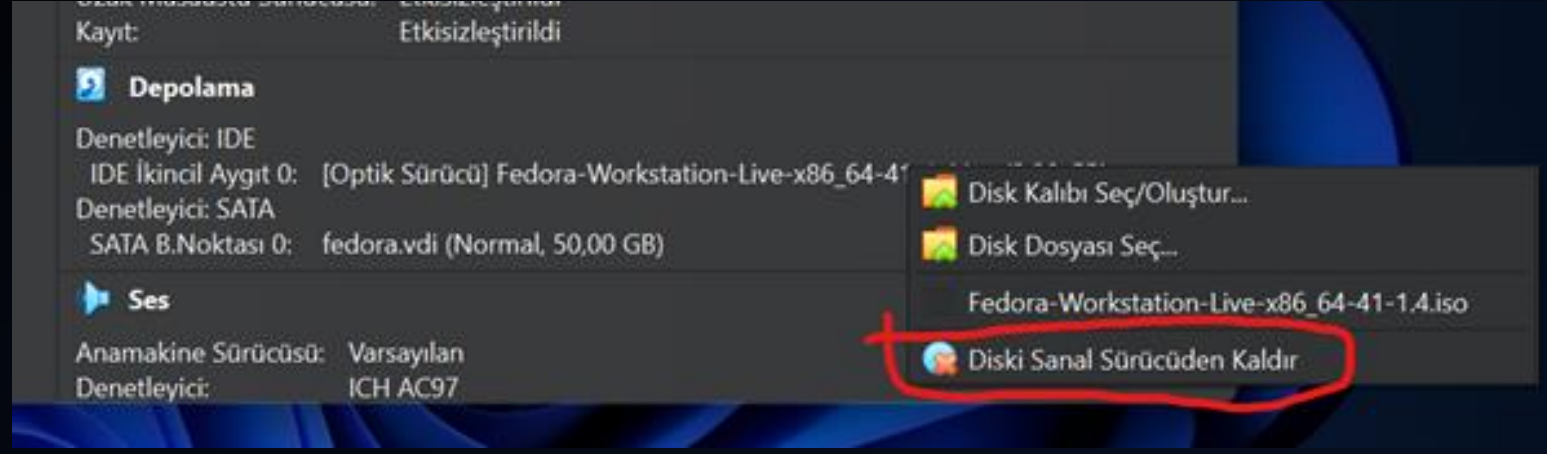
- Artık «Kurulumu Başlat» diyerek diskimize kurulmasnı bekleyeibliriz. Bu 10 ila 20 dakika arası sürecektir.



- Kurulumu bitir tuşuna bastıktan sonra sağ üstteki menüden gücü kapat diyerek devam edelim çünkü VirtualBox üzerinde bir ayar yapmamız gerek.







Sol üst fotoğraftaki gibi gücü kapatıp yönetim ekranına döndükten sonra sağ üstteki fotoğrafta [Optik Sürücü] yazan kısma tıklayalım ve açılan pop-up menüde Diski Sanal Sürücüden Kaldır tuşuna basalım ve tekrar sanal makineyi başlatalım. Artık Fedora sanal makinemiz kullanıma hazır.

Linux Çekirdeği Nedir?

- Linux çekirdeği, işletim sisteminin merkezinde bulunan ve donanım ile yazılım arasındaki etkileşimi sağlayan en temel bileşendir.
- Çekirdek, bilgisayarın donanım kaynaklarını (CPU, bellek, depolama, aygıtlar) yöneten bir yazılımdır ve uygulamaların bu kaynaklara güvenli ve düzenli bir şekilde erişimini sağlar.
- Bilgisayar üzerinde çalışan her türlü uygulama, çekirdeğin sağladığı hizmetlerle donanımı kullanır, böylece uygulamaların donanımla doğrudan iletişimi engellenir ve güvenlik sağlanır.
- Linux çekirdeği ilk kez 1991'de Linus Torvalds tarafından geliştirildi ve o zamandan beri sürekli geliştirilmekte ve geniş bir topluluk tarafından desteklenmektedir.
- Açık kaynaklı olması, dünyanın dört bir yanındaki geliştiricilerin katkıda bulunabilmesine olanak tanır.

Çekirdeğin Temel Görevleri

- Bellek Yönetimi: Çekirdek, RAM'in etkin kullanımını sağlar, uygulamalar arasında bellek paylaşımını yönetir ve her uygulamanın yalnızca kendi bellek alanına erişmesini garanti eder. Bu sayede sistemin kararlılığı ve güvenliği artırılır.
- Süreç Yönetimi: Çekirdek, işlemcide çalışan süreçlerin (process) oluşturulmasını, yürütülmesini ve sonlandırılmasını yönetir. Ayrıca her sürecin ne kadar CPU zamanı alacağını belirleyen bir zamanlama mekanizması (scheduler) kullanır.
- Dosya Sistemi Yönetimi: Çekirdek, dosyaların sabit disk gibi depolama aygıtlarına nasıl kaydedileceğini ve alınacağını yönetir. Linux'ta her şeyin bir dosya olarak temsil edilmesi, donanım aygıtlarına erişimi bile bir dosya işlemi gibi ele alır.
- Aygıt Sürücü Yönetimi: Donanım aygıtlarının (örn. klavye, fare, ağ kartı) çekirdek tarafından tanınmasını ve kullanılmasını sağlayan yazılım bileşenleridir. Aygıt sürücüleri donanım ile çekirdeğin iletişimini sağlar.
- Ağ Yönetimi: Linux çekirdeği, ağ trafiğinin yönetimi, paketlerin yönlendirilmesi ve güvenlik duvarları aracılığıyla ağ güvenliğinin sağlanması gibi işlevleri üstlenir.

Linux Çekirdeğinde Modülerlik

- Linux çekirdeği modüler bir yapıya sahiptir, yani çekirdeğe ihtiyaç duyulduğunda belirli işlevleri yerine getiren modülller eklenebilir veya çıkarılabilir.
- Modüller, örneğin bir donanım aygıtı sürücüsü gibi çekirdeğe yüklenip çalıştırılabilen küçük yazılım bileşenleridir.
- Bu modüler yapı, sistemin donanım sürücülerini veya belirli özelliklerini dinamik olarak yönetmesine olanak tanır. Örneğin, bir aygıt sürücüsü gerektiğinde insmod komutu ile yüklenebilir, artık gerekmediğinde rmmod komutu ile kaldırılabilir.
- Modüllerin yüklenmesi, sistemin yeniden başlatılmasına gerek kalmadan işlevselliğin artırılmasını sağlar.
- Ayrıca modüller gerektiğinde otomatik olarak yüklenip kaldırılabilir, böylece sistemde gereksiz yüklenmiş modüller bulunmaz.

Çekirdek Modüllerinin İşleyişi

- Çekirdek modülleri, sistemde çalışırken bağımsız olarak çalıştırılabilen çekirdek bileşenleridir.
- Bunlar, özellikle donanım sürücüleri ve belirli dosya sistemi işlevleri için kullanılır.
- Modüller, çekirdek yapılandırmasının daha esnek olmasını sağlar ve bu modüller ihtiyaç duyulduğunda çekirdeğe eklenip çıkarılabilir.
- lsmod komutu ile mevcut çekirdek modüllerini listeleyebilir, modprobe komutu ile bir modülü otomatik olarak ekleyebilirsiniz.
- Bu modüler yapı, çekirdeğin gereksiz parçalarla büyümesini engeller ve sistemin gereksiz yere kaynak tüketmesini önler.

Bellek Yönetimi ve Sanal Bellek

- Linux çekirdeği, belleği verimli bir şekilde yönetmek için sanal bellek mekanizmasını kullanır. Bu sayede uygulamalar fiziksel belleği aşmadan daha büyük bellek alanları kullanabilir.
- Sanal bellek, bellekte yeterli yer kalmadığında disk üzerinde ayrılan bir alanı kullanarak uygulamaların çalışmaya devam etmesini sağlar. Bu disk üzerindeki alan swap alanı olarak bilinir.
- Bellek yönetiminde, her uygulamanın belleği izole edilir ve uygulamaların birbirinin belleğine erişmesi engellenir.
- Linux, bellek yönetimi sırasında sayfalama (paging) ve segmentasyon gibi teknikler kullanır. Sayfalama, bellek parçalarının küçük bölümlere ayrılarak yönetilmesini sağlar.

Süreç Yönetimi ve Zamanlayıcı (Scheduler)

- Linux’ta süreç (process), çalışan bir programın bir örneğidir. Her süreç bir PID (Process ID) ile tanımlanır ve çekirdek bu süreçlerin takibini yapar.
- Çekirdek, aynı anda birçok süreci çalıştırır ve hangi süreçlerin CPU’yu ne kadar süre kullanacağını belirleyen bir zamanlayıcı kullanır.
- Linux çekirdeğinde farklı zamanlayıcı algoritmalar bulunur. Örneğin, Completely Fair Scheduler (CFS), her sürece adil bir şekilde işlemci süresi verilmesini sağlar.
- Ayrıca çekirdek, süreçlerin önceliğine göre kaynakları yönetir. Yüksek öncelikli süreçler daha fazla CPU zamanı alırken düşük öncelikli süreçler arka planda çalıştırılır.
- Bir uygulama yeni bir süreç başlattığında, çekirdek bunu izler ve her sürecin ne kadar kaynak tükettiğini yönetir.

Dosya Sistemi Yönetimi

- Linux, farklı dosya sistemlerini destekler (örn. EXT4, FAT32, NTFS). Çekirdek, dosya sistemlerinin nasıl organize edileceğini ve dosyaların depolanacağını yönetir.
- Linux'ta her şey bir dosya olarak ele alınır; bu, cihaz dosyaları (donanım bileşenleri), dizinler, soketler ve hatta bellek bileşenleri için geçerlidir.
- Bir dosyanın açılması, okunması veya yazılması gibi işlemler, çekirdeğe yapılan sistem çağrıları ile gerçekleştirilir. Örneğin, open() dosya açma, read() okuma ve write() yazma için kullanılır.
- Dosya sistemleri, çekirdeğin kullanıcı programları ile disk üzerindeki veriler arasında bir köprü görevi görür. Bu köprü, verilerin güvenli ve verimli bir şekilde yönetilmesini sağlar.
- Linux ayrıca sanal dosya sistemi (VFS) adı verilen bir katman sunar. Bu katman, çekirdeğin farklı dosya sistemleri ile etkileşime geçmesini ve onları tek bir arabirimden yönetmesini sağlar.

Linux Çekirdeğinin Önemi

- Linux çekirdeği, donanım ile yazılım arasındaki temel köprüyü kurarak sistemin tüm işleyişini sağlar.
- Güçlü bellek yönetimi, süreç yönetimi, ağ yönetimi ve dosya sistemi yönetimi sayesinde yüksek performans ve güvenlik sağlar.
- Modüler yapısı, sistemin genişletilebilir olmasını sağlar ve yeni donanım desteği, sistemin yeniden başlatılmasına gerek kalmadan eklenebilir.
- Açık kaynaklı bir proje olarak Linux çekirdeği, küresel bir topluluk tarafından sürekli geliştirilmektedir ve dünya genelinde sunuculardan gömülü sistemlere kadar geniş bir yelpazede kullanılmaktadır.
- Sonuç olarak, Linux çekirdeği, esnekliği, güvenilirliği ve geniş topluluk desteği ile modern bilişim dünyasında kritik bir rol oynamaktadır.

Kabuk (Shell) Nedir?

- Kabuk (Shell), kullanıcı ile işletim sistemi arasında bir arayüz işlevi gören bir programdır. Komutları alır, bu komutları işletim sistemine iletir ve sonuçları kullanıcıya döner. Kabuk, kullanıcının donanım kaynaklarını ve sistem yazılımlarını doğrudan kontrol etmesini sağlar.

Kabukların Görevleri

- Kullanıcı komutlarını yorumlar.
- Program başlatır ve yönetir.
- Dosya sistemine erişim sağlar.
- Sistem kaynaklarının durumunu raporlar (örn. bellek kullanımı, işlem durumu).

Kabuk Türleri

- Verimlilik: CLI ortamında işlemler grafik arayüzlerden daha hızlı yapılabilir.
- Güçlü Kontrol: Tüm sistem kaynaklarına erişim ve detaylı kontrol imkânı sağlar.
- Otomasyon: Karmaşık işlemler ve iş akışları, script yazarak kolayca otomatikleştirilebilir.
- Esneklik ve Özelleştirme: Kullanıcılar kendi alias'larını (takma adlar) ve komutlarını tanımlayabilir.

Kabuk Nasıl Çalışır?

- Kullanıcı, kabuğa bir komut girer.
  - Örnek: ls –l
- Kabuk, bu komutu yorumlar ve işletim sistemine iletir.
- İşletim sistemi, komutun gerektirdiği işlemi yapar (örneğin, dosyaları listeler).
- Kabuk, işlem sonucunu ekrana döker.

Kabuk ve İşlem Yönetimi

- Kabuk, her komut için bir işlem (process) başlatır.
- Her işlem, bir PID (Process ID) ile tanımlanır.
- Kabuğun bu işlemleri yönetmesi ve gerektiğinde sonlandırması mümkündür.

Temel Kabuk Komutları

Bu komutlar, Linux ve Unix sistemlerinde sıkça kullanılır:

- **ls:** Bulunduğunuz dizindeki dosya ve dizinleri listeler.
- **cd:** Başka bir dizine geçiş yapar.
- **mkdir:** Yeni bir dizin oluşturur.
- **rm:** Dosya veya dizinleri siler.
- **cp:** Dosya veya dizin kopyalar.
- **mv:** Dosya veya dizin taşır veya yeniden adlandırır.
- **cat:** Dosya içeriğini görüntüler.
- **echo:** Ekrana metin yazar, genellikle scriptlerde kullanılır.

Örnek bir Shell kullanımı

- mkdir proje # proje adında dizin oluşturur
- cd proje # proje dizinine girer
- echo "Merhaba Dünya!" > merhaba.txt
  # merhaba.txt adında bir dosya içine Merhaba Dünya! girdisini yazdırır
- cat merhaba.txt
  # merhaba.txt dosyasının içeriğini terminale yazdırır

```
bfljpg@vbox:~$ mkdir proje
bfljpg@vbox:~$ cd proje/
bfljpg@vbox:~/proje$ echo "Merhaba Dünya!" >> merhaba.txt
bfljpg@vbox:~/proje$ cat merhaba.txt
Merhaba Dünya!
bfljpg@vbox:~/proje$
```

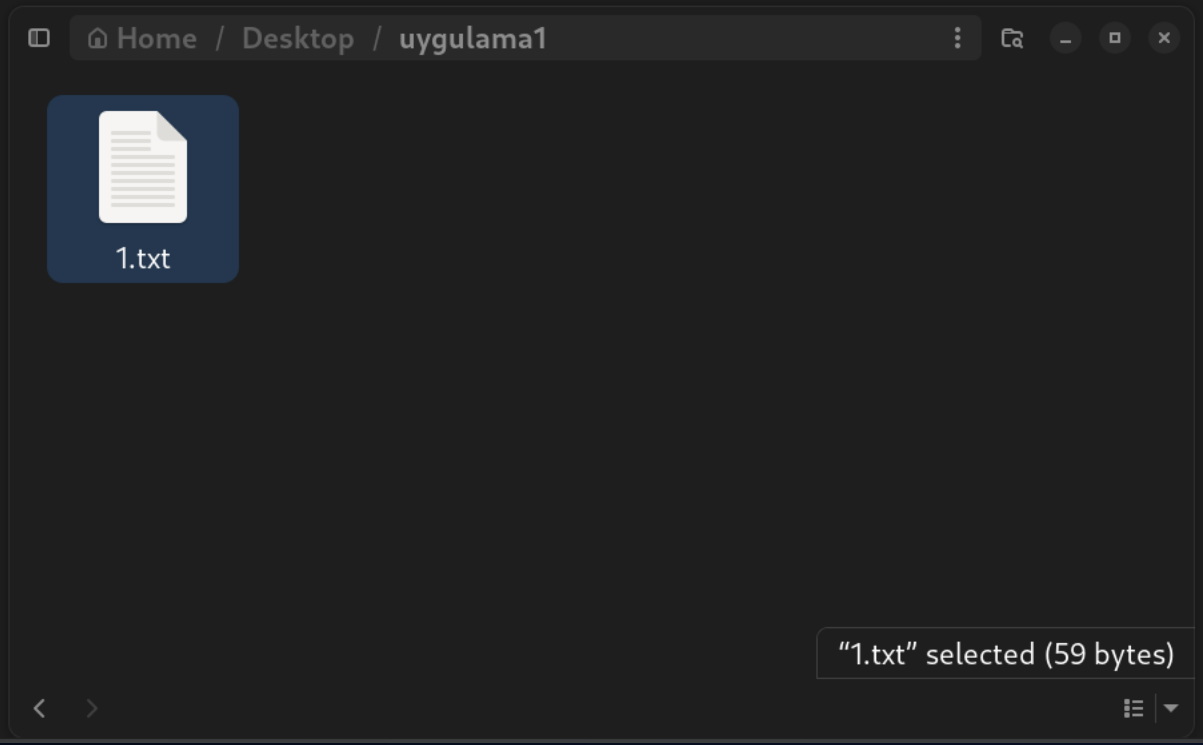
- Her komut satırı başarıyla çalıştırıldığında buna benzer bir görüntü elde etmeniz gerekir.
- İlk satırda proje adında bir dizin açtık, ikinci satırda bu dizinimizde çalışmaya başladık.
- Üçüncü satırda tırnak içinde yazdığımız içeriğimizi merhaba.txt dosyasına kaydettik ve dördüncü komut satırında dosyamızın içeriğini terminale başarıyla yazdırdık.

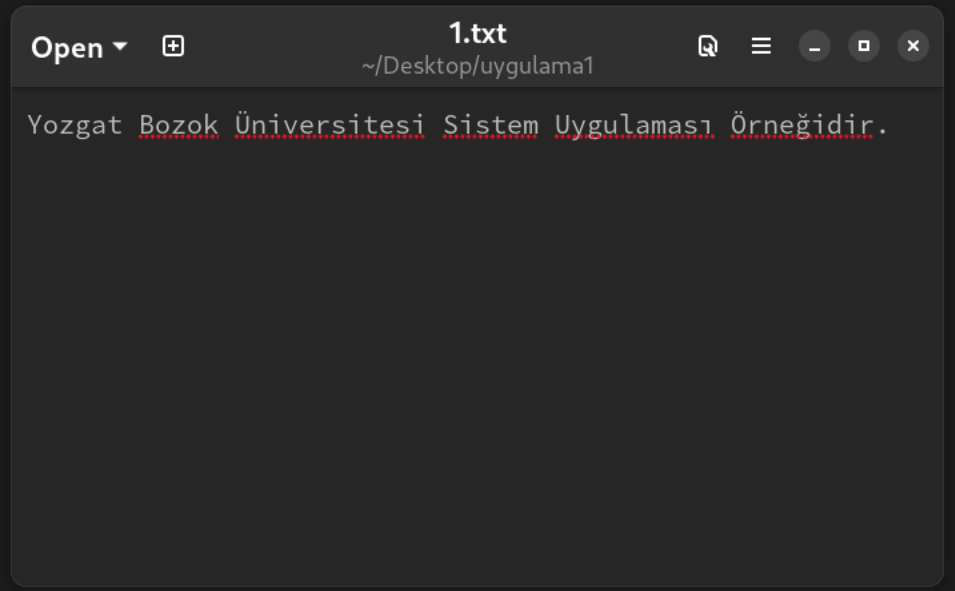
Örnek 2:

- cd Desktop // Desktop dizinine girer
- mkdir uygulama1 // Desktop dizininde uygulama1 dizini açar
- cd uygulama1 // uygulama1 dizinine girer
- echo "Yozgat Bozok Üniversitesi Sistem Uygulaması Örneğidir." >> 1.txt
  // tırnak içindeki yazıyı 1.txt dosyasına yazdır

bfljpg@vbox:~/Desktop/uygulama1

```
bfljpg@vbox:~$ cd Desktop
bfljpg@vbox:~/Desktop$ mkdir uygulama1
bfljpg@vbox:~/Desktop$ cd uygulama1
bfljpg@vbox:~/Desktop/uygulama1$ echo "Yozgat Bozok Üniversitesi Sistem Uygulaması Örneğidir." >> 1.txt
bfljpg@vbox:~/Desktop/uygulama1$ cat 1.txt
Yozgat Bozok Üniversitesi Sistem Uygulaması Örneğidir.
bfljpg@vbox:~/Desktop/uygulama1$
```





Kabuk kullanarak Uygulama Kurulumu

- Bu derste kullanacağımız temel bileşenlerden birisi olan gcc programını kurmanız için temel komutumuz şu şekildedir:
  - «sudo dnf install gcc»
- Bunu başka programları kurarken de deneyebilirsiniz. Örneğin; neofetch, lshw, tar vs.
  - «sudo dnf install neofetch»
  - «sudo dnf install lshw»

```
bfljpg@vbox:~$ sudo dnf install gcc
Copr repo for PyCharm owned by phracek          2.1 kB/s | 2.9 kB     00:01
Fedora 40 openh264 (From Cisco) - x86_64        1.9 kB/s | 989  B     00:00
Fedora 40 - x86_64 - Updates                     26 kB/s |  24 kB     00:00
Fedora 40 - x86_64 - Updates                    973 kB/s | 4.9 MB     00:05
google-chrome                                   4.4 kB/s | 1.9 kB     00:00
RPM Fusion for Fedora 40 - Nonfree - NVIDIA Dri 3.9 kB/s | 5.2 kB     00:01
RPM Fusion for Fedora 40 - Nonfree - Steam      963  B/s | 1.5 kB     00:01
Dependencies resolved.
================================================================================
 Package       Architecture     Version                 Repository         Size
================================================================================
Installing:
 gcc           x86_64           14.2.1-3.fc40           updates            37 M

Transaction Summary
================================================================================
Install  1 Package

Total download size: 37 M
Installed size: 104 M
Is this ok [y/N]: y
```

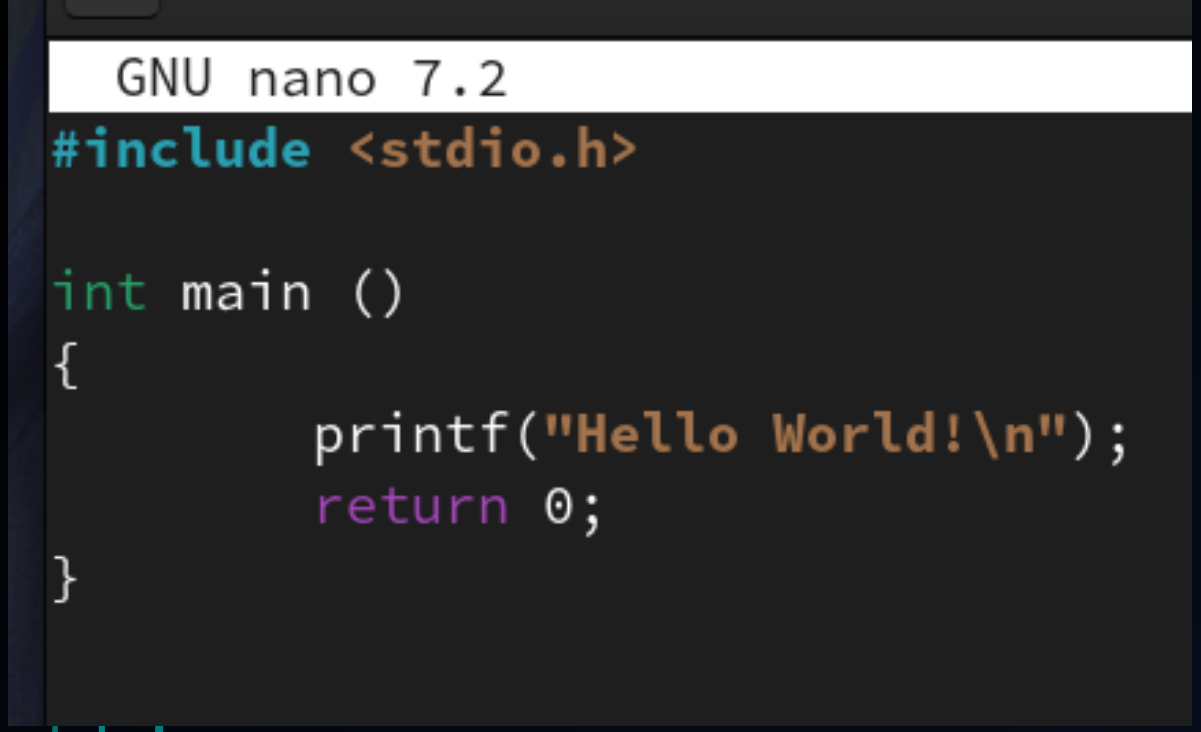
```
Installed:
  gcc-14.2.1-3.fc40.x86_64

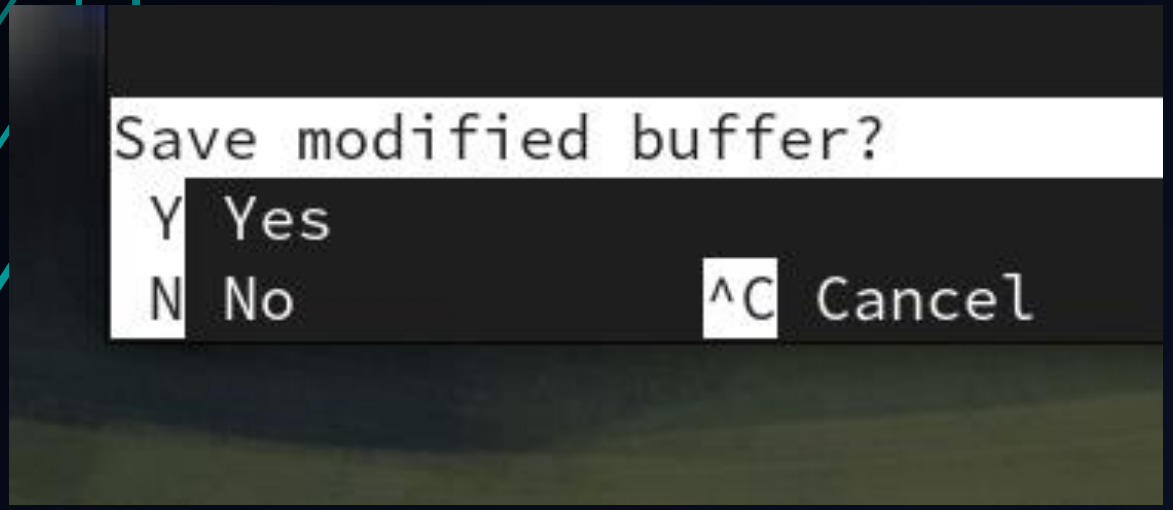
Complete!
bfljpg@vbox:~$
```

- Komutu gösterildiği gibi çalıştırdığımızda bize «Kurulsun mu?» tarzında bir onay sorusu sormaktadır.
- «y» yazıp enter a basarak kurulumu onaylıyoruz.
- İkinci resimde gösterildiği gibi gcc programımız başarıyla kuruldu.
- gcc programının temel kullanımı bir sonraki slaytta anlatılıyor olacak.

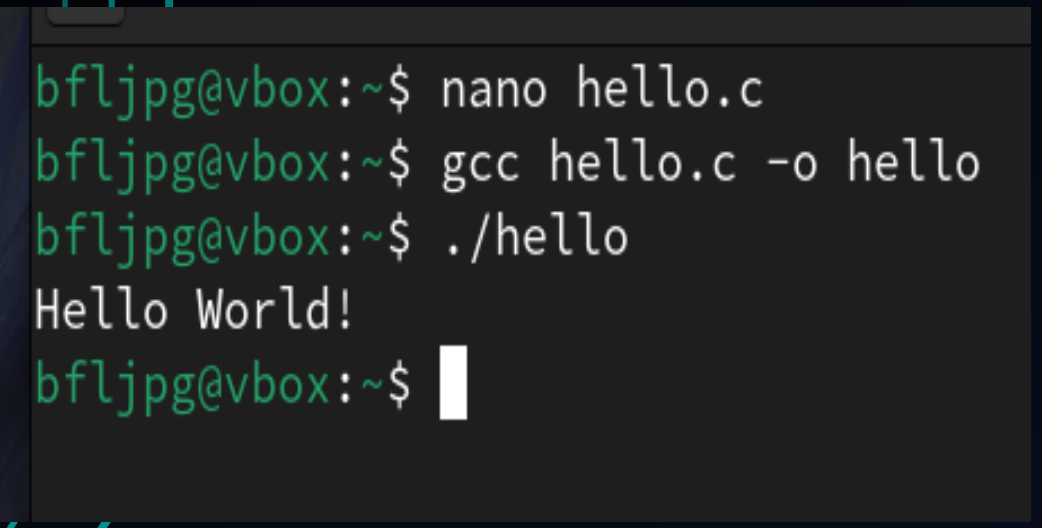
Örnek: gcc Programının Temel Kullanımı

- gcc programı (GNU C Compiler adının baş harflerinden oluşuyor) temel bir C derleyicisidir.
- Temel kullanımını öğrenmek için basit bir şekilde «Hello World!» çıktısı veren basit bir program yazdırıp gcc ile derleyelim.
- Programı yazdıktan sonra «gcc hello.c –o hello» yazarak derleyin. Bu komutta hello.c kodumuzun adı veya adresini belirtir. «-o hello» optionı ile derlenmiş uygulamamızın adını veriyoruz. Boş bırakırsanız uygulamanızı «a.out» adında derleyecektir.
- Daha sonra «./hello» yazarak uygulamamızı çalıştıracağız. Burada hello «-o» ile ayarladığımız derlenmiş uygulamamızın adıdır.





- İlk resimde görüldüğü üzere örnek «Hello World» örneğimiz bulunuyor.
- «nano hello.c» diyerek dosyamızı açıyoruz. Daha sonra örnek kodumuzu yazıp Ctrl+X tuşlarına basarak kaydediyoruz.
- Ctrl+X tuşlarına bastığımızda bizden «düzenlediğiniz dosyayı kaydetmek ister misiniz?» sorusuna cevap bekliyor. «Y» yi tuşlayarak ve enter tuşuna basarak onaylıyoruz.



- «gcc hello.c –o hello» komutuyla derlemeyi başlatıyoruz.
- Derleme başarıyla tamamlandığında hiçbir mesaj yazdırmadan alt satıra geçecektir. Eğer hata verirse hatalarınızı satır numaralarıyla beraber kontrol edin.
- «./hello» komutunu kullanarak uygulamamızı çalıştırıyoruz. Görüldüğü üzere başarıyla derlememizi almış ve «Hello World!» mesajını almış bulunuyoruz.

Kabuk ile Gelişmiş Sistem Yönetimi

- Sistem izleme ve yönetimi: CPU, bellek, disk kullanımı gibi bilgileri anında görüntüleyebiliriz.
  - Örnek: top, htop, df, du.
- Ağ yönetimi: Ağ arayüzlerini, trafiği ve bağlantıları yönetebiliriz.
  - Örnek: ifconfig, ping, netstat, traceroute.

Dosya Sistemi

- Genel olarak UNIX tabanlı sistemler için şöyle bir özdeyiş vardır: UNIX sistemlerinde her şey birer dosyadır; eğer bir şey dosya değilse o zaman process'dir.
- Bu yaklaşım Linux için de geçerlidir. Bazı dosyaların diğerlerinden daha özel anlamlarının da olabilmesiyle birlikte (aygıt dosyaları, domain soketler, pipe, link vb.), pek çok işlem dosya sistematiği içerisinde kalınarak gerçekleşmektedir.
- Dosya sistemi arayüzüne verilen önem, proc yalancı dosya sistemi gibi (pseudo file system) örneklerde de kendini gösterir. Bu dosya sistemini bir dizine bağlayıp (mount) kullanmaya başladığınızda içerisinde görülen dizin ve dosyalar, çalışan kernel içeriğinin bir yansımasından başka bir şey değildir.

Aygıt Dosyaları

- Unix sistemlerde fiziksel aygıtlara erişmek için aygıt dosyaları (device files) kullanılır. Aygıt dosyaları sayesinde çekirdek içerisinde hangi aygıt sürücüsünün kullanılacağı belirlenmiş olur.
- Bu yöntemle disk, seri port gibi fiziksel aygıtlara erişilebildiği gibi, /dev/random gibi fiziksel bir karşılığı olmayan, sanal aygıt dosyaları üzerinden erişimler de mümkündür.

- Aygıt dosyalarının ilişkili olduğu aygıt sürücüleri, çekirdek içerisindeki belirli bir kod kümesinden ibarettir. Örnek olarak SCSI veriyolu üzerindeki birinci disk üzerinde işlem yapmak için */dev/sda* dosyası kullanıldığında çekirdek içerisinde SCSI katmanıyla ilgili bir agıt sürücüsü devreye girerken, sisteme takılan ilk USB Seri Çevirici cihazına erişim için */dev/ttyUSB0* dosyası kullanıldığında USB - Seri çeviricilere ait sürücü kodları devreye girecektir. Peki acaba çekirdek içerisinde, erişilen aygıt dosyasının ismini metin karşılaştırma yöntemiyle kontrol edip, sda ise şu fonksiyonu çalıştır şeklinde bir tablo mu bulunmaktadır?

- Yukarıdaki sorumuza vereceğimiz yanıt hayır olacaktır. Esasen /dev dizini altındaki aygıt dosyalarının isminin ne olduğunun hiç bir önemi bulunmamaktadır. Aşağıdaki komutun çıktısını inceleyelim:

```
bfljpg@vbox:~$ ls -l /dev/sda
brw-rw----. 1 root disk 8, 0 Oct 14 12:16 /dev/sda
bfljpg@vbox:~$
```

- Yukarıdaki çıktıda normal bir dosyadan farklı olarak 3 farklı bilgi daha yer almaktadır. Bunlar:
  - Satırın ilk karakteri olan b, blok tabanlı bir aygıt dosyası olduğunu gösterir. Karakter tabanlı aygıt dosyaları için c kullanılır.
  - İlgili aygıta ait Major numarası olarak 8 rakamı görünmektedir.
  - İlgili aygıta ait Minor numarası olarak 0 rakamı görünmektedir.

- Özetle aygıt dosyaları üzerindeki bu ek bilgilerle, dosyanın karakter tabanlı mı yoksa blok tabanlı erişim gerektirdiği, çekirdek içerisinde kullanılacak Major ve Minor numaralarının neler olduğu bilgisi saklanır.
- Çekirdek açısından bu 3 bilginin aynı olduğu tüm aygıt dosyaları üzerinden kullanıcı kipinden yapılan her türlü erişim, çekirdek içerisinde aynı sürücü tarafından karşılanır. Yani aygıt tipi, major ve minor numarası /dev/sda ile aynı olacak şekilde /dev/birinci-scsi-disk, /etc/sda, /mnt/disk vb. herhangi isimlerde aygıt dosyaları üretsek de bu dosyalar üzerinden yapılacak erişimler hep aynı etkiyi üretir.
- Peki çekirdek için aygıt dosya isimlerinin bir anlam taşımadığını öğrendiğimize göre isimlendirme kurallarına hatta aygıt dosyalarına neden ihtiyaç vardır?

Kullanıcı, Grup ve Erişim Yetkileri

- Linux, çok kullanıcılı bir işletim sistemidir. Çalışan her uygulama mutlaka bir kullanıcı ve grup no ile ilişkilidir; kullanıcı veya grubu olmayan bir uygulamanın olması söz konusu değildir. Uygulama içerisinden yapılan tüm erişimler, bu kullanıcı ve grup hakları doğrultusunda gerçekleştirilir.
- Benzer şekilde dosya sistemi katmanında da her dosya ve dizin mutlaka bir kullanıcı ve grupla ilişkili durumdadır. Linux sanal dosya sistemi katmanı (Virtual Filesystem Switch) ile dosya sistemlerinin (ext4, xfs, btrfs vb.) alt seviye implementasyon detayları soyutlanmış ve ayrı bir katman oluşturulmuştur.
- Dosyaların kullanıcı, grup sahiplikleri ve erişim yetkileri, VFS katmanında tanımlanmış bir özelliktir. FAT32 gibi kendi içinde bu özelliklere sahip olmayan bir dosya sistemine Linux altında erişim yapılırken, kullanıcı, grup sahiplikleri ve erişim yetkileri öntanımlı değerlerle simüle edilir.

Okuma, Yazma ve Çalıştırma Yetkileri

- Dosya ve dizinler üzerindeki yetki kontrolleri temel olarak okuma (read - r), yazma (write - w) ve çalıştırma (execute - x) bitleri üzerinden kontrol edilir. Bu bilgiler dosya sistemi üzerinde inode yapılarında saklanmaktadır.
- Bu yetki bitlerinin dosya ve dizinler üzerindeki anlamları aşağıdaki gibidir:

Yetki	Açıklama
r	Dosyalar için okuma yetkisi, dosya içeriğine ulaşılabilmesi anlamını taşır. Dizinlerde olduğunda ise, dizin içeriğinin listelenebilmesini (dizinlerin okunması gibi düşünülebilir) sağlar.
w	Dosyalar için yazma yetkisi, dosya içeriğinin değiştirilebilmesini kontrol eder. Dizinler içinse, dizin içerisinde yeni dosya ve dizin oluşturma/silme operasyonlarını kontrol eder.
x	Dosyalar için çalıştırma yetkisi anlamı taşır. Dizinler için ilgili dizine geçilip geçilemeyeceğini belirtir.

SUID (Set User ID) Yetkisi

- SUID yetkisi dosyanın sahibiyle ilgili erişim yetkileri arasında yer alıp, SUID biti üzerinden kontrol edilir.
- Bir uygulamada SUID biti aktif ise, o uygulamayı hangi kullanıcı çalıştırırsa çalıştırsın, uygulama dosyasının sahibi kim ise, onun haklarıyla çalışır.
- En tipik örnek passwd uygulamasıdır. Bu uygulamayı kullanarak sistemdeki her kullanıcı kendi parolasını değiştirebilir. Ancak parola değişikliğinin yapılabilmesi için sadece root kullanıcısının yazma yetkisi olan /etc/shadow dosyasında değişiklik yapılması gereklidir.

- Sistemdeki /usr/bin/passwd dosyasının sahibi root kullanıcısı olmakla birlikte, biz herhangi bir normal kullanıcı olarak bu dosyayı çalıştırdığımızda, ilgili process kendi kullanıcı ID değerimizle ve bu değerin sistem ve dosya sistemi üzerindeki haklarıyla kısıtlı olarak faaliyet gösterecektir. Dolayısıyla parola değişimi için gereken /etc/shadow dosyasına yazma yetkisi olmadığından işlemi gerçekleştiremeyecek, hata alacaktır.
- Bu problemin çözümü için, bazı uygulamaların çalıştıran kullanıcının kim olduğundan bağımsız olarak sistemde belirli bir kullanıcının yetkileriyle (örneğimizde root) çalışabilmesi için bir olanak sunulması gereklidir. SUID biti bu noktada devreye girer ve /usr/bin/passwd uygulamasına eklenen SUID biti sayesinde, uygulama çalıştırıldığı andan itibaren uygulama dosyasının dosya sistemindeki sahibi olan root kullanıcısının haklarıya çalışır.

SGID (Set Group ID) Yetkisi

- SUID biti ile benzer mantıkta, bir uygulamanın, kimin çalıştırdığına bakılmaksızın uygulama dosyasının grup sahibinin grup erişim yetkileri doğrultusunda çalıştırılmasını sağlamaktadır. SGID biti, SUID bitine oranla pratikte daha az kullanım alanı bulmaktadır.

Sticky Bit

- Unix tabanlı işletim sistemlerinde /tmp dizini geçici dosya oluşturmak için tüm kullanıcıların ve uygulamaların kullanımına açılmıştır. Root kullanıcısının haklarıyla çalışan bir uygulama da, farklı kullanıcıların haklarıyla çalışan diğer uygulamalar da herhangi bir anda kısa veya uzun süreliğine geçici bir dosya oluşturma ihtiyacı duyabilir. Sistemde yazılabilir bir dizinin deneme/yanılma ile aranması makul bir süreç olmadığından, /tmp dizini bu işler için ayrılmıştır.

- Bu noktada karşımıza önemli bir problem çıkmaktadır: /tmp dizini tüm kullanıcılar tarafından yazılabilir durumda ise, A kullanıcısının oluşturduğu /tmp/test.txt dosyasına B kullanıcısı tarafından yazılması veya dosyanın tamamen silinmesi nasıl engellenecektir?
- İşte sticky bit (t olarak gösterilir) bu özel durumun çözümünde kullanılır.
- Bir dizin üzerinde sticky bit aktif ise, o dizin altında her kullanıcı yeni dosya oluşturabilir ve kendi oluşturduğu dosyaları silebilir. Diğer kullanıcıların oluşturduğu dosyaları ise silemez.

Erişim Yetkilerinin Görüntülenmesi

- Bir dosya üzerindeki erişim yetkilerinin görüntülenmesi için ls komutu -l parametresi ile çalıştırılmalıdır:

```
bfljpg@vbox:~$ ls -l /usr/lib/fedora-release
-rw-r--r--. 1 root root 26 Apr 12  2024 /usr/lib/fedora-release
bfljpg@vbox:~$
```

- Dizinler için aynı parametrelerle çalıştırıldığında, ilgili dizinin içeriği listelenecektir. Dizinin kendisiyle ilgili erişim yetkilerini görmek içinse komut -d parametresi eklenerek kullanılmalıdır:

```
bfljpg@vbox:~$ ls -ld /etc/rc.d/init.d
drwxr-xr-x. 1 root root 30 Oct  6 22:35 /etc/rc.d/init.d
bfljpg@vbox:~$
```

- ls komutunun çıktısındaki erişim yetkileriyle ilgili ilk karakter aşağıdaki tablo doğrultusunda anlamlandırılır:
  - -: standart dosya (regular file)
  - d: dizin
  - c: karakter tabanlı aygıt dosyası (/dev/console vb.)
  - b: blok tabanlı aygıt dosyası (/dev/sda3 vb.)
  - s: özel dosya (unix domain soket vb.)
- Erişim yetkileriyle ilgili ilk karakterden sonraki 3'lü blok, dosya/dizin sahibinin dosya/dizin üzerindeki erişim yetkilerini gösterir.
- Bir sonraki 3'lü blok, dosya/dizin'in grup sahibinin dosya/dizin üzerindeki erişim yetkilerini gösterir.
- Sonraki ve son 3'lü blok ise, dosya/dizin için sahibi veya grup sahibi kısmına girmeyen kullanıcılar için (other) tanımlanmış olan erişim yetkilerini gösterir.

Erişim Yetkilerinin Düzenlenmesi

- Dosya sisteminde yer alan bir dosya/dizin için erişim yetkilerinin düzenlenmesi işlemi chmod komutu ile yapılır. Değiştirilmek istenen yetkilerin neler olduğu ve üçlü erişim yetki bloklarından hangisi veya hangileriyle ilişkili olduğu parametre olarak belirtilir. Parametrelerde erişim yetki grubu belirtildikten sonra, + veya - ile hangi yetkilerin ekleneceği veya çıkartılabileceği belirtilebileceği gibi, = ile tam olarak hangi yetkilere sahip olacağı da belirtilebilmektedir.

`chmod` Erişim Yetki Grupları

Karakter	Etkilediği Grup
u	Dosya/dizin sahibi
g	Dosya/dizin grup sahibi
o	Dosya/dizin sahibi ve grup sahibi dışında kalanlar (others)
a	Dosya/dizin sahibi, grup sahibi ve bunların dışında kalanlar dahil her üç blok

`chmod` Erişim Yetki Parametreleri

Karakter	Yetki
r	Okuma
w	Yazma
x	Çalıştırma
X	Sadece dizinler için çalıştırma yetkisi
s	Çalışma zamanında **user id** veya **grup id** değişimi (suid bit)
S	Suid biti aktif fakat çalıştırma yetkisi yok
t	Sticky bit
u	Dosyanın kullanıcı (user) olarak sahibinin yetkileri
g	Dosyanın grup sahibi yetkileri
o	Dosya ile ilgili diğer (other) kullanıcıların yetkileri

- Aşağıdaki file1 dosyası ve dir1 dizini için erişim yetki değişikliği örneklerini inceleyiniz:

Komut	Açıklama
`chmod u+rwx file1`	Dosyanın sahibine okuma, yazma, çalıştırma yetkisi ver
`chmod u-wx file1`	Dosyanın sahibinden yazma ve çalıştırma yetkisini kaldır
`chmod u+w-r file1`	Dosyanın sahibine yazma yetkisini ver, okuma yetkisini kaldır
`chmod u=rw file1`	Dosyanın sahibine okuma ve yazma yetkisi ver, çalıştırma yetkisi verme
`chmod u+x,g+wx file1`	Dosyanın sahibine çalıştırma yetkisi ekle, aynı zamanda grup sahibine yazma ve çalıştırma yetkisi ekle
`chmod g=rw file1`	Dosyanın grup sahibine okuma ve yazma yetkisi ver, çalıştırma yetkisini kaldır

`chmod o=r file1`	Sahip veya grup sahibi dışında kalan diğer kullanıcılara sadece okuma yetkisi ver
`chmod +x file1`	Tüm erişim gruplarına (sahip, grup ve diğer), çalıştırma yetkisi ekle
`chmod -x file1`	Tüm erişim gruplarından çalıştırma yetkisini kaldır
`chmod a=r file1`	Tüm erişim gruplarına (a = all) okuma yetkisi ver, yazma ve çalıştırma yetkisini kaldır
`chmod u+s file1`	SUID yetkisini ekle
`chmod g+s file1`	SGID yetkisini ekle
`chmod +t file1`	Sticky bitini aktifleştir
`chmod -R g-X dir1/`	Dizin altında recursive olarak dolaş, sadece bulunan alt dizinlerin grup sahiblerinden dizin içine geçme (x) yetkisini kaldır

Sekizli Sistemle Gösterim

- chmod komutu yukarıdaki bölümde anlattığımız şekilde kullanılabileceği gibi, chmod 755 dir örneğindeki gibi erişim yetkilerinin bir sayı ile ifade edilmesi yöntemiyle de kullanılabilir.
- Erişim yetkilerinin sayısal değerlerle ifade edildiği bu modelin kullanımı kulağa daha zor gelse de, en sık kullanılan 3-5 erişim yetki deseni için bir süre sonra numaralarının öğrenilmesi neticesinde pratikte sayısal değerlerin kullanımının oldukça yaygın olduğunu söyleyebiliriz.
- Örnek olarak, dosyanın sadece sahibinin okuma/yazma yapıp, grup sahibi ve geri kalanların sadece okuma yaptığı 644 sayısı veya tüm kullanıcı ve gruplara tüm hakların verilmesini ifade eden 777 sayısı pek çok kullanıcı tarafından ezbere bilinir.

- İstediğimiz erişim yetki bitleri için doğru sayıyı bulabilmek için aşağıdaki tablodan faydalanabilirsiniz. Soldan sağa doğru, read, write ve execute şeklinde sıralanan erişim bitlerinin aktif olma veya olmama durumuna göre (1 veya 0), elde edilen rakam önemlidir.
- Her 3 erişim yetki grubu için bu 3 bitin durumundan elde edilen sayı yazılıp birleştirildiğinde chmod için kullanılacak değer öğrenilmiş olur.
- Yani 644 olarak sayıyı görmek yerine, 1. üçlü erişim biti kombinasyonu değerinin 6 (rw-), 2. üçlü erişim biti kombinasyonu değerinin 4 (r--) ve son olarak 3. erişim biti kombinasyonu değerinin de 4 (r--) olduğunu düşünmeliyiz. Bu rakamları yanyana yazdığımızda 644'ü elde ediyoruz.

8'li değer	Read	Write	Execute
7	r	w	x
6	r	w	-
5	r	-	x
4	r	-	-
3	-	w	x
2	-	w	-
1	-	-	x
0	-	-	-

- Peki SUID, SGID ve Sticky Bit'inin rakam olarak ifadesini nasıl yapacağız?
- 3 adet olan bu özel bitleri de, ls komutu çıktısında gösterildiği sırasıyla, sst (rwx) biçiminde görebiliriz.

SUID	SGID	Sticky Bit
s	s	t

- Bu özel bitlerin aktif olup olmama durumuna göre gene 0-7 arasında bir sayı değeri elde edilecektir. Örneği SUID ve SGID biti işaretlenmemiş ancak Sticky Bit işaretlenmişse bu bloktan elde edilecek sayı değeri 1 olacaktır. Aynı şekilde sadece SUID işaretliyse 2 olacaktır. Sadece SUID işaretliyse 4 olacaktır. Bunlarının hepsinin toplamı 7'dir ve bu durumda tüm bitler aktif olur.
- Geriye kalan bitler bu oluşan bitin sonuna eklenir. Sekizli sistemde eğer 755 olarak erişim izni verilecekse bu 1755 veya 2755 veya 7755 olarak ifade edilir.

Process (İşlem) Nedir?

- Linux'ta **süreç (process)**, çalışan bir programın bir örneğidir. Her süreç bir **PID** (Process ID) ile tanımlanır ve çekirdek bu süreçlerin takibini yapar.
- Çekirdek, aynı anda birçok süreci çalıştırır ve hangi süreçlerin CPU'yu ne kadar süre kullanacağını belirleyen bir **zamanlayıcı** kullanır.
- Linux çekirdeğinde farklı zamanlayıcı algoritmalar bulunur. Örneğin, **Completely Fair Scheduler (CFS)**, her sürece adil bir şekilde işlemci süresi verilmesini sağlar.
- Ayrıca çekirdek, **süreçlerin önceliğine** göre kaynakları yönetir. Yüksek öncelikli süreçler daha fazla CPU zamanı alırken düşük öncelikli süreçler arka planda çalıştırılır.
- Bir uygulama yeni bir süreç başlattığında, çekirdek bunu izler ve her sürecin ne kadar kaynak tükettiğini yönetir.

Process Oluşumu ve Yaşam Döngüsü

Process Yaratma:

- Linux'ta her işlem bir başka işlem tarafından yaratılır. İşlemleri yaratmak için kullanılan mekanizma, **forking** olarak bilinir.
- Linux işletim sistemi, hiyerarşik bir yapı kullanarak her işlemi **ebeveyn (parent)** ve **alt (child)** işlemler olarak organize eder. En üstte tüm işlemlerin ebeveyni olan **init** ya da **systemd** adlı bir işlem bulunur.

2. fork() ve exec() Sistem Çağrıları:

- **fork():**
  - fork() sistemi çağrısı, yeni bir child process yaratır. Bu yeni işlem, parent işlemden kopyalanmış olur, yani başlangıçta parent ile aynı kodu çalıştırır.
  - fork() çağrıldığında iki işlem çalışmaya başlar: biri parent, diğeri child. Bu nedenle, fork() iki kez döner; parent işlem için child'in PID'si ile, child işlem için ise sıfır değeriyle.
- **exec():**
  - exec() sistem çağrısı, bir işlemin bellekteki kodunu değiştirmek için kullanılır. Yani yeni bir process yaratılmadan, var olan bir process üzerinde farklı bir program çalıştırılır.
  - Genellikle fork() ile yeni bir child process yaratıldıktan sonra, exec() ile child process üzerinde farklı bir program çalıştırılır.
- **wait() sistemi çağrısı:**
  - Parent işlemler, child işlemlerinin tamamlanmasını beklemek için wait() çağrısını kullanır.Eğer child işlem wait() kullanılmadan biterse, zombi işlem (zombie process) olarak sistemde kalır. Zombi işlemler çok fazla biriktiğinde sistem performansı düşebilir.

Process ID (PID) ve Ebeveyn-Çocuk İlişkisi

- Her işlemin, sistem tarafından atanan benzersiz bir kimliği (PID) vardır. PID sayesinde işlemler birbirinden ayırt edilir. Parent işlem, kendi yarattığı child işlemin PID'sini bilir ve onun üzerinde kontrol sağlayabilir.
- Parent-Child İlişkisi: Bir parent işlem, child işlemlerinin bitmesini beklemek için wait() sistem çağrısını kullanabilir. Bu şekilde parent, child işlem sona ermeden devam etmez.

Process Durumları ve Yaşam Döngüsü:

Bir process'in yaşam döngüsü boyunca geçebileceği bazı durumlar şunlardır:

- Running (Çalışıyor):İşlem şu anda CPU tarafından yürütülüyor.
- Waiting (Bekleme):İşlem, belirli bir olayın gerçekleşmesini bekliyor (örneğin bir giriş/çıkış işlemi tamamlanana kadar).
- Sleeping (Uyuma):İşlem kısa süreliğine çalışmayı durdurmuş ancak geri dönecek.
- Stopped (Durduruldu):İşlem dış bir sinyal ile durdurulmuş, ancak tamamen sonlandırılmamış.
- Terminated (Sonlanmış):İşlem sona erdi ve sistemden kaldırıldı. Ölü işlemler, sonlanmış olsalar da hala sistemde kalabilir. Bu tür işlemler zombie (zombi) olarak adlandırılır.

Örnek Kod: fork() ve exec() ile Process Yaratma

- fork() kullanılarak yeni bir child process yaratılır. Child processüzerinde execlp() ile ls komutu çalıştırılır. Parent process, wait() kullanarak child işleminin bitmesini bekler.
- «touch process.c» yazıp yeni dosya oluşturalım ve «nano process.c» diyerek kod dosyamızı yazmaya başlayalım.
- Programı görüntülerdeki satır numaralarına dikkat ederek yazın. gcc programını kurmayı ihmal etmeyin. Program «sudo dnf install gcc» ile kurulmalıdır.
- Kod yazımı bittikten sonra «gcc process.c –o fake_process» diyerek derlemeye başlayalım.
- Koddan sonraki ekran görüntüsünde gösterildiği gibi çalıştıralım ve deneyelim.

GNU nano 7.2 fake_process.c

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();  // Yeni bir child process yarat

    if (pid == 0) {
        // Child process
        printf("Child process çalışıyor, PID = %d\n", getpid());
        execlp("/bin/ls", "ls", NULL);  // Child process ls komutunu çalıştırıyor
    } else if (pid > 0) {
        // Parent process
        printf("Parent process çalışıyor, PID = %d\n", getpid());
        wait(NULL);  // Child process'in bitmesini bekle
    } else {
        // fork başarısız oldu
        printf("Fork başarısız oldu!\n");
    }

    return 0;
}
```

```
bfljpg@vbox:~$ nano process.c
bfljpg@vbox:~$ gcc process.c -o process
bfljpg@vbox:~$ ./process
Parent process çalışıyor, PID = 3661
Child process çalışıyor, PID = 3662
Desktop    donanim.txt  extract_region    fake_process  Pictures  process.c  Public     vdso.out
Documents  Downloads    extract_region.c  Music         process   proje      Templates  Videos
bfljpg@vbox:~$
```

- Programımızı başarıyla derleyip çalıştırdığımızda yukarıdaki ekran görüntüsüne benzer bir çıktı almalısınız.
- Bu çalıştırmamızda parent processimiz 3661 PID siyle oluşturulmuş ve child processimiz 3662 PID siyle oluşturulmuş. Bu PID ler her çalıştırmada değişecektir.
- Child processimiz başarıyla çalıştığında kodda da açıkça görüldüğü gibi ls komutunu çalıştırdı.

Dosya İşlemleri

- Unix tabanlı sistemlerde genel olarak, bir şey okunabiliyor veya yazılabiliyorsa, dosya arayüzü üzerinden de kullanılabilir. Örnek olarak ağ iletişiminde kullanılan soketler, seri port, dizinler, pipe, timer, sinyal vb. pek çok bileşen üzerinde dosya arayüzü ile işlem yapmak mümkündür.
- Dosya arayüzünü açmak için öncelikle ilgili dosyayı açmak ve dosya betimleyiciyi file descriptor elde etmek gereklidir.

- File descriptor elde etme yolları kullanılan sisteme göre değişir. Örneğin standart bir dosya için open() sistem çağrısı kullanılırken soket açmak istiyorsak socket(), timer oluşturmak istiyorsak timerfd_create() gibi ilgili sisteme özgü özel bir sistem çağrısı kullanılır.
- Tüm bu çağrıların ortak özelliği, başarılı olunması durumunda geriye int tipinde bir file descriptor dönülüyor olmasıdır. File descriptor elde edildikten sonra artık bu dosya ile ilgili tüm işlemlerde, dosya kapatılana kadar ilgili int değeri kullanılıyor olacaktır.

Proses Dosya Tablosu

- İşletim sistemindeki proses tablosunda, her proses ile ilgili açık dosyaların listesi tutulur. Bu liste yapısı içerisinde açık dosyalarla ilgili ek bir takım bilgiler de saklanır (örneğin normal bir dosya ise inode numarası, erişim yetkileri, son erişim zamanı, geçerli file position vb.)
- Çekirdek içerisinde yer alan bu proses tablosunun daha basit bir haline /proc dosya sistemi üzerinden erişmek de mümkündür. Her bir proses id (PID) için sonraki slaytlarda bilgilerin alınması mümkündür:

/proc/PID/fd

- İlgili PID için açık dosyaların int descriptor numaralarını içeren sembolink linkler bulunur. Bu sembolink linklerin bulunduğu dizine ls -l komutuyla bakılacak olursa veya uygulamanızda readlink() fonksiyonuyla sembolik linkin işaret ettiği yerle ilgili bilgileri okumaya çalıştığınızda type:[inode] formatında bir yanıt gelir.
- Dosya sistemi üzerinde fiziksel bir *inode* olmadığı senaryolar için de inode sistematiği içinde kalınarak özgün bir numara verilir.

- Örnek olarak açık soket bağlantıları için readlink() sonrasında socket:[205846] şeklinde bir yanıt döner. Burada tip olarak soket bağlantısını olduğunu görmekteyiz, inode bilgisi olarak da 205846 numarası dönülmüştür. Bu numaranın soketler için anlamlandırılması /proc/net altındaki ilgili dosyaların okunmasıyla yapılır. Eğer bu soket bağlantısı bir TCP soketine aitse, /proc/net/tcp dosyasında, UDP soketine aitse /proc/net/udp dosyasında vb. burada görülen inode numarasıyla eşleşecek şekilde kayıtlar bulunduğu görülecektir.
- Eğer inode konseptiyle henüz ilişkilendirilemeyen epoll_create(), inotify_init(), signalfd() gibi bir fonksiyonla elde edilmiş file descriptor söz konusu ise bu durumda anon_inode:<file type> formatında bir çıktı verecektir.

/proc/PID/fdinfo

- Linux 2.6.22 versiyonu ve sonrasında bu dizin yapısı üzerinden ek bilgiler elde edilmesi mümkündür. Dizin içerisinde her bir int file descriptor ile aynı isimde dosyalar bulunur. Herhangi bir dosyanın içerisinde baktığımızda aşağıdaki bilgiler elde edilir.

```
pos:    0
flags:  02
mnt_id: 7
```

- Bu dosyada file pointer pozisyonu, dosya açılırken kullanılan flag'ler, dosyanın ilişkili olduğu mount point referansı yer almaktadır.

/proc/PID/fd ve /proc/PID/fdinfo için bir örnek

- İşlem dosya tanımlayıcılarına (/proc/[PID]/fd) ve dosya tanımlayıcı bilgilerine (/proc/[PID]/fdinfo) erişim sağlayan bir C programı yazabiliriz. Bu program kendi process ID'si ile /proc dizininde ilgili dosyaları okuyacak ve dosya tanımlayıcılarını (file descriptors) listeleyecek, ardından her dosya tanımlayıcısının detaylı bilgisini gösterecek.
- Programı görüntülerdeki satır numaralarına dikkat ederek yazın. gcc programını kurmayı ihmal etmeyin. Program «sudo dnf install gcc» ile kurulmalıdır.

```c
#include <stdio.h>
#include <stdlib.h>
#include <dirent.h>
#include <unistd.h>
#include <string.h>

#define BUFFER_SIZE 1024  // sabit bir buffer size belirliyoruz

// belirtilen process ID ve dosya tanımlayıcıya ait fdinfo dosyasını okuma fonksiyonu
void read_fdinfo(char *pid, char *fd) {
    char fdinfo_path[BUFFER_SIZE];
    // fdinfo dosyasının tam yolunu oluştur
    snprintf(fdinfo_path, BUFFER_SIZE, "/proc/%s/fdinfo/%s", pid, fd);

    FILE *fdinfo_file = fopen(fdinfo_path, "r");
    if (fdinfo_file == NULL) {
        perror("fdinfo dosyası açılamadı");  // dosya açılamazsa hata mesajı bas
        return;
    }

    char line[BUFFER_SIZE];
    printf("FD %s için FDINFO:\n", fd);
    while (fgets(line, sizeof(line), fdinfo_file) != NULL) {
        printf("%s", line);  // fdinfo dosyasının her satırını ekrana yazdır
    }
    fclose(fdinfo_file);  // dosyayı kapat
    printf("\n");
}

```

```c
// process ID'ye ait /proc/[PID]/fd dizinini okuyarak dosya tanımlayıcıları listeleyen fonksiyon
void list_fd(char *pid) {
    char fd_dir[BUFFER_SIZE];
    snprintf(fd_dir, BUFFER_SIZE, "/proc/%s/fd", pid);  // fd dizin yolunu oluştur

    DIR *dir = opendir(fd_dir);
    if (dir == NULL) {
        perror("fd dizini açılamadı");  // dizin açılamazsa hata mesajı bas ve çık
        exit(EXIT_FAILURE);
    }

    struct dirent *entry;
    // fd dizinindeki her dosya tanımlayıcıyı oku
    while ((entry = readdir(dir)) != NULL) {
        if (entry->d_name[0] != '.') {  // '.' ve '..' dizinlerini atla
            char fd_path[BUFFER_SIZE];
            snprintf(fd_path, BUFFER_SIZE, "/proc/%s/fd/%s", pid, entry->d_name);  // fd sembolik bağlantısının tam yolunu oluştur

            char link_target[BUFFER_SIZE];
            ssize_t len = readlink(fd_path, link_target, sizeof(link_target) - 1);  // Sembolik bağlantıyı çöz
            if (len != -1) {
                link_target[len] = '\0';  // sonlandırma karakteri ekle
                printf("FD %s -> %s\n", entry->d_name, link_target);  // dosya tanımlayıcısını ve nereye işaret ettiğini yazdır
                read_fdinfo(pid, entry->d_name);  // bu dosya tanımlayıcı için fdinfo'yu oku
            }
        }
    }

    closedir(dir);  // dizini kapat
}
```

```c

int main() {
    pid_t pid = getpid();  // programın process ID'sini al
    char pid_str[BUFFER_SIZE];
    snprintf(pid_str, BUFFER_SIZE, "%d", pid);  // PID'yi string olarak sakla

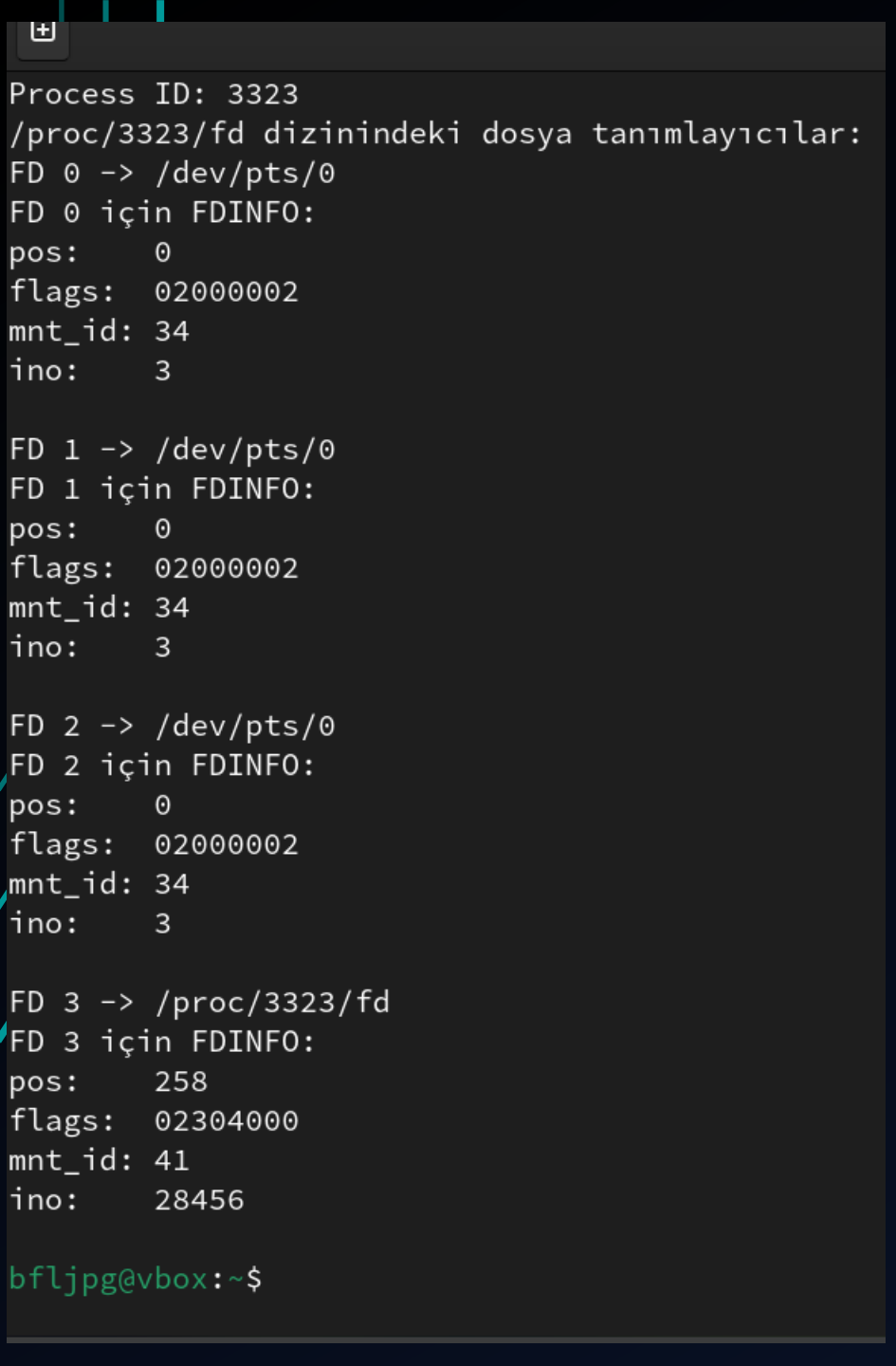
    printf("Process ID: %s\n", pid_str);
    printf("/proc/%s/fd dizinindeki dosya tanımlayıcılar:\n", pid_str);

    list_fd(pid_str);  // process ID ile fd dizinini ve fdinfo'yu listele

    return 0;
}

```

- Derlemeden önce gcc kurmayı ihmal etmeyin. Kurulumu «sudo dnf install gcc» komutu ile yapmalısınız.
- Kodu yazdıktan sonra «gcc processinfo.c –o processinfo» yazarak derleyin ve derlendikten sonra «./processinfo» yazarak çalıştırmayı deneyin. Bir sonraki slayttaki örnek ekran görüntüsündeki benzer sonucu alacaksınız.



- Programı derleyip çalıştırdığımızda karşımıza böyle bir sonuç çıkması gerekir. /proc/[PID]/fd ve /proc/[PID]/fdinfo dizinlerine erişemediği durumda hata mesajı vererek uygulama sonlanır.
- Çıktıdaki file descriptorların rolleri:
- FD 0, 1, 2: Standart giriş, çıkış ve hata akışları genellikle /dev/pts/X ile ilişkili olur (terminal).
- FD 3: Uygulamanın kendi file descriptor bilgisidir ve fdinfo dosyasında dosyanın konumu, erişim bayrakları gibi bilgiler gösterilir.
- pos, flags, mnt_id ve ino (index node) ile ilgili detaylı açıklamalar bir sonraki sayfada.

- FD 0, FD 1, FD 2 (/dev/pts/0):
  - pos: Bu dosya tanımlayıcılarının mevcut okuma/yazma konumunu temsil eder. Burada pos: 0 olarak görünmektedir, yani okuma veya yazma işlemi henüz yapılmamıştır.
  - flags: Dosya tanımlayıcının izin ve mod bilgilerini belirtir. 02000002 değeri, dosyanın salt okunabilir (O_RDONLY) ve dosyanın terminal veya karakter cihazına bağlı olduğunu (O_CTTY) belirtir.
  - mnt_id: Bağlı olduğu dosya sisteminin ID'sini gösterir. Aynı dosya sistemindeki dosyalar için bu ID aynı olabilir.
  - ino: inode numarası, dosyanın dosya sistemindeki benzersiz kimliğidir. Burada ino: 3 olarak görünmekte; tüm tanımlayıcılar /dev/pts/0 dosyasını temsil ettiği için inode numarası aynıdır.

- FD 3 (/proc/3354/fd):
  - pos: pos: 258 olarak verilmiş; bu, dosya içerisinde veri okumada nereye gelindiğini gösterir.
  - flags: 02304000 değeri, burada dosyanın sadece okuma ve yürütme iznine sahip (O_PATH) olduğunu gösterir.
  - mnt_id ve ino: Bağlı olduğu dosya sisteminin ID'sini ve inode numarası belirtilmiştir. Inode değeri 28456 olan dosya, bu tanımlayıcıyı /proc/3323/fd dizinine açmış durumdadır.

Açık Dosya Limitleri

- Unix tabanlı sistemlerde, bir prosesin aynı anda açabileceği dosya sayısının bir limiti bulunmaktadır.

- Bu limitler soft limit ve hard limit adında 2 başlıkta toplanır. Açık dosya limitini «ulimit» uygulamasıyla veya «setrlimit()» fonksiyonuyla artırmak mümkündür. Ancak bu değer hard limit'i aşamaz. Bu limit ancak root kullanıcısı tarafından değiştirilebilir.

```
bfljpg@vbox:~$ ulimit -a
real-time non-blocking time  (microseconds, -R) unlimited
core file size              (blocks, -c) unlimited
data seg size               (kbytes, -d) unlimited
scheduling priority                 (-e) 0
file size                   (blocks, -f) unlimited
pending signals                     (-i) 23515
max locked memory           (kbytes, -l) 8192
max memory size             (kbytes, -m) unlimited
open files                          (-n) 1024
pipe size                (512 bytes, -p) 8
POSIX message queues         (bytes, -q) 819200
real-time priority                  (-r) 0
stack size                  (kbytes, -s) 8192
cpu time                   (seconds, -t) unlimited
max user processes                  (-u) 23515
virtual memory              (kbytes, -v) unlimited
file locks                          (-x) unlimited
bfljpg@vbox:~$ ulimit -n 2048
bfljpg@vbox:~$ ulimit -n
2048
bfljpg@vbox:~$
```

- ulimit, bir kullanıcı veya işlem için sistem kaynaklarını sınırlandırmak amacıyla kullanılan bir komuttur. Bu sınırlandırmalar, sistemin stabil çalışmasını sağlamak ve kaynakların tükenmesini önlemek için oldukça faydalıdır. Buradaki limitler her bir işlemin limitlerini belirler. İşte bazı temel limitler:
- CPU Süresi (-t): Bir işlemin kullanabileceği maksimum CPU süresini sınırlar. Bu süre dolduğunda işlem sonlandırılır.
- Bellek Kullanımı (-m): Bir işlemin kullanabileceği maksimum fiziksel bellek miktarını belirler.

- Açık Dosya Tanımlayıcıları (-n): Aynı anda açık tutulabilecek maksimum dosya tanımlayıcı sayısını sınırlar; yüksek değerler sistem kaynaklarını zorlayabilir.
- Yığın Boyutu (-s): Bir işlem için izin verilen maksimum yığın (stack) boyutunu belirler, bellek taşması riskini azaltır.
- Dosya Boyutu (-f): Bir işlemin oluşturabileceği maksimum dosya boyutunu belirtir. Bu sınır, dosyaların diskte çok fazla yer kaplamasını engellemek için kullanılabilir.
- Sanal Bellek Boyutu (-v): Bir işlemin kullanabileceği toplam sanal bellek miktarını sınırlar. Bu limit, işlemlerin fazla bellek tüketimini önleyerek sistem belleğinin tükenmesini engeller.

- Veri Segmenti Boyutu (-d): İşlem başına kullanılabilecek veri segmentinin (data segment) maksimum boyutunu tanımlar. Bu, işleme özgü veri alanlarının boyutunu sınırlandırarak bellek kullanımı kontrolü sağlar.
- Çekirdek Dosya Boyutu (-c): Bir işlem çöktüğünde oluşturulabilecek çekirdek dosyasının (core dump) maksimum boyutunu belirler. Çekirdek dosyaları, hataların izlenmesi için kullanılır, ancak büyük boyutlu olabilirler.
- İş Parçacığı Sayısı (-u): Aynı anda çalıştırılabilecek maksimum iş parçacığı (thread) sayısını belirler. Bu, çok sayıda iş parçacığı oluşturan uygulamalarda CPU ve bellek kullanımını sınırlamak için yararlıdır.
- Bu limitler, sistem güvenliği ve performansı için ayarlanarak kaynak kullanımını kontrol altında tutar.

ulimit ile ilgili bir örnek

- Bir sonraki sayfada örnek uygulama ile parametre vererek aynı anda kaç dosya açabileceğini test edebilir, ulimit -n komutuyla limitleri değiştirip tekrar testi gerçekleştirebilirsiniz.
- Bir sonraki slaytta ulimit.c adında bir dosya ve ondan sonraki slaytta debug.h adında başka bir kütüphane dosyası oluşturacağız.
- Eğer bilgisayarınızda yoksa gcc programını kurmayı unutmayın. Program «sudo dnf install gcc» komutu ile kurulmalıdır.

- Program koduyla aynı dizinde local olarak oluşturacağımız debug.h kütüphanesi. Bunu derlemenize gerek yok, sadece ulimit-debug.c ile aynı dizin içerisinde oluşturmanız yeterli.

GNU nano 7.2 debug.h

```
#ifndef DEBUG_H
#define DEBUG_H

#include <stdio.h>
#include <stdlib.h>

// Hata ayıklama mesajı basan fonksiyon
#define debugf(fmt, ...) \
    do { \
        fprintf(stdout, "DEBUG: " fmt "\n", ##__VA_ARGS__); \
    } while (0)

// Hata mesajı basan fonksiyon
#define errorf(fmt, ...) \
    do { \
        fprintf(stderr, "ERROR: " fmt "\n", ##__VA_ARGS__); \
        exit(EXIT_FAILURE); \
    } while (0)

#endif /* DEBUG_H */
```

- ulimit-debug.c dosyası

```c
#include <stdio.h>      // printf, fprintf gibi standart I/O fonksiyonları için gerekli
#include <unistd.h>     // open, getpid gibi POSIX sistem çağrıları için gerekli
#include <sys/types.h>  // POSIX veri türleri için gerekli (örneğin: pid_t)
#include <sys/stat.h>   // dosya izinleriyle ilgili makrolar için gerekli
#include <fcntl.h>      // open fonksiyonu ve dosya modları için gerekli
#include <stdlib.h>     // atoi, exit gibi standart fonksiyonlar için gerekli
#include "debug.h"      // debugf ve errorf için local olarak oluşturduğumuz kendi başlık dosyamız

int main (int argc, char *argv[]) {
    int opened = 0;  // Açılan dosya sayısını tutmak için bir değişken
    int wanted;      // Kullanıcının açmak istediği dosya sayısı
    int i;           // Döngü değişkeni

    // Eğer program doğru sayıda (1 argüman) argüman almadıysa, hata mesajı göster
    if (argc != 2) {
        printf("Usage: %s NumberOfFiles\n", argv[0]);  // Kullanım hakkında bilgi verir
        exit(1);  // Hatalı kullanım nedeniyle program sonlandırılır
    }
```

```
19
20      // Kullanıcının verdiği argümanı tam sayıya çevirir (kaç dosya açmak istediğini belirler)
21      wanted = atoi(argv[1]);
22
23      // 'wanted' sayısı kadar dosya açmayı dener
24      for (i = 0; i < wanted; i++) {
25          // /dev/zero dosyasını okuma modunda açar
26          // Eğer açma işlemi başarısız olursa hata mesajı verir ve döngüden çıkar
27          if (open("/dev/zero", O_RDONLY) < 0) {
28              errorf("Couldn't open file");  // Hata mesajı yazdırır ve programı sonlandırır
29              break;  // Dosya açma başarısız olduğu için döngüden çıkar
30          }
31          opened++;  // Başarılı bir şekilde dosya açıldıysa, açılan dosya sayısını artır
32      }
33
34      // Tüm dosyalar açıldıktan sonra, kaç dosyanın başarıyla açıldığını yazdırır
35      debugf("Total of %d files opened", opened);
36
37      return 0;  // Program normal şekilde sonlanır
38  }
```

```
bfljpg@vbox:~$ nano ulimit-debug.c
bfljpg@vbox:~$ gcc ulimit-debug.c -o ulimit-debug
bfljpg@vbox:~$ ulimit -n
1024
bfljpg@vbox:~$ ulimit -n 2048
bfljpg@vbox:~$ ulimit -n
2048
bfljpg@vbox:~$ ./ulimit-debug 2045
DEBUG: Total of 2045 files opened
bfljpg@vbox:~$ ./ulimit-debug 2046
ERROR: Couldn't open file
bfljpg@vbox:~$ ./ulimit-debug 2045
DEBUG: Total of 2045 files opened
bfljpg@vbox:~$
```

nano ile yazıp gcc ile derlediğimizde çalıştırmadan önce «ulimit –n» ile açabileceğimiz dosya limitini kontrol ediyoruz.

Not: gcc kütüphanesi «sudo dnf install gcc» komutuyla yüklenmelidir.

1024 ayarlanmış haldeyken 2048 ayarladıktan sonra programımızla ilk denemeyi gerçekleştiriyoruz. 2045 dosyayı başarıyla açıyor. Ancak 2046 veya daha yüksek bir sayıda bir dosya açamıyoruz.

2048 ayarladığımız halde açamamasının temel sebebi önceki sayfalarda da gördüğümüz /dev/pts/0 file descriptorların da buna dahil olmasıdır.

+10 Puan Vize Ödevinden Örnekler

- Ödev sorusu:
- Kullanıcının Desktop klasöründe “kilitli_klasor” adında bir dizin ve onun içinde herhangi bir dosya açınız. Bu dizin ve dosya, dosya yöneticisinde şifre erişimi gerektirecek şekilde yetkilerini yeniden düzenleyiniz. Bu dizin ve klasörün yetkilerini yeni kullanıcıya aktarınız.
- Bu konuda aşağı yukarı en çok yapılan 2 örnek var: birincisi encfs ile o klasörü şifreleyerek kilitlemek, diğeri ise chmod ve chown ile dosya ve klasörün sahibini değiştirerek şu anda açık olan kullanıcıya engellenmesini sağlamak.

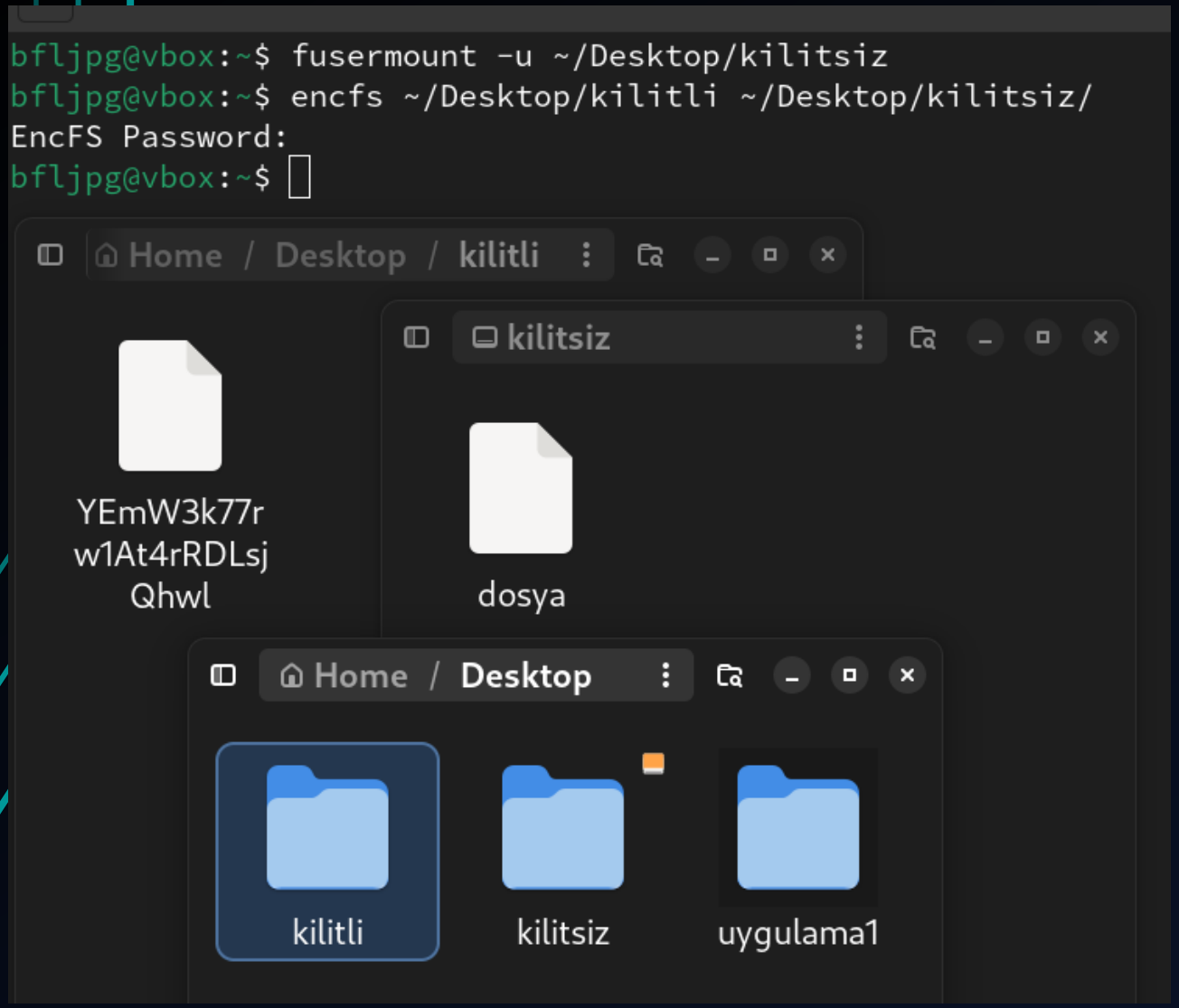
Encfs ile gerçekleştirilen şifreleme metodu

```
bfljpg@vbox:~$ mkdir -p ~/Desktop/kilitli
bfljpg@vbox:~$ mkdir -p ~/Desktop/kilitsiz
bfljpg@vbox:~$ encfs ~/Desktop/kilitli ~/Desktop/kilitsiz/
Creating new encrypted volume.
Please choose from one of the following options:
 enter "x" for expert configuration mode,
 enter "p" for pre-configured paranoia mode,
 anything else, or an empty line will select standard mode.
?> p

Paranoia configuration selected.

Configuration finished.  The filesystem to be created has
the following properties:
Filesystem cipher: "ssl/aes", version 3:0:2
Filename encoding: "nameio/block", version 4:0:2
Key Size: 256 bits
Block Size: 1024 bytes, including 8 byte MAC header
Each file contains 8 byte header with unique IV data.
Filenames encoded using IV chaining mode.
File data IV is chained to filename IV.
File holes passed through to ciphertext.
```

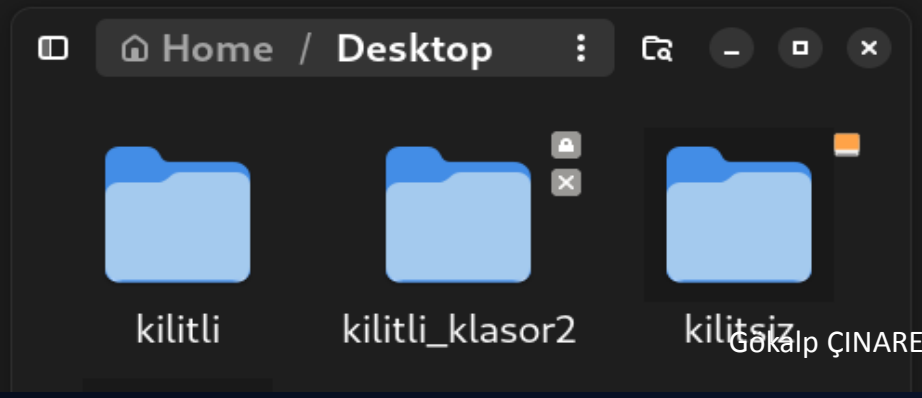
- Solda ekran görüntüsü verildiği üzere şifreli ve şifresiz olacak şekilde iki klasör oluşturuyoruz.
- Burada şifresiz bir klasör daha oluşturmamızın sebebi şifresi çözülen klasöre bir bağlantı noktası oluşturmak.
- encfs komutuyla eğer varsa o anki şifreli klasör bağlanabiliyor. Aksi halde ekranda da görüldüğü gibi yeni bir şifreleme gerçekleştirebiliyor.



- Terminaldeki ilk komut oluşturduğumuz şifreli klasörün bağlantısını kesmemize yarıyor.
- İkinci komut ise oluşturduğumuz şifreli klasörün şifresiz başka klasöre bağlantısını gerçekleştiriyor.
- Ekranda Desktop klasöründe klasörlerin görüldüğü üzere şifresiz olan klasörde bir sanal bağlantı işareti var.
- Şifreli klasör verileri kriptolayarak saklamaktadır. Şifresiz ise kriptosu çözülmüş halde dosyayı kullanmamızı sağlıyor.

Chmod ve chown ile kullanıcı sahipliği değiştirmek

```
bfljpg@vbox:~$ sudo useradd yeni
useradd: warning: the home directory /home/yeni already exists.
useradd: Not copying any file from skel directory into it.
Creating mailbox file: File exists
bfljpg@vbox:~$ sudo passwd yeni
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
bfljpg@vbox:~$ mkdir ~/Desktop/kilitli_klasor2
bfljpg@vbox:~$ touch ~/Desktop/kilitli_klasor2/dosya
bfljpg@vbox:~$ echo "bu şifreli bir dosya. " >> ~/Desktop/kilitli_klasor2/dosya
bfljpg@vbox:~$ sudo chown yeni:yeni ~/Desktop/kilitli_klasor2/
bfljpg@vbox:~$ sudo chown yeni:yeni ~/Desktop/kilitli_klasor2/dosya
bfljpg@vbox:~$ sudo chmod 700 ~/Desktop/kilitli_klasor2/
bfljpg@vbox:~$ sudo chmod 600 ~/Desktop/kilitli_klasor2/dosya
bfljpg@vbox:~$
```



- Yanda gösterilen ekran görüntüsündeki komutları kullanarak yeni bir kullanıcı açıyoruz ve şifresini belirliyoruz.
- Bu metoddaki temel prensip şu anda açık olan kullanıcının erişimini engellemek ve bunu dosya yöneticisi uygulamasında ispatlamaktır.
- Desktop klasöründe yeni bir kilitli klasör ve içinde dosya açıyoruz.
- chown ile dosyaların sahipliğini yeni kullanıcımıza veriyoruz.
- Dosyaların sahipliğini aktardığımızda yeni kullanıcının erişimi için chmod ile yeniden dosya sahibine erişim yetkilerimizi ayarlıyoruz.
- Klasör için verilen chmod 700 komutu sadece dosyanın yeni sahibi olan kullanıcı için tüm erişim yetkilerini vermesine yarıyor. Dosya için verilen chmod 600 komutu ise bundan farklı olarak execute yetkisi hariç tüm yetkilerin yeni sahibine sağlıyor.

Dosya İşlemlerinde Temel C Fonksiyonları

Dosyalar, programların dış dünyayla veri alışverişi yapmasını sağlayan temel araçlardır. C programlama dilinde, dosyalarla çalışmak için geniş bir standart kütüphane desteği sunulur. Bu kütüphane fonksiyonları sayesinde:

- Dosyaları oluşturabilir, silebilir veya yeniden adlandırabilirsiniz.
- Geçici dosyalarla çalışabilir ve rastgele dosya adları oluşturabilirsiniz.
- Program içinde dosya verilerini okuyup yazabilirsiniz.

Özellikle system, remove, rename, tmpfile ve tmpnam gibi fonksiyonlar, dosya yönetiminde sıkça kullanılan ve çeşitli ihtiyaçlara hızlı çözümler sunan işlevlerdir. Her biri, hem performans hem de kullanım kolaylığı açısından C dilinin güçlü özelliklerinden biridir.

Remove Fonksiyonu

Bu fonksiyon, belirtilen dosyayı sistemden silmek için kullanılır. İşlem, dosya adı veya yolu bir karakter dizisi olarak girilerek gerçekleştirilir. remove, C dilinin standart kütüphanesinde yer alan ve basit bir şekilde dosya kaldırma işlemini sağlayan bir fonksiyondur.

filename parametresi, silinmek istenen dosyanın yolunu veya adını içeren bir karakter dizisidir. Bu dizi, NULL karakteri ('\0') ile sonlandırılmış olmalıdır. Dosya, sistemin erişebileceği bir konumda olmalı ve işlem için gerekli izinlere sahip olunmalıdır.

Dönüş Değeri:

- Başarı: Fonksiyon başarılı olursa 0 değeri döndürülür.
- Hata: Eğer işlem sırasında bir sorun oluşursa (örneğin, dosya bulunamazsa ya da erişim izni yoksa), sıfırdan farklı bir hata kodu döndürülür. Hatanın detayını incelemek için errno kullanılabilir.

Remove Fonksiyonu ile ilgili bir Örnek

```
  GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp;

    if((fp = fopen("ornek.txt", "w+")) == NULL)
    {
        printf("Dosya açılamadı.\n");
        exit(1);
    }

    fputs("Bilgisayar", fp);
    fclose(fp);

    if (remove("ornek.txt") == 0) {
        printf("Dosya başarıyla silindi.\n");
    } else {
        perror("Dosya silinemedi");
    }
    return 0;
}
```

- Görmüş olduğunuz örnekte uygulamamız ornek.txt adında bir dosya oluşturacak ve remove fonksiyonu ile ir dosyayı silmeye çalışıyoruz. «if» yapısı kullanarak dosyanın silinmesini kontrol ederek çıktı vermesini sağlıyoruz.
- «touch remove.c» komutuyla dosyanızı oluşturup «nano remove.c» komutuyla düzenlemeye başlayabilirsiniz. Kodunuzu yazdıktan sonra Ctrl+X tuşlarıyla kaydettikten sonra «gcc remove.c –o remove» komutuyla remove uygulamamızı oluşturmuş olacağız.

```
bfljpg@vbox:~$ nano remove.c
bfljpg@vbox:~$ gcc remove.c -o remove
bfljpg@vbox:~$ ./remove
Dosya başarıyla silindi.
bfljpg@vbox:~$ S
```

- Derledik ve çalıştırdık. Bu örnekte kullandığımız dosya oluşturma ve silme aşamalarının temel amacı yazmış olduğumuz uygulamaların oluşturacağı geçici dosyaların C temelinde kontrolünün nasıl yapıldığını anlamak olacaktır.

Rename fonksiyonu

- Bu fonksiyon, bir dosyanın adını veya konumunu değiştirmek için kullanılır. Belirtilen mevcut dosya adı (oldname) yerine yeni bir ad veya farklı bir konum (newname) belirleyerek işlem gerçekleştirilir. rename fonksiyonu, C dilinin standart kütüphanesinde yer alır ve dosya yönetiminde temel bir işleve sahiptir.
- Fonksiyon gösterimi:
- int rename(const char *oldname, const char *newname);

Parametreler:

- oldname: Değiştirilecek dosyayı tanımlayan yolu veya adı içeren bir karakter dizisidir. Bu dizi, NULL karakteri ('\0') ile sonlandırılmış olmalıdır.
- newname: Dosyanın yeni adını veya taşınacağı yeni yolu belirten karakter dizisidir. Bu dizi de NULL karakteri ile sonlandırılmış olmalıdır.

Dönüş Değeri:

- Başarı: İşlem başarıyla tamamlanırsa 0 değeri döndürülür.
- Hata: Eğer işlem sırasında bir hata oluşursa (örneğin, oldname ile belirtilen dosya bulunamazsa ya da newname zaten mevcutsa), sıfırdan farklı bir hata kodu döner. Hatanın detayını anlamak için errno değişkeni kullanılabilir.

Rename Fonksiyonu için Örnek Kullanım

```
  GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>

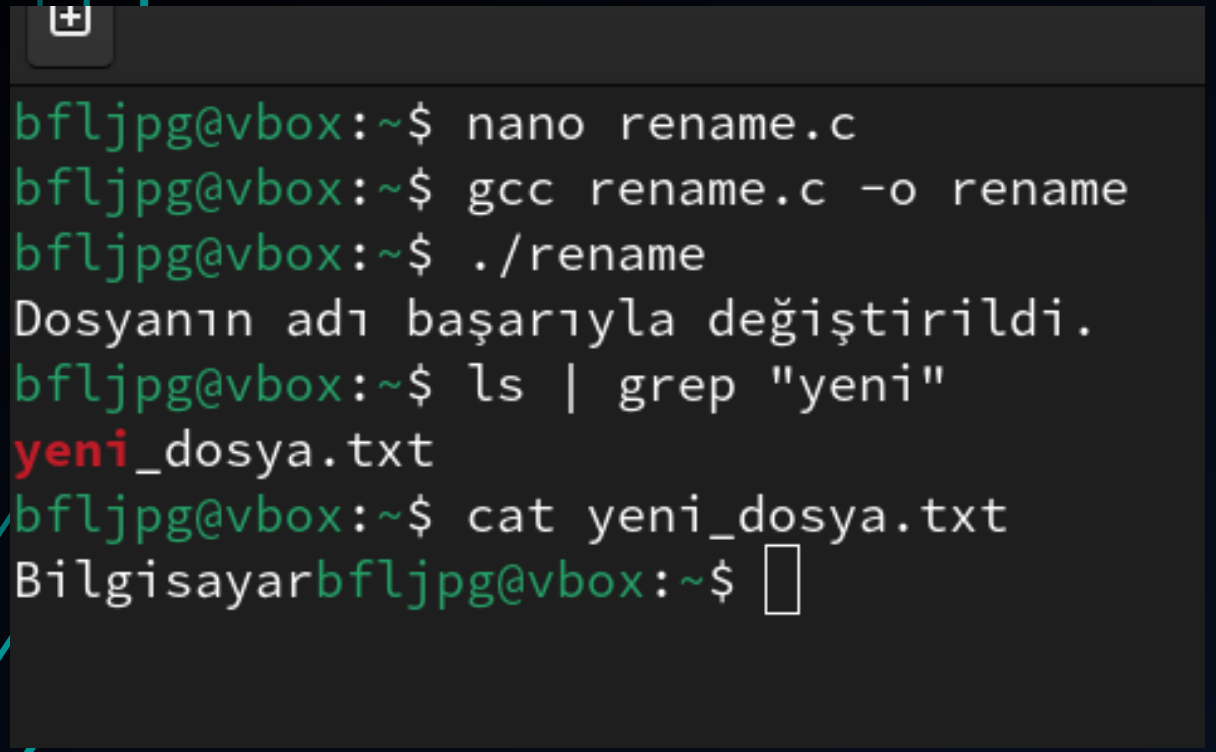
int main() {
    FILE *fp; // Dosyamızı adresleyen pointer

    // Dosyayı oluşturma aşaması
    if((fp = fopen("eski_dosya.txt", "w+")) == NULL)
    {
        printf("Dosya açılamadı.\n");
        exit(1);
    }

    // Oluşuturulan dosyaya içerik yazdırma
    // ve dosya adreslemesini kapama
    fputs("Bilgisayar", fp);
    fclose(fp);

    // rename fonksiyonu ile dosya adı değiştirme
    if (rename("eski_dosya.txt", "yeni_dosya.txt") == 0) {
        printf("Dosyanın adı başarıyla değiştirildi.\n");
    } else {
        perror("Dosya adı değiştirilemedi");
    }
    return 0;
}
```

- Görmüş olduğunuz örnekte rename fonksiyonu ile eski_dosya.txt adında bir dosya oluşturuyoruz ve bu dosyanın ismini yeni_dosya.txt olarak değiştirmeye çalışıyoruz. «if» yapısı kullanarak dosyanın silinmesini kontrol ederek çıktı vermesini sağlıyoruz.
- «touch rename.c» komutuyla dosyanızı oluşturup «nano rename.c» komutuyla düzenlemeye başlayabilirsiniz. Kodunuzu yazdıktan sonra Ctrl+X tuşlarıyla kaydettikten sonra «gcc rename.c –o rename» komutuyla rename uygulamamızı oluşturmuş olacağız.



- Derledik ve çalıştırdık. «./rename» komutu ile çalıştırıyoruz.
- «ls | grep yeni» komutuyla dosyamızın oluştuğunu teyit ediyoruz.
- Bu örnek uygulamanın da temel amacı tıpkı remove fonksiyonunda olduğu gibi yazılan uygulama içinde bu tip dosya işlemeleri için metodları öğretmektir.

Tmpfile Fonksiyonu

tmpfile() fonksiyonu, geçici bir dosya oluşturmak için kullanılır. Bu fonksiyon, stdio.h başlık dosyasında tanımlıdır ve programın çalışma süresi boyunca geçici olarak kullanılan bir dosya sağlar. Fonksiyonun oluşturduğu dosya, program sonlandırıldığında veya dosya kapatıldığında otomatik olarak silinir.

Çalışma Prensibi:

- tmpfile() tarafından oluşturulan geçici dosya, genellikle rastgele bir ad alır ve sistemin geçici dosyalar için ayrılmış dizininde yer alır.
- Bu dosya, ikili modda (binary mode) ve güncelleme (read/write) işlemleri için açılır. Hem okuma hem de yazma işlemlerine olanak tanır.
- Dosya oluşturulamazsa (örneğin, geçici dizine yazma izni yoksa), fonksiyon NULL işaretçisi döndürür ve hata durumunu belirlemek için errno kontrol edilebilir.

Dönüş Değeri:

- Başarı: Geçici dosyanın tanımlayıcısı olan bir FILE * işaretçisi döner.
- Hata: Eğer dosya oluşturulamazsa, NULL döndürülür.

Kullanım Alanları:

- Geçici verilerin saklanması ve işlenmesi.
- Programın yalnızca çalışma süresi boyunca gereken dosyalar için güvenli bir çözüm.
- Daha az dosya yönetimi karmaşası, çünkü dosya otomatik olarak silinir.

Tmpfile Fonksiyonu için Örnek Kullanım

GNU nano 7.2 tmpfile.c

```c
#include <stdio.h>

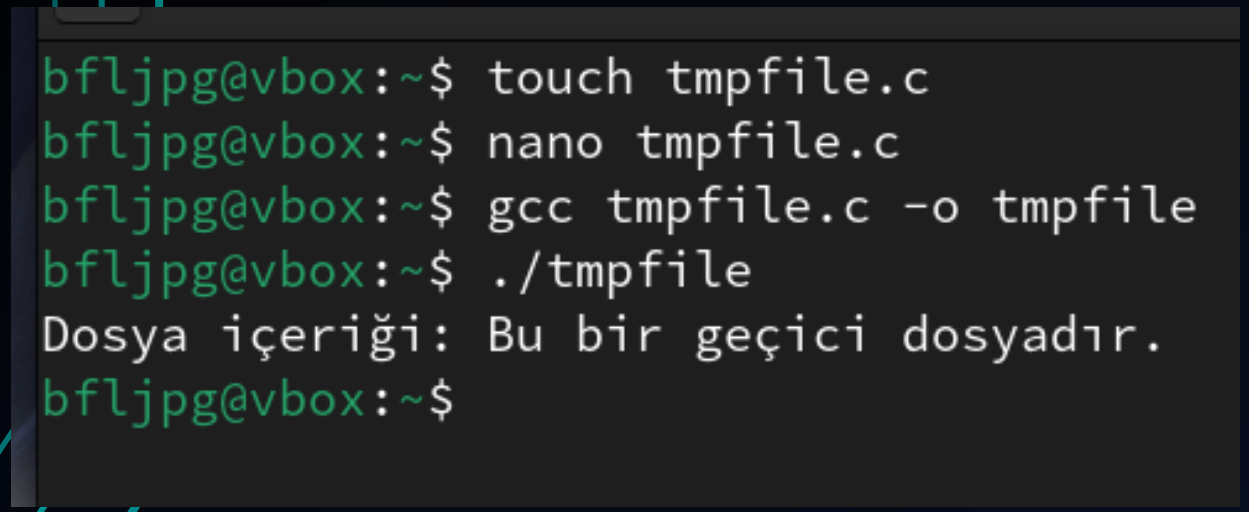
int main() {
    FILE *temp_file = tmpfile();
    if (temp_file == NULL) {
        perror("Geçici dosya oluşturulamadı");
        return 1;
    }

    fprintf(temp_file, "Bu bir geçici dosyadır.\n");
    rewind(temp_file);

    char buffer[100];
    while (fgets(buffer, sizeof(buffer), temp_file) != NULL) {
        printf("Dosya içeriği: %s", buffer);
    }

    fclose(temp_file); // Geçici dosya silinir.
    return 0;
}
```

- tmpfile() ile geçici dosya oluşturulur: Eğer dosya oluşturulamazsa hata mesajı yazdırılır ve program sonlandırılır.
- Örnek olarak "Bu bir geçici dosyadır." metni dosyaya eklenir.
- rewind() fonksiyonu, dosya işaretçisini dosyanın başlangıcına alır.
- fgets() ile dosyanın içeriği okunur ve terminale yazdırılır.
- fclose() fonksiyonu çağrıldığında, geçici dosya otomatik olarak silinir.
- Bu örnek, tmpfile() fonksiyonunun geçici dosyalarla kolay ve güvenli bir şekilde çalışmayı nasıl sağladığını gösterir.



- Yanda gösterildiği gibi yazıp çalıştırdığımızda buna benzer bir çıktı göreceksiniz.
- «touch tmpfile.c» ile dosyamızı oluşturup «nano tmpfile.c» ile kodumuzu yazmaya başladık. «gcc tmpfile.c –o tmpfile» komutuyla derlememizi aldık.
- «./tmpfile» komutuyla uygulamamızı çalıştırdık.

Tmpnam Fonksiyonu

- tmpnam() fonksiyonu, benzersiz ve geçerli bir geçici dosya adı oluşturmak için kullanılır. Bu fonksiyon, dosya adını belirtilen karakter dizisine (filename) atar veya içsel bir statik arabellekten döndürür. Fonksiyon, C dilinin standart kütüphanesinde tanımlıdır ve genellikle geçici dosya adı oluşturulması gereken durumlarda kullanılır.
- Çalışma Prensibi:
- Benzersiz Dosya Adı: Fonksiyon, işletim sisteminin geçici dosyalar için ayrılmış alanında benzersiz bir dosya adı oluşturur. Ancak, bu isim dosya sisteminde hali hazırda kullanımda olabilir; bu nedenle, ek doğrulama gerekebilir.
- Dosya Adı Uzunluğu: Oluşturulan dosya adı, sistem sabiti olan L_tmpnam değerinden uzun olamaz.
- Deneme Sayısı: Fonksiyon, maksimum TMP_MAX deneme sayısına kadar benzersiz dosya adı üretmeye çalışır. Bu sınır aşıldığında ya da sistemde uygun bir ad bulunamazsa NULL döndürür.

Parametreler:

- filename:
- Eğer bir işaretçi sağlanırsa, oluşturulan dosya adı bu işaretçinin gösterdiği alana yazılır ve işaretçi geri döndürülür.
- Eğer filename parametresi NULL olarak verilirse, fonksiyon bir dahili statik arabellek kullanır ve bu arabelleğin adresini döndürür.

Dönüş Değeri:

- Başarı: Uygun bir dosya adı oluşturulursa, bu adın bulunduğu karakter dizisinin işaretçisi döndürülür.
- Hata: Eğer geçerli bir dosya adı oluşturulamazsa, NULL döndürülür.
- Kullanım Alanları:
- Geçici dosyalar oluşturulmadan önce benzersiz bir ad oluşturmak.
- Dosya adı çakışmalarını önlemek için güvenli bir çözüm geliştirmek.

Tmpnam Fonksiyonu için Örnek Kullanım

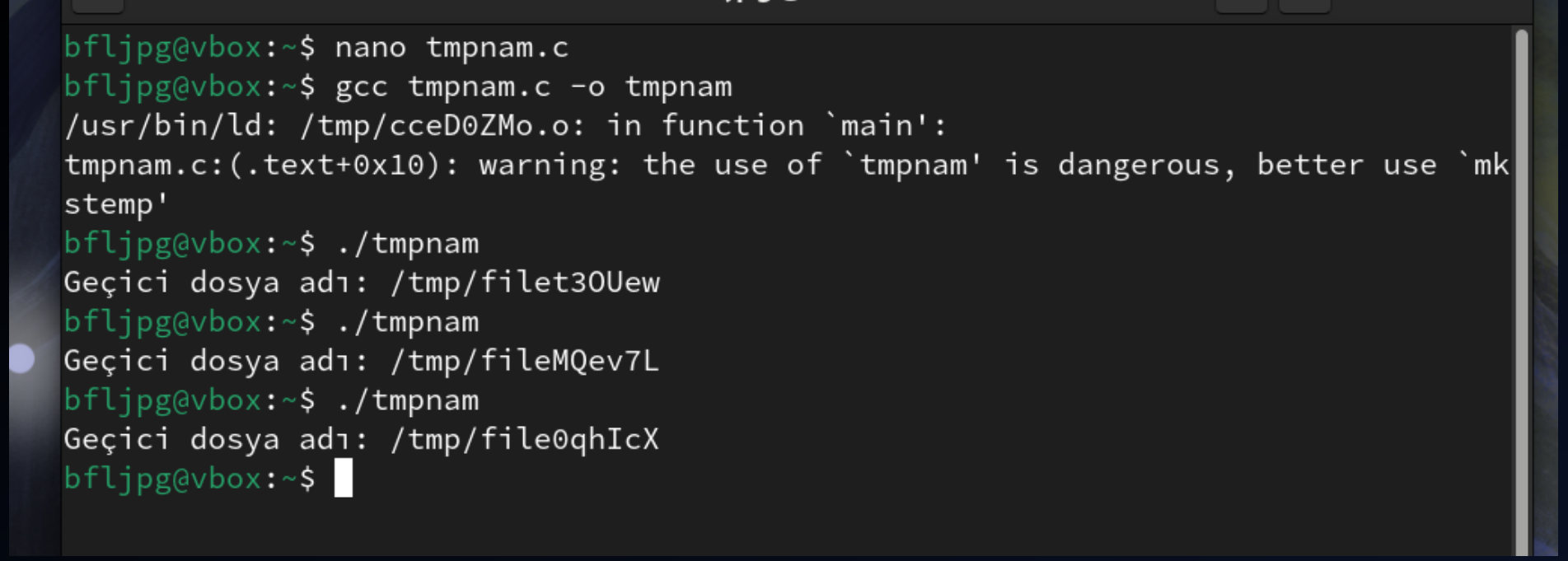
```
  GNU nano 7.2
#include <stdio.h>

int main() {
    // tmpnam için rastgele bir isim oluştur
    char dosya_adi[L_tmpnam];

    // Geçici dosya adı oluşturma
    if (tmpnam(dosya_adi) != NULL) {
        printf("Geçici dosya adı: %s\n", dosya_adi);
    } else {
        printf("Geçici dosya adı oluşturulamadı.\n");
    }

    return 0;
}
```

- Bu örnekte, tmpnam() fonksiyonu ile benzersiz bir geçici dosya adı oluşturuluyor ve bu ad dosya_adi dizisine kaydediliyor.
- Eğer dosya adı başarıyla oluşturulursa, tmpnam() döndürülen adı ekrana yazdırmak için printf() kullanılır.
- Eğer dosya adı oluşturulamazsa, NULL döndürülür ve kullanıcıya hata mesajı gösterilir.
- Bu basit örnek, geçici dosya adı oluşturmanın temel işleyişini gösterir.



- Gördüğünüz üzere derleyip çalıştırdığımızda her oluşturduğu dosyanın benzersiz bir isme sahip olduğunu ve geçici dosyaların tutulduğu /tmp dizininine oluşturulduğunu göreceksiniz.
- Derlerken alabileceğiniz ekran görüntüsünde de olan uyarı bu metodun artık güvenilirliğini yitirdiğini ve mkstemp adında başka bir fonksiyonun kullanılmasını önerdiğini anlatıyor

I/O Modelleri ve Aşamaları

- Girdi İşlemlerinin İki Ayrı Aşaması
- Girdi/çıktı (I/O) işlemleri bağlamında genellikle iki ana aşama bulunur:
  - 1. Veri Bekleme: Bu aşama, verinin hazır hale gelmesini beklemeyi içerir. Bu durum, verinin bir ağdan gelmesi, bir disk okuma işleminin tamamlanması veya diğer herhangi bir I/O işlemi için geçerlidir.
  - 2. Veri Kopyalama: Veri hazır hale geldikten sonra, ikinci aşama, verinin çekirdek (kernel) alanından kullanıcı alanına (user space) kopyalanmasını içerir; böylece uygulama veriyi işleyebilir.
- Bu iki aşamayı anlamak önemlidir çünkü farklı I/O modelleri bu aşamaları çeşitli şekillerde ele alır; bu da uygulamaların verimliliği ve yanıt verme süresi üzerinde doğrudan etkilidir.

Beş Farklı I/O Modeli

- Engelleyici (Blocking) I/O
- Engellemeyen (Nonblocking) I/O
- I/O Çoklama (Multiplexing) (select ve poll)
- Sinyal Tabanlı I/O (SIGIO)
- Asenkron I/O (aio_)

- Her bir modeli ayrıntılı olarak inceleyelim.

Engelleyici (Blocking) I/O

- Engelleyici I/O modelinde, uygulama her iki I/O aşamasında da engellenir (yani uyku moduna geçer). Veri hazır olana kadar bekler ve daha sonra veriyi kullanıcı alanına kopyalar. Bu süreç boyunca uygulama tamamen bloke olmuş durumdadır ve başka hiçbir işlem gerçekleştiremez.

Çalışma Akışı:

- Uygulama read() sistem çağrısını yapar.
- Veri hemen hazır değilse, çekirdek uygulamayı uyku durumuna alır.
- Veri hazır hale gelene kadar bu durumda kalır.
- Veri hazır olduğunda, veri çekirdekten kullanıcı alanına kopyalanır.
- Sistem çağrısı döner ve uygulama işlemine devam eder.

Kullanım Alanları:

- Kodlamanın kolaylığının performanstan daha önemli olduğu basit uygulamalarda kullanılır.
- Uygulamanın I/O işlemleri sırasında başka iş yapmasının gerekmediği senaryolar için uygundur.

```c
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

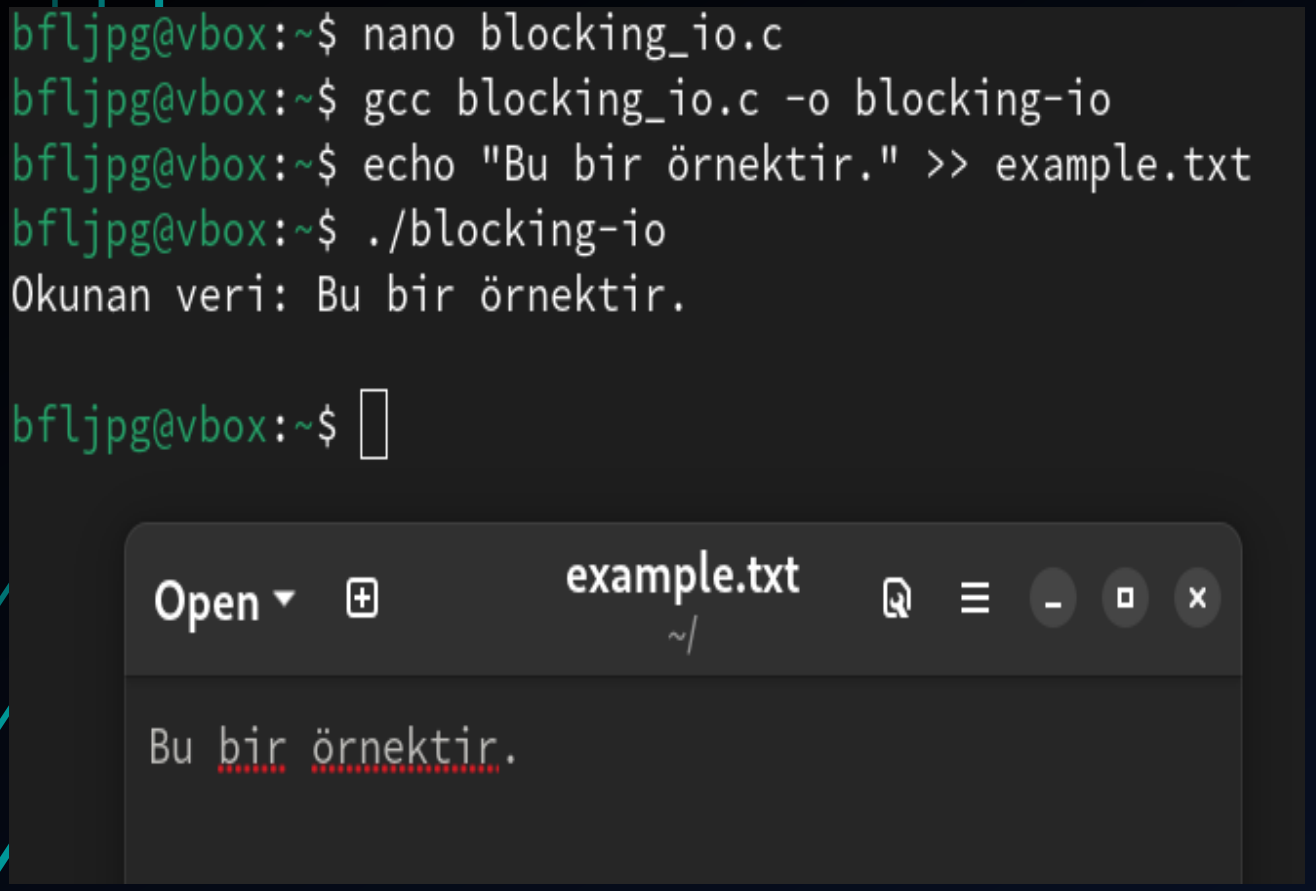
int main() {
    // Dosyayı okuma modunda aç
    int fd = open("example.txt", O_RDONLY);
    char buffer[100]; // Veriyi tutacak bir buffer

    if (fd == -1) {
        perror("Dosya açılamadı"); // Hata kontrolü
        return 1;
    }

    // Blocking read
    ssize_t bytesRead = read(fd, buffer, sizeof(buffer)); // Veriyi oku
    if (bytesRead == -1) {
        perror("Okuma hatası"); // Okuma hatası kontrolü
    } else {
        // Okunan veriyi ekrana yazdır
        printf("Okunan veri: %.*s\n", (int)bytesRead, buffer);
    }

    close(fd); // Dosyayı kapat
    return 0;
}
```


- Yandaki örnek programı yazıp derleyin ve çalıştırın.
- Çalıştırmadan önce uygulamayı derlediğiniz dizin içerisinde istediğiniz bir girdiyi example.txt içerisine yazmayı unutmayın.
- Program oluşturduğunuz dosyayı okurken (16. satır) bloke olacak ve başka işlem gerçekleştirmeyecek.
- Engelleyici I/O modelinde, dosyadan okuma işlemi başlatıldığında uygulama veriyi okuyup kullanıcı alanına kopyalayana kadar bekler.



- Derleyip çalıştıktan sonra yandaki ekran görüntüsüne benzer bir çıktı göreceksiniz.
- Başarıyla Blocking I/O modeli programla, oluşturduğumuz dosya okundu ve terminale yazdırdı.

Engellemeyen (Nonblocking) I/O

- Engellemeyen I/O modelinde, uygulama I/O işleminden hemen döner ve veri henüz hazır değilse beklemez. Bunun yerine, veri hazır olduğunda periyodik olarak kontrol eder veya bu süre zarfında başka işlemler yapabilir.

Çalışma Akışı:

- Uygulama, dosya tanımlayıcısını engellemeyen moda ayarlar.
- Uygulama read() sistem çağrısını yapar.
- Veri hazır değilse, sistem çağrısı hemen bir hata (genellikle EWOULDBLOCK) ile döner.
- Uygulama, diğer işlemleri yapabilir ve daha sonra okuma işlemini tekrar deneyebilir.

Kullanım Alanları:

- Birden fazla I/O kaynağını aynı anda ele almak isteyen uygulamalar.
- Çok sayıda dosya tanımlayıcısının izlenmesi gereken durumlar.

Nonblocking I/O için bir örnek program

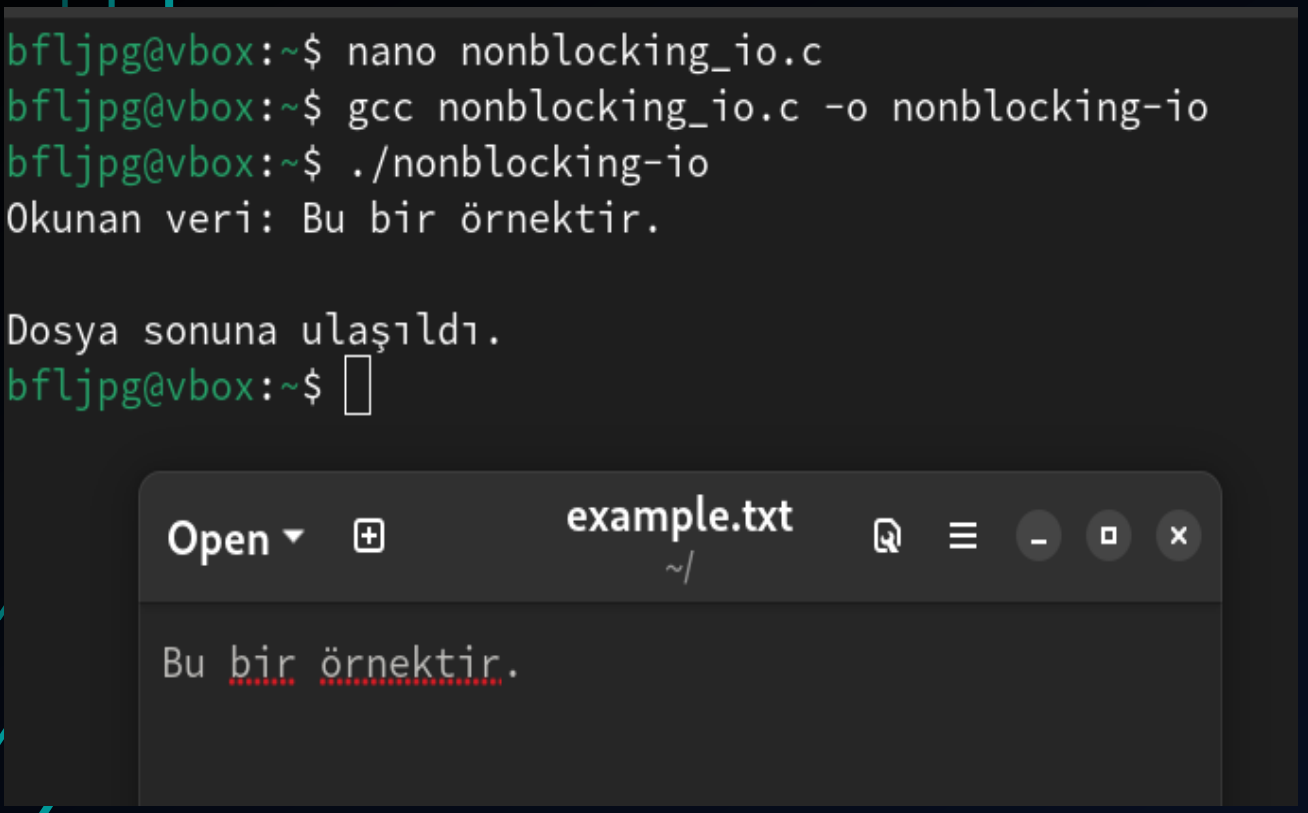
```c
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>

int main() {
    // Dosyayı non-blocking modda aç
    int fd = open("example.txt", O_RDONLY | O_NONBLOCK);
    char buffer[100]; // Veriyi tutacak bir buffer

    if (fd == -1) {
        perror("Dosya açılamadı"); // Hata kontrolü
        return 1;
    }

    while (1) {
        ssize_t bytesRead = read(fd, buffer, sizeof(buffer)); // Veriyi oku
        if (bytesRead == -1) {
            if (errno == EAGAIN) {
                printf("Veri hazır değil, tekrar deneniyor...\n"); // Veri hazır değil
                sleep(1); // Başka işlemler yapılabilir
                continue;
            } else {
```

```
24                perror("Okuma hatası"); // Okuma hatası kontrolü
25                break;
26            }
27        } else if (bytesRead == 0) {
28            printf("Dosya sonuna ulaşıldı.\n"); // Dosya sonu
29            break;
30        } else {
31            // Okunan veriyi ekrana yazdır
32            printf("Okunan veri: %.*s\n", (int)bytesRead, buffer);
33        }
34    }
35
36    close(fd); // Dosyayı kapat
37    return 0;
38 }
39
```



- Derleyip çalıştıktan sonra yandaki ekran görüntüsüne benzer bir çıktı göreceksiniz.
- Başarıyla Nonblocking I/O modeli programla, oluşturduğumuz dosya okundu ve terminale yazdırdı.

I/O Çoklama (Multiplexing) (select ve poll)

- I/O çoklama modelinde, birden fazla I/O kaynağını kontrol etmek mümkündür. select veya poll sistem çağrıları ile birden fazla dosya tanımlayıcısının durumunu izlemek için beklenir. Bu yöntem, birden fazla I/O işlemi olan uygulamalar için etkilidir.

Çalışma Akışı:

- Uygulama select veya poll çağrısını yaparak birden fazla dosya tanımlayıcısını izler.
- Herhangi bir dosya tanımlayıcısı veri almak için hazır hale geldiğinde sistem çağrısı döner.
- Uygulama, hazır tanımlayıcıları kullanarak I/O işlemlerini gerçekleştirir.

Kullanım Alanları:

- Birden fazla I/O kaynağını aynı anda ele almak isteyen uygulamalar.
- Çok sayıda dosya tanımlayıcısının izlenmesi gereken durumlar.

```c
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/select.h>

int main() {
    // İki dosyayı non-blocking modda aç
    int fd1 = open("example1.txt", O_RDONLY | O_NONBLOCK);
    int fd2 = open("example2.txt", O_RDONLY | O_NONBLOCK);
    char buffer[100]; // Veriyi tutacak bir buffer

    if (fd1 == -1 || fd2 == -1) {
        perror("Dosya açılamadı"); // Hata kontrolü
        return 1;
    }

    fd_set readfds; // Okuma dosya tanımlayıcıları için bir set
    int maxfd = fd1 > fd2 ? fd1 : fd2; // En büyük dosya tanımlayıcısını bul

    while (1) {
        FD_ZERO(&readfds); // Set'i temizle
        FD_SET(fd1, &readfds); // fd1'i ekle
        FD_SET(fd2, &readfds); // fd2'yi ekle

        // Dosya tanımlayıcılarını kontrol et
        int ready = select(maxfd + 1, &readfds, NULL, NULL, NULL);
        if (ready == -1) {
            perror("Select hatası"); // Hata kontrolü
            break;
        }

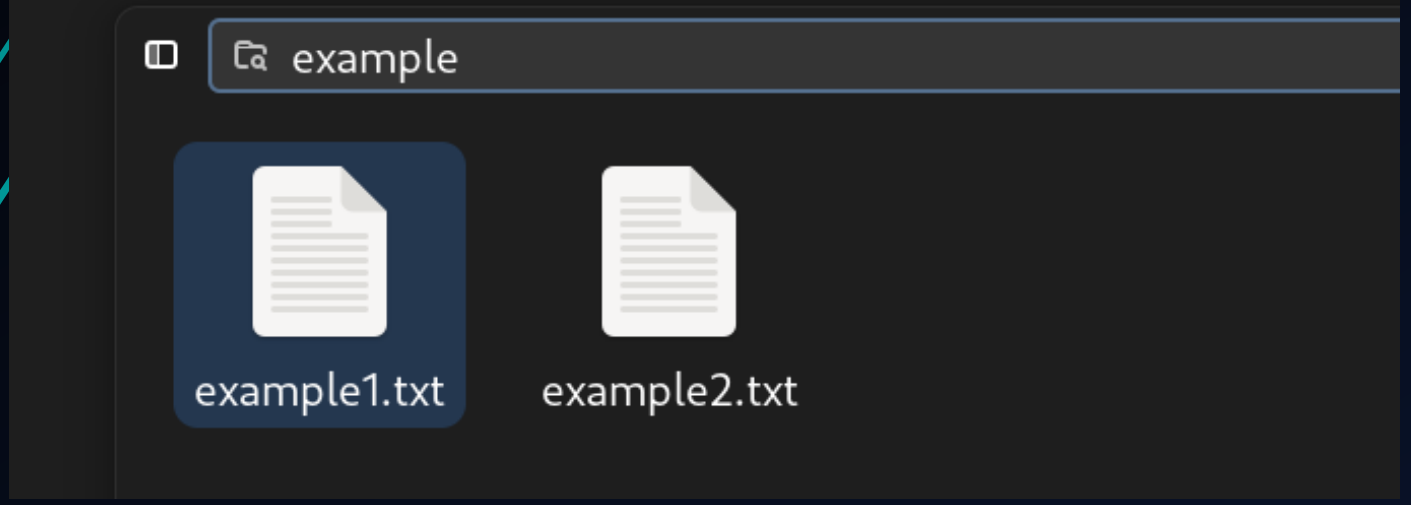
```

```
31
32          // Eğer fd1 veri okumaya hazırsa
33          if (FD_ISSET(fd1, &readfds)) {
34              ssize_t bytesRead = read(fd1, buffer, sizeof(buffer)); // Veriyi oku
35              if (bytesRead > 0) {
36                  printf("example1.txt verisi: %.*s\n", (int)bytesRead, buffer);
37              }
38          }
39
40          // Eğer fd2 veri okumaya hazırsa
41          if (FD_ISSET(fd2, &readfds)) {
42              ssize_t bytesRead = read(fd2, buffer, sizeof(buffer)); // Veriyi oku
43              if (bytesRead > 0) {
44                  printf("example2.txt verisi: %.*s\n", (int)bytesRead, buffer);
45              }
46          }
47      }
48
49      close(fd1); // Dosyaları kapat
50      close(fd2);
51      return 0;
52  }
```

```
bfljpg@vbox:~$ nano multiplexing.c
bfljpg@vbox:~$ gcc multiplexing.c -o multiplexing
bfljpg@vbox:~$ ./multiplexing
Dosya açılamadı: No such file or directory
bfljpg@vbox:~$ echo "Bu birinci dosya." >> example1.txt
bfljpg@vbox:~$ echo "Bu ikinci dosya." >> example2.txt
bfljpg@vbox:~$ ./multiplexing
example1.txt verisi: Bu birinci dosya.

example2.txt verisi: Bu ikinci dosya.

example1.txt dosyasının sonuna ulaşıldı.
bfljpg@vbox:~$
```



- Programı derleyip direkt çalıştırdığımızda hata aldığını göreceksiniz. Sebebi ise örnek programın example1.txt ve example2.txt dosyalarının olmaması.
- echo komutu ile bu örnek dosyaları oluşturduktan sonra tekrar deneyin ve sonuçları kontrol edin.
- Başarıyla I/O Multiplexing modeli programla, oluşturduğumuz dosya okundu ve terminale yazdırdı.

Sinyal Tabanlı I/O (SIGIO)

- Sinyal tabanlı I/O modelinde, uygulama bir dosya tanımlayıcısına sinyallerin etkinleştirilmesini sağlar. I/O işlemi mümkün hale geldiğinde, çekirdek uygulamaya bir SIGIO sinyali gönderir ve böylece I/O işlemi yapılabileceğini bildirir.

Çalışma Akışı:

- Uygulama, dosya tanımlayıcısının SIGIO sinyali üretmesini sağlar.
- Veri hazır olduğunda, çekirdek uygulamaya bir SIGIO sinyali gönderir.
- Uygulama, sinyal işleyici içinde I/O işlemini gerçekleştirir.

Kullanım Alanları:

- I/O mümkün hale geldiğinde hemen haberdar edilmek isteyen ancak diğer işlerle ilgilenmesi gereken uygulamalar.
- Bildirimler için sinyal işlemenin tercih edildiği senaryolar.

Asenkron I/O (aio_)

- Asenkron I/O modelinde, uygulama bir I/O işlemi başlatır ve işlemin tamamlanmasını beklemeden diğer işlemlerine devam eder. I/O işlemi tamamlandığında çekirdek, uygulamayı sinyal veya geri çağırma (callback) yoluyla bilgilendirir.

Çalışma Akışı:

- Uygulama, aio_read() gibi bir asenkron I/O işlemi başlatır.
- Uygulama diğer işlemlerine devam eder.
- Çekirdek, I/O işlemi tamamlandığında uygulamayı (sinyal veya geri çağırma yoluyla) bilgilendirir.

Kullanım Alanları:

- Yüksek performans gerektiren ve sürekli olarak I/O işlemlerine bağımlı olmayan uygulamalar.
- I/O işlemleri sırasında diğer işlerin yapılmasının yararlı olduğu senaryolar.

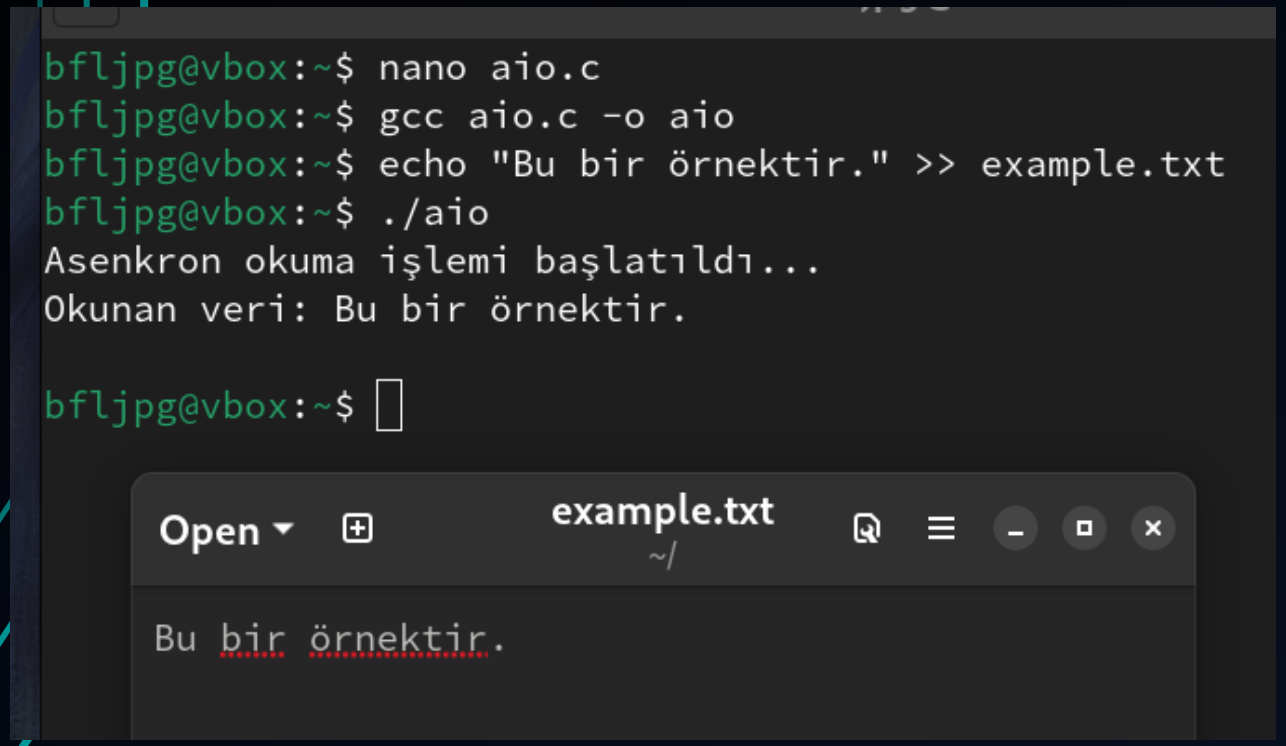
```c
#include <stdio.h>
#include <aio.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>

// Asenkron okuma tamamlandığında çalışacak fonksiyon
void aio_completion_handler(sigval_t sigval) {
    struct aiocb *req = (struct aiocb *)sigval.sival_ptr; // İstek bilgisi
    if (aio_error(req) == 0) {
        ssize_t bytesRead = aio_return(req); // Okunan byte sayısını al
        printf("Okunan veri: %.*s\n", (int)bytesRead, (char *)req->aio_buf); // Veriyi ekrana yazdır
    } else {
        perror("Asenkron okuma hatası"); // Hata kontrolü
    }
}

int main() {
    struct aiocb cb; // Asenkron I/O kontrol bloğu
    char buffer[100]; // Veriyi tutacak bir buffer

    memset(&cb, 0, sizeof(struct aiocb)); // Kontrol bloğunu sıfırla
    cb.aio_fildes = open("example.txt", O_RDONLY); // Dosyayı aç
    if (cb.aio_fildes == -1) {
        perror("Dosya açılamadı"); // Hata kontrolü
        return 1;
    }
```

```
28
29      cb.aio_buf = buffer; // Okunan verinin adresini ayarla
30      cb.aio_nbytes = sizeof(buffer); // Okunacak byte sayısını ayarla
31      cb.aio_offset = 0; // Başlangıç ofsetini ayarla
32
33      // Sinyal işleyicisi ayarları
34      cb.aio_sigevent.sigev_notify = SIGEV_THREAD;
35      cb.aio_sigevent.sigev_notify_function = aio_completion_handler;
36      cb.aio_sigevent.sigev_notify_attributes = NULL;
37      cb.aio_sigevent.sigev_value.sival_ptr = &cb;
38
39      // Asenkron okuma işlemini başlat
40      if (aio_read(&cb) == -1) {
41          perror("Asenkron okuma başlatılamadı"); // Hata kontrolü
42          close(cb.aio_fildes);
43          return 1;
44      }
45
46      printf("Asenkron okuma işlemi başlatıldı...\n");
47      sleep(2); // Başka işlemler yapılabilirken bekleniyor
48
49      close(cb.aio_fildes); // Dosyayı kapat
50      return 0;
51  }
52
```



- Derleyip çalıştıktan sonra yandaki ekran görüntüsüne benzer bir çıktı göreceksiniz.
- Başarıyla Asynchronous I/O modeli programla, oluşturduğumuz dosya okundu ve terminale yazdırdı.

Özet ve Karşılaştırma

Beş farklı I/O modeli, I/O işlemlerini ele alma konusunda farklı yaklaşımlar sunar ve her birinin kendine göre avantajları ve dezavantajları vardır:

- Engelleyici I/O, sadedir ancak tüm süreci bloke edebilir, bu yüzden yüksek yanıt verme gerektiren uygulamalar için uygun değildir.
- Engellemeyen I/O, sürecin yanıt vermeye devam etmesine izin verir ancak kontrol etme veya periyodik olarak yeniden deneme ihtiyacı nedeniyle karmaşıklık katabilir.
- I/O Çoklama, birden fazla I/O kaynağını ele almak için etkilidir ancak dikkatli yönetilmezse bloklama yaşayabilir.
- Sinyal Tabanlı I/O, sinyaller aracılığıyla hızlı bildirim sağlar ancak sinyal işleme ve yönetimini karmaşıklaştırabilir.
- Asenkron I/O, en yüksek verimliliği sunar ve sürecin beklemeden devam etmesine olanak tanır ancak daha karmaşık bir yapılandırma ve yönetim gerektirir.

Uygun I/O modelini seçmek, uygulamanın gereksinimlerine, çalışma ortamına ve ihtiyaç duyduğu kullanım senaryolarına bağlıdır. Bu modelleri anlamak, geliştiricilerin daha verimli, hızlı yanıt veren ve ölçeklenebilir uygulamalar tasarlamalarına yardımcı olur.

Sinyaller

- Sinyal mekanizması, sistemde yeni bir olay (**event**) oluştuğunda çalışan uygulamaların asenkron biçimde haberdar edilebilmesine olanak verir. Bu yapısı nedeniyle sinyal mekanizması genellikle yazılım tabanlı kesmeler (*software interrupt*) şeklinde de adlandırılır. Tıpkı donanım kesmelerinde olduğu gibi (*hardware interrupt*) sinyaller de bir uygulamanın normal akışını kesintiye uğratır ve uygulamanın ne zaman bir sinyal alacağı önceden bilinemez.

- Sinyaller 3 temel durumda oluşturulur:
- Donanım tarafında istisnai bir durum oluştuğunda sinyal üretilir. Örnek olarak uygulamanın kendi izin verilen adres alanının dışındaki bir bölgeye erişmeye çalışması (Segmentation Fault), sıfıra bölme işlemi içeren bir makine kodu üretilmesi vb. gösterilebilir.
- Kullanıcı tarafından konsolda CTRL-C veya CTRL-Z gibi tuş kombinasyonlarının kullanımı, konsol ekranının yeniden boyutlandırılması ya da kill uygulaması ile sinyal gönderim isteğinin oluşturulması
- Uygulama içerisinde kurulan bir timer'ın dolması, uygulamaya verilen CPU limitinin aşılması, açık bir file descriptor'e veri gelmesi vb.

- Sinyal kavramı Unix'in ilk versiyonlarından beri bulunmaktadır. Önceleri sinyal işlemleriyle ilgili Unix versiyonları arasında çeşitli farklılıklar mevcuttu. Sonradan sinyal yönetimi için POSIX standardizasyonu yapıldı ve Linux dahil diğer Unix türevleri de bu standartları takip etti. Bu nedenle kimi dokümanlarda karşılaşabileceğiniz Unix Sinyalleri ve POSIX Sinyalleri kavramları aradaki farklılıklara işaret etmektedir.

Temel Kavramlar
Sinyal Numaraları

- Sinyaller sistemde 1'den başlamak suretiyle nümerik değerlerle gösterilirler.
- Örnek olarak 1 nolu sinyal, hemen her sistemde HUP sinyali olarak değerlendirilirken 9 nolu sinyal de KILL sinyali olarak bilinir.
- Bununla birlikte uygulamalarımızda sinyal kullandığımızda bu rakamların kullanılması kesinlikle önerilmez. POSIX sinyalleri için uygulama içerisinde <signal.h> dosyası include edilmeli ve SIGHUP, SIGKILL gibi ilgili numaralara ait sabit tanımları kullanılmalıdır.
- Sisteminizdeki /usr/include/signal.h dosyasını inceleyecek olursanız, dosya içerisinde __USE_POSIX, __USE_XOPEN, __USE_POSIX199309 gibi değerlerin tanımlarına bakılarak yapılan ek işlemleri ve include edilen diğer dosyaları görebilirsiniz. Linux sistemlerdeki sinyal numaralarını uygulama kodunuzda doğrudan include etmeniz gerekmeyen /usr/include/asm-generic/signal.h dosyasında bulabilirsiniz.

Sinyal Üretimi ve Gönderimi

- Sinyal üretimi, bir olay nedeniyle gerçekleşir. Ancak sinyalin ilgili uygulamaya gönderilmesi (deliver) işlemi, sinyalin üretimi ile aynı anda olmaz.
- Sinyalin uygulamaya gönderilebilmesi için uygulamanın o an CPU kaynağına sahip ve Running durumda çalışıyor olması gereklidir. Dolayısıyla belirli bir uygulamaya gönderilmek üzere üretilen bir sinyalin gönderimi, ilgili uygulamanın context-switch sonrası tekrar çalışmaya başlamasıyla birlikte gerçekleşir.

Bekleyen (Pending) Sinyal Kavramı

- Sinyalin üretim anından gönderilme işlemi gerçekleşene kadarki süre zarfında, sinyaller pending olarak işaretlenir ve beklemede tutulur.
- Bir process için bekleme durumundaki sinyal sayısı ve izin verilen pending sinyal sayısına, proc dosya sisteminden ilgili process id (PID) dizini altındaki /proc/PID/status dosyasında SigQ: 2/15235 örneğindeki gibi erişilebilir.

Sinyal Maskeleri ve Bloklama

- Sinyallerin asenkron doğası nedeniyle uygulama tarafından ne zaman geleceğinin öngörülemez olması, uygulamada kritik bir işlem yapılıyorken kesintiye uğrama riskini de beraberinde getirir.
- Bu şekildeki istenmeyen durumların önüne geçebilmek için sinyal maskeleri kullanarak kritik bir işlem öncesi bazı sinyalleri bloklama, kritik olan kısım tamamlandıktan sonra da tanımlanmış blokları kaldırma imkanı sağlanmıştır. Bu işlem uygulama geliştiricinin sorumluluğundadır.
- Uygulama tarafından bir sinyal bloklandığında, blok kaldırılana kadar üretilen aynı tipteki diğer sinyaller bekleme (pending) durumunu korur. Uygulamada blok kaldırıldığın anda bekleyen sinyallerin gönderimi de sağlanır.
- Bu şekilde blok anında beklemeye geçen sinyallerden aynı tipte olanlar, normal kullanımda blok kaldırıldıktan sonra uygulamaya sadece 1 defa gönderilir. Realtime sinyallerde ise durum farklıdır.

Sinyal Türleri

- Sinyal türlerine göre öntanımlı eylemler değişkenlik gösterebilmektedir. Eğer uygulama içerisinde ilgili sinyali karşılayan bir *signal handler* fonksiyonu tanımlanmamış ise, öntanımlı eylem gerçekleşecektir. Bu bazen uygulamanın sonlandırılması bazen de sinyalin gözardı edilmesi anlamına gelir.
- Bazı sinyallerin ise uygulama katmanında yakalanması mümkün değildir, bu tarz sinyaller her zaman öntanımlı eylemi gerçekleştirirler (**KILL** sinyali gibi).
- Uygulamanın sonlanmasına yol açan bazı eylemlerde ek olarak **core dump** dosyası da üretilir. Core dump dosyaları, ilgili process'in sanal bellek tablosunun diske yazılması suretiyle, sonraki aşamalarda debug araçları ile process'in sonlanmadan önceki durum bilgisinin incelenebilmesi amacıyla oluşturulur.

Sinyal	No	Öntanımlı Eylem	Yakalanabilir mi?
SIGHUP	1	Uygulamayı sonlandır	Evet
SIGINT	2	Uygulamayı sonlandır	Evet
SIGQUIT	3	Uygulamayı sonlandır (core dump)	Evet
SIGILL	4	Uygulamayı sonlandır (core dump)	Evet
SIGTRAP	5	Uygulamayı sonlandır (core dump)	Evet
SIGABRT	6	Uygulamayı sonlandır (core dump)	Evet
SIGFPE	8	Uygulamayı sonlandır (core dump)	Evet
SIGKILL	9	Uygulamayı sonlandır	Hayır
SIGBUS	10	Uygulamayı sonlandır (core dump)	Evet
SIGSEGV	11	Uygulamayı sonlandır (core dump)	Evet
SIGSYS	12	Uygulamayı sonlandır (core dump)	Evet
SIGPIPE	13	Uygulamayı sonlandır	Evet
SIGALRM	14	Uygulamayı sonlandır	Evet
SIGTERM	15	Uygulamayı sonlandır	Evet

SIGUSR1	16	Uygulamayı sonlandır	Evet
SIGUSR2	17	Uygulamayı sonlandır	Evet
SIGCHLD	18	Yoksay	Evet
SIGTSTP	20	Durdur	Evet
SIGURG	21	Yoksay	Evet
SIGPOLL	22	Uygulamayı sonlandır	Evet
SIGSTOP	23	Durdur	Hayır
SIGCONT	25	Durdurulmuşsa sürdür	Evet
SIGTTIN	26	Durdur	Evet
SIGTTOU	27	Durdur	Evet
SIGVTALRM	28	Uygulamayı sonlandır	Evet
SIGPROF	29	Uygulamayı sonlandır	Evet
SIGXCPU	30	Uygulamayı sonlandır (core dump)	Evet
SIGXFSZ	31	Uygulamayı sonlandır (core dump)	Evet

Sinyal Yakalama ve Gönderme
Sinyal Yakalama

- Uygulama tarafından yakalanabilmesine izin verilen sinyaller, ilgili fonksiyonlar kullanılarak uygulama içerisinde yakalanıp işlenecek hale getirilir. Bunun için temel olarak, yakalamak istediğimiz sinyalleri belirlemek ve bu sinyalleri işleyecek callback fonksiyonlarını hazırlamamız gerekir.
- Sinyal işleme sürecinde aynı callback fonksiyonunu birden fazla sinyal için kullanmamız da mümkündür. Asenkron olarak çağırılacak callback fonksiyonunun prototipi aşağıdaki gibidir:

```
typedef void (*sighandler_t)(int);
```

- Yakalamak istediğimiz sinyal ve karşılayacak callback fonksiyonunu belirtmek için signal veya sigaction fonksiyonları kullanılır. Kullanım prototipleri aşağıdaki gibidir:

```
sighandler_t signal(int signum, sighandler_t handler);


int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);


struct sigaction {
    void        (*sa_handler)(int);
    void        (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t    sa_mask;
    int         sa_flags;
    void        (*sa_restorer)(void);
};
```

- signal kullanımı artık önerilmemektedir, bunun yerine daha fazla kontrol sunan sigaction kullanımı tercih edilmelidir. Zaten günümüzde glibc içerisindeki signal gerçekleştirimi de sigaction fonksiyonunu sarmalayan bir yapıdan ibarettir.
- Bir sonraki slayttaki örnek uygulamayı signalornek.c adında kaydediniz.
- Sonrasında
  - gcc signalornek.c –o signalornek
  - ./signalornek

  komutlarıyla derleme işlemini yapıp çalıştıralım.

```c
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

static int exit_flag = 0;  // Programın bitiş koşulunu kontrol eden bayrak

// Sinyal yakalama fonksiyonu (signal handler)
static void handler (int signum) {
    if (signum == SIGTERM) {
        printf("TERM signal received, exiting program\n");
        exit_flag = 1;  // SIGTERM alındığında programdan çıkmak için bayrağı set et
    } else {
        // Diğer sinyalleri ekrana yazdır
        printf("Not a TERM signal: %d (%s)\n", signum, strsignal(signum));
    }
}

int main (int argc, char *argv[]) {
    struct sigaction act;
```

```c
    // `act` yapısını sıfırla ve sinyal yakalama fonksiyonunu ayarla
    memset(&act, '\0', sizeof(act));
    act.sa_handler = &handler;

    // SIGTERM sinyalini yakalamak için sigaction kullan
    if (sigaction(SIGTERM, &act, NULL) < 0) {
        perror ("sigaction");  // Eğer sigaction başarısız olursa hata mesajı ver
        return 1;
    }

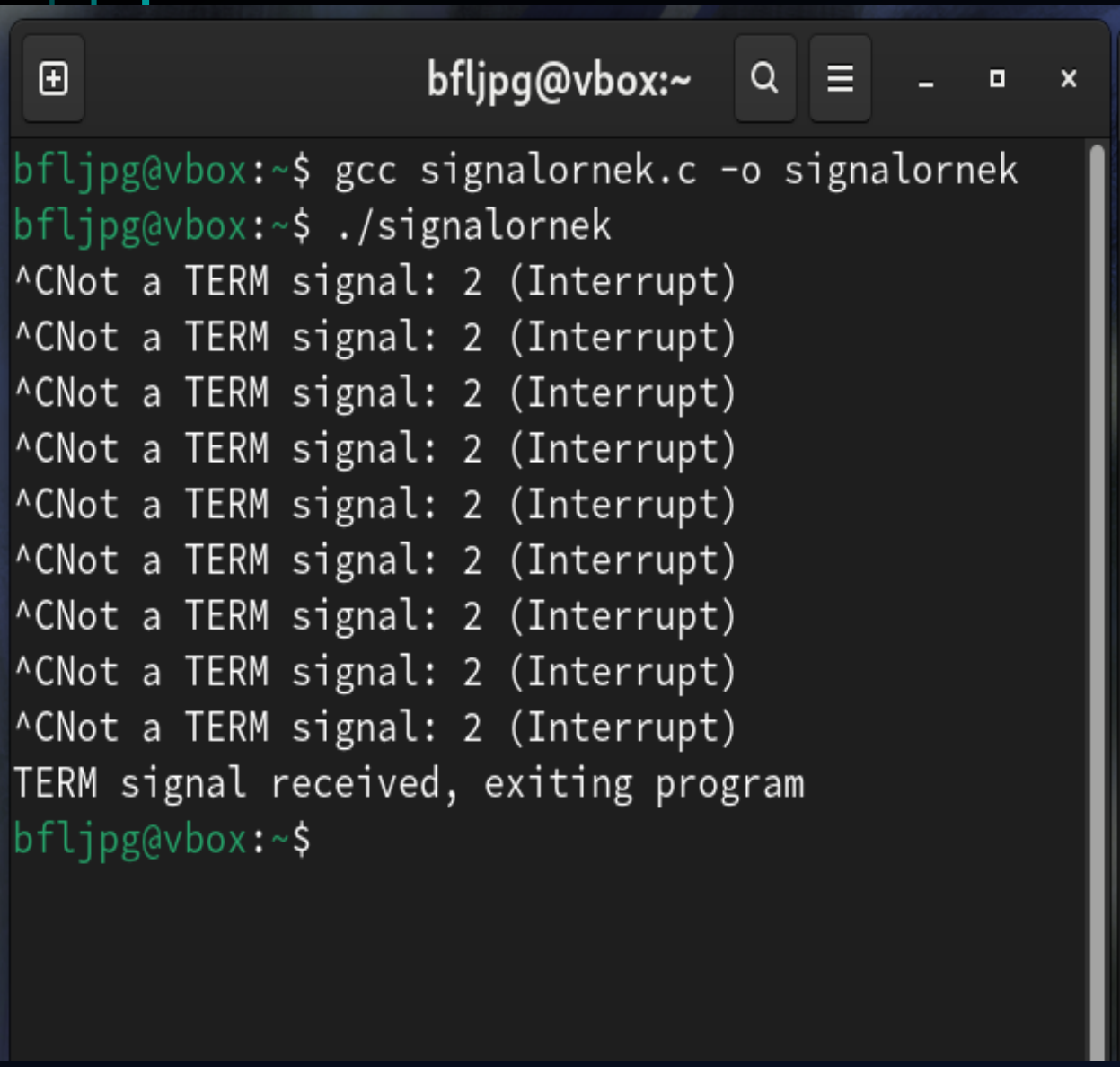
    // SIGINT (Ctrl+C) sinyalini de aynı handler ile yakala
    if (sigaction(SIGINT, &act, NULL) < 0) {
        perror ("sigaction");
        return 1;
    }

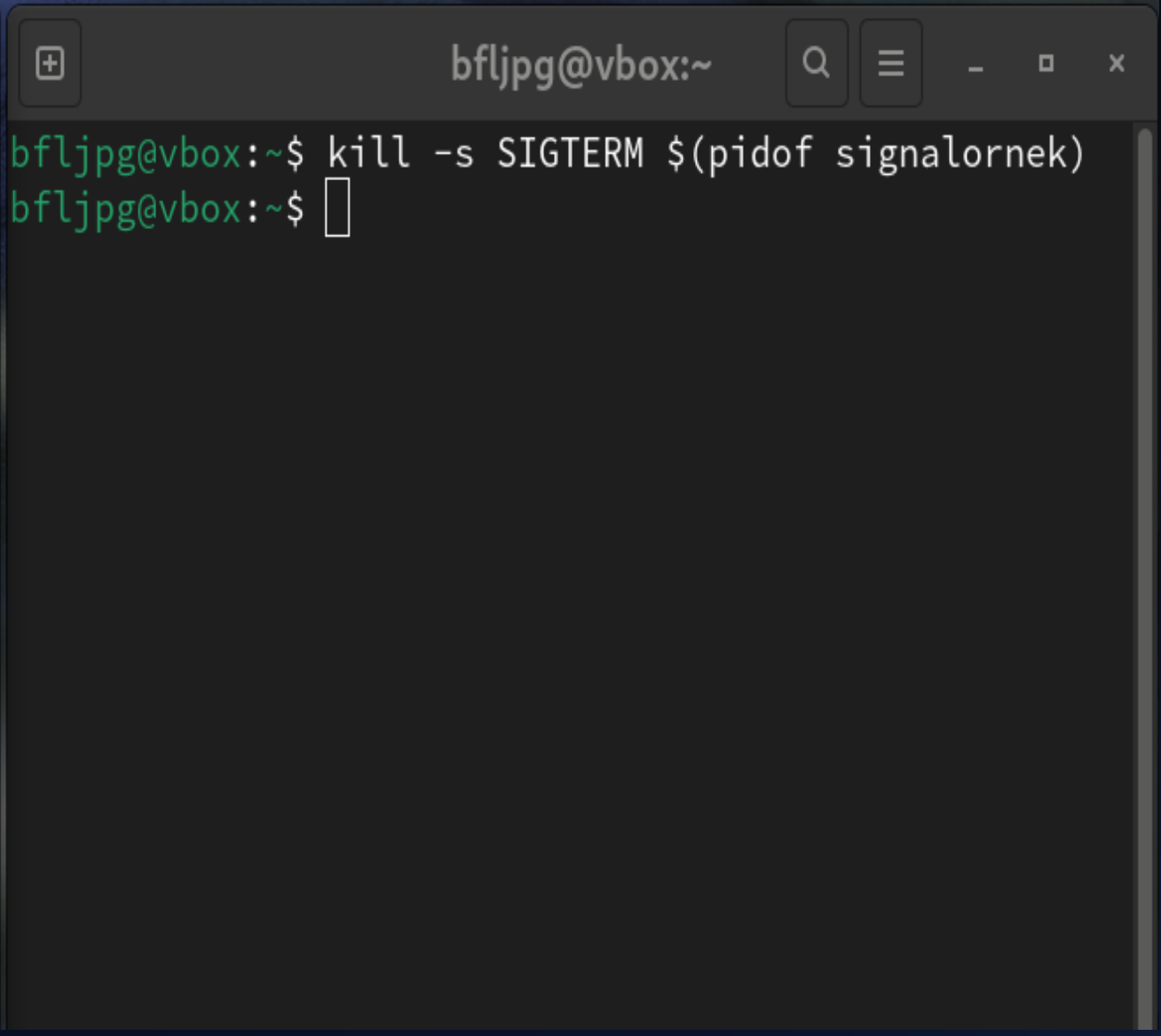
    // Program sonlanana kadar bekle (exit_flag 1 olana kadar)
    while (!exit_flag) usleep(100000);  // 100ms uyuyarak CPU yükünü azalt

    return 0;
}
```

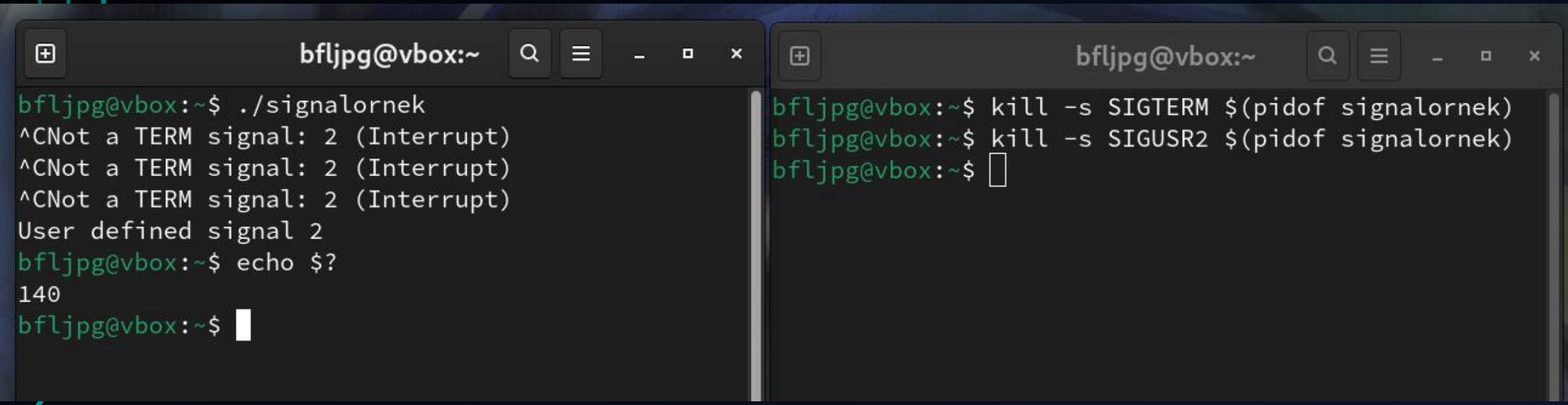
- Uygulamayı çalıştırdıktan sonra CTRL-C tuş kombinasyonu ile INT sinyali ürettiğimizde sigaction ile belirttiğimiz callback fonksiyonumuzun devreye girdiğini, TERM sinyali dışında bir sinyal geldiği için uyarı mesajı verip çalışmasına devam ettiğini görmekteyiz.

- Başka bir konsoldan uygulamamıza kill komutu ile TERM sinyalini gönderdiğimizde ise beklediğimiz gibi exit_flag değişkeninin değerinin 1 yapılmak suretiyle sonlandığını görmekteyiz. Bu şekilde uygulama sonlandığında çıkış değerinin de $? kabuk değişkeninde beklediğimiz üzere 0 olduğunu görüyoruz:


bfljpg@vbox:~
bfljpg@vbox:~$ gcc signalornek.c -o signalornek
bfljpg@vbox:~$ ./signalornek
^CNot a TERM signal: 2 (Interrupt)
^CNot a TERM signal: 2 (Interrupt)
^CNot a TERM signal: 2 (Interrupt)
^CNot a TERM signal: 2 (Interrupt)
^CNot a TERM signal: 2 (Interrupt)
^CNot a TERM signal: 2 (Interrupt)
^CNot a TERM signal: 2 (Interrupt)
^CNot a TERM signal: 2 (Interrupt)
^CNot a TERM signal: 2 (Interrupt)
TERM signal received, exiting program
bfljpg@vbox:~$


bfljpg@vbox:~
bfljpg@vbox:~$ kill -s SIGTERM $(pidof signalornek)
bfljpg@vbox:~$

- Bu noktada şu soruyu kendimize soralım: Test uygulamamız çalışıyorken diğer konsoldan uygulama içerisinde işlediğimiz TERM ve INT sinyalleri dışında, örneğin USR2 sinyalini gönderirsek ne olur?



- Yukarıda görüldüğü üzere, uygulamamız **USR1** sinyalini aldığında, *User defined signal 2* mesajıyla sonlandı. Bu bizim uygulama içerisinden bastırdığımız bir mesaj değil. Ayrıca uygulamanın çıkış değeri önceki örnekte olduğu gibi **0** değil, **140** oldu.

Sinyal Gönderme

- Uygulama içerisinden sinyal göndermek için prototipi aşağıda belirtilen kill fonksiyonu kullanılır: `int kill(pid_t pid, int sig);`
- Fonksiyon basitçe parametre olarak verilmiş olan PID değerine sahip uygulamaya istenen sinyalin gönderimini sağlamaktadır. İşlemin gerçekleşebilmesi için ilgili uygulamaya sinyal gönderebilme yetkisinin olması gerekir. Örnek olarak A kullanıcısı B kullanıcısının uygulamalarına sinyal gönderemez, ancak kendi sahip olduğu uygulamalara sinyal gönderebilir.
- Sistemdeki root kullanıcısı veya Linux Capabilities API üzerinden CAP_KILL kabiliyetine sahip olan uygulamalar ise herhangi bir uygulamaya sinyal gönderebilirler.

Signal-Safe Kavramı

- Unix türevi sistemlerde iyi uygulama yazmanın temel kurallarından biri signal safety ve thread safety kavramlarının anlaşılmasıdır.

- Signal-Safe kavramı yoğunlukla Async-Signal-Safe biçiminde de ifade edilir.

Reentrancy

- Konuya geçmeden önce reentrant kod kavramını incelememizde fayda var. Bir fonksiyon eğer aynı anda birden fazla task içinden çağrılsa da içerde kullanılan/üretilen verilerde herhangi bir bozulmaya yol açmayacak şekilde dizayn edilmiş ise, reentrant olduğunu söyleyebiliriz.

- Dolayısıyla reentrant bir fonksiyonun karakteristik özelliklerini aşağıdaki gibi sıralayabiliriz:
  - Farklı çağrımlarında statik bir alan üzerinde değişiklik yapmaya çalışmaz
  - Sadece yerel değişkenler kullanır veya global bir değişken kullanımı zorunlu ise, yerel bir kopyasını çıkartarak işlemlerde yerel kopyasını kullanır
  - Geri dönüş değeri olarak asla herhangi bir statik alanın adresini dönmez
  - Fonksiyon içerisinden reentrant olmayan başka bir fonksiyon çağrılmaz

- Bir thread-safe fonksiyon ise aynı anda birden fazla thread içerisinden, paylaşımlı bir alan kullanılsa dahi güvenle çağrılabilir. Paylaşımlı alan kullanımı söz konusu ise, ilgili alana erişimin thread-safe fonksiyonlarda gerekli kilit mekanizmalarıyla sıralı halde yapılması garanti edilir.

- Bir fonksiyon eğer thread-safe ise aynı zamanda reentrant'dır.

Sinyal Kesmeleri

- Sinyaller donanım kesmelerine benzer şekilde yazılımın akışını bir anda kesip ayrı bir yere dallanma (sinyal işleyici fonksiyonu) sonra kaldığı yerden devam etme özelliğine sahiptir. Tek işlemcili ve tek bir thread ile çalışan uygulamalarda dahi, signal-safety önemli bir problemdir.
- Konunun daha iyi anlaşılması için görece zararsız örneklerden yola çıkalım. Örnek olarak uygulamanızın belirli bir bölümünde printf fonksiyonu ile konsola bir çıktı gönderdiğinizi varsayalım. Eğer printf fonksiyonu henüz çalışmasını bitirmemişken işlemin yarısında uygulamaya sinyal gelirse ve sinyalin işlendiği fonksiyon içerisinde de ayrı bir printf fonksiyonu çalışıyorsa, sinyalin işlemi tamamlanıp asıl uygulamaya geri dönülüp ilerlendiğinde konsoldaki çıktılar birbirine karışacaktır.

- Şimdi daha tehlikeli bir senaryoya göz atalım. Uygulamanızda malloc() ile heap alanından bir miktar bellek talep ettiğinizi düşünelim. İşlem henüz tamamlanmamışken bir sinyal ile kesintiye uğrar ve sinyal işleyici fonksiyonumuzda doğrudan veya dolaylı olarak malloc() fonksiyonunu çağırırsak ne olur?

- Malloc işlemi performans açısından uygulama içerisindeki allokasyonları bağlı liste yapıları ile tutar. Bu liste yapısı güncelleniyorken sinyal nedeniyle yeniden çağrılması kritik hatalara yol açabilecektir.

Sinyal Maskeleme

- Signal-safe olmayan fonksiyonların kullanımına yönelik 2 temel çözüm bulunur:
  - signal-safe olmayan fonksiyonların kullanıldığı bölümlerde sinyalleri maskelemek
  - sinyallerin işlendiği callback fonksiyonlarında signal-safe olmayan fonksiyonların kullanımından kaçınmak
- Maskeleme işlemleri için sinyal kümeleri ve bloklama konularına geçelim.

Sinyal Kümeleri

- Sinyal kümeleri, sinyal bloklama gibi işlemlerde tek bir küme ile birden fazla sinyali gösterebilmek için kullanılır.
- Kümeler glibc içerisinde sigset_t tipiyle tutulur.
- Bir sinyal kümesini ilklendirmek için 2 yol bulunur:
  - sigemptyset(sigset_t *set) fonksiyonu ile hiç sinyal bulunmayan boş bir küme oluşturmak
  - sigfillset(sigset_t *set) fonksiyonu ile tüm sinyallerin dahil olduğu bir küme oluşturmak
- Sinyal kümeleri ile çalışırken mutlaka bu iki fonksiyondan biri kullanılarak kümenin ilklendirilmesi gerekir.

- Bir sinyal kümesine belirli bir sinyali eklemek için sigaddset (sigset_t *set, int signum) fonksiyonu kullanılır.

- Bir sinyal kümesinden belirli bir sinyali çıkarmak içinse sigdelset (sigset_t *set, int signum) fonksiyonu kullanılmaktadır.

- Belirli bir sinyalin küme içindeki varlığını test etmek içinse sigismember (const sigset_t *set, int signum) fonksiyonu kullanılmaktadır.

Sinyal Bloklama

- Sinyalleri bloklama ihtiyacı temel olarak aşağıdaki 2 durumda ortaya çıkmaktadır:
  - Belirli bir sinyal işleyici fonksiyon içerisinde kritik bir işlem yapılıyorsa, farklı bir sinyal ile kesintiye uğramamak için
  - Uygulama içerisinde aynı zamanda sinyal işleyici fonksiyon üzerinden de değiştirilmesine izin verilen bir global değişkeni değiştiriyorsak, sinyal ile kesintiye uğramamak için
- Not: İşletim sistemi sinyal nedeniyle ilgili uygulamadaki sinyal için tanımlı fonksiyonu çalıştırmayı başlatırken süreç tamamlanmadan aynı tipte gelen sinyallerde, sinyal işleyici fonksiyonu kesintiye uğratmaz. Ancak fonksiyon çalışmasına devam ederken farklı tipte bir sinyal gelirse, sinyal işleme fonksiyonu kesintiye uğrayacaktır.

Sinyal ile Veri Gönderimi

- Sinyaller, slaytta da anlatıldığı üzere genellikle süreçler arasında olayların bildirilmesi için kullanılır. Ancak, standart sinyaller yalnızca belirli bir olayın gerçekleştiğini haber verirken, realtime sinyaller veri iletimi yeteneği sunar. Bu bölümde, sinyal ile veri gönderiminin nasıl yapıldığını, standart ve realtime sinyaller arasındaki farkları ve sigqueue() gibi araçlarla veri iletimi yapmanın yöntemlerini detaylı bir şekilde ele alacağız.

RealTime Sinyaller

- Gerçek zamanlı sinyaller, orijinal sinyallerle benzerlik gösterir; çünkü sinyal numaralarına sahiptirler ve orijinal sinyal işlevleri (örneğin «kill()») kullanılarak işlenebilirler.
- Gerçek zamanlı sinyaller, signal.h dosyasındaki iki sabit olan SIGRTMIN ve SIGRTMAX arasındaki sinyal numaralarına sahip olan sinyallerdir.
- GNU C kütüphanesi ile, SIGRTMIN aslında bir fonksiyon çağrısıdır, iş parçacığı kütüphanesine göre bazı gerçek zamanlı sinyalleri kendi kullanımı için rezerve edebilir. Bu durum işletim sistemlerine göre farklılık gösterebilir.

- SIGRTMIN ve SIGRTMAX arasındaki sinyallerin standart bir amacı yoktur ve bir program bunları istediği şekilde kullanabilir. POSIX, en az 8 gerçek zamanlı sinyalin mevcut olmasını zorunlu kılar. Linux'ta, 32 ile 63 arasındaki sinyal numaraları kullanılabilir, ancak bir önceki sayfada belirtildiği gibi, bazıları (şu anda 2, geçmişte 3, 32 ile 33 veya 34 arasında) GNU C kütüphanesinin iş parçacığı desteği tarafından kullanılabilir.
- Bu durum eskiden geçerli olduğunu belirtmemiz gerek ve şu anda halihazırda bu gerekli sinyallerin artık sürekli olarak rezerve edilmiş olabileceği düşünülüyor.

Teslim Önceliği

- Daha düşük numaralı sinyaller, daha yüksek numaralı sinyallere göre önceliklidir. Linux'ta bu, tüm sinyallerin aralığında geçerlidir, bu da normal sinyallerin, tüm gerçek zamanlı sinyallerden daha yüksek önceliğe sahip olduğu anlamına gelir (POSIX, gerçek zamanlı sinyallerin sıradan sinyallerle karşılaştırıldığında önceliğinin belirsiz olduğunu söyler ve yalnızca gerçek zamanlı sinyallerin birbirlerine göre belirtilmiş bir önceliği vardır).

Sinyal Kuyruğu

- Linux'taki normal sinyaller ile gerçek zamanlı sinyaller arasındaki temel farklardan biri, bireysel gerçek zamanlı sinyallerin birden fazla örneğinin kuyruğa alınabilmesidir (ve aslında mümkünse her zaman kuyruğa alınır) ve bunun yerine zaten bekleyen bir örnekle birleştirilmezler. Normal (gerçek zamanlı olmayan) sinyaller ise, bir sinyal zaten bekliyorsa, birleştirilir, bu da GNU libc el kitabında açıklanmıştır; ancak bu her işletim sistemi için geçerli değildir.
- Linux'ta, kuyruklanan sinyallerin sayısına dair sistem çapında bir limit vardır. Bu, bir hizmet reddi saldırısının kaynağı olabilir. Ancak, bir kullanıcının biriken sinyalleri kontrol etmek için bir kullanıcı başına kaynak limiti vardır.

Ek Veriler ve Sinyal Alımı

- Başka bir fark, normal sinyaller ile gerçek zamanlı sinyaller arasındaki farklardan biri, gerçek zamanlı sinyallerin ek sinyal verisi taşıyabilmesidir. Bu verilerin tam içeriği, sinyalin türüne ve kaynağına bağlıdır.
- Gerçek zamanlı sinyalle gönderilmiş bir ek veriyi programda işleyebilmek için yeni bir «struct sigaction» yapı bulunuyor.
- Fonksiyon gösterimi: void (*sa_sigaction)(int, siginfo_t *, void *);

Sinyal Kuyruğa Alma

- sigqueue fonksiyonu, bir süreç (process) için belirli bir sinyali kuyruklamak ve bu sinyalle birlikte ek veri göndermek için kullanılır. Bu, kill fonksiyonuna benzer bir işlev görür ancak sigqueue, ek veri gönderme yeteneğine ve daha fazla kontrol imkanına sahiptir. Fonksiyon gösterimi ise şu şekildedir:
- int sigqueue(pid_t pid, int signo, const union sigval value);

Parametreler

signo:

- Gönderilecek sinyal numarası. Bu, genelde standart sinyallerden (SIGKILL, SIGUSR1, vb.) biri veya gerçek zamanlı sinyallerden biri (SIGRTMIN + n) olur.

value:

- Gönderilecek ek veriyi içeren bir union sigval yapısı. Bu yapı iki üyeye sahiptir:
- sival_int: Basit bir tamsayı verisi.
- sival_ptr: Bir bellek adresine işaret eden bir işaretçi.

pid:

- Hedef sürecin işlem kimliği (Process ID).
- Eğer pid > 0 ise, belirtilen PID'ye sahip tek bir sürece sinyal gönderilir.
- Eğer pid == 0 ise, sinyal gönderen sürecin süreç grubundaki tüm süreçlere sinyal gönderilir.
- Eğer pid == -1 ise, sinyal tüm süreçlere gönderilir (izin verilenler hariç).
- Eğer pid < -1 ise, -pid süreç grubundaki tüm süreçlere sinyal gönderilir.

Dönüş Değeri

- 0: Sinyal başarıyla gönderildi.
- -1: Bir hata oluştu; errno değişkeni hata nedenini belirler.

Hata Kodları

- EAGAIN: Sistem kuyruk kapasitesi dolu, sinyal kuyruklanamıyor.
- EACCESS: Sinyali gönderen süreç, hedef sürece sinyal gönderme yetkisine sahip değil.
- EINVAL: Geçersiz bir sinyal numarası (signo).
- ESRCH: Belirtilen PID'ye sahip bir süreç yok.

Kullanımı ve Özellikleri

- sigqueue, gerçek zamanlı sinyalleri kuyruğa alabilir ve ek veri gönderebilir. Bu, standart sinyallerde mümkün değildir.
- Gerçek zamanlı sinyaller birden fazla kez kuyruğa alınabilir (Linux üzerinde). Standart sinyaller ise sadece bir kez kuyruğa alınır; yeni bir sinyal aynı türde önceki sinyalle birleştirilir.
- Kuyruğa alınan sinyaller, hedef sürece gönderilir ve sigaction ile tanımlanan bir sinyal işleyicisi tarafından işlenebilir.

Sinyal Kuyruğu ve RealTime Sinyaller ile ilgili Örnek

- Bu örnekte, sigaction ile bir sinyal işleyicisi (sa_sigaction) belirleyeceğiz ve sinyali kuyruğa alacağız. Ek olarak, sinyalle birlikte ek veri göndereceğiz.
- Bir sonraki sayfada, SIGRTMIN sinyalini işleyen bir program örneği bulunmaktadır.
- sigaction ile bir sinyal işleyicisi (sa_sigaction) tanımlıyoruz. SA_SIGINFO bayrağını ayarlayarak sinyali daha fazla bilgiyle alıyoruz.
- sig_handler fonksiyonu, sinyali aldığında, sinyalin numarasını ve ek verileri (eğer varsa) yazdırır.
- sigqueue fonksiyonu ile, SIGRTMIN sinyalini ve ek veri (burada 42 değeri) gönderiyoruz.
- Program, sinyali aldıktan sonra ek veri ile ilgili bilgiyi yazdırır.

```c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <string.h>

void sig_handler(int signo, siginfo_t *info, void *context) {
    // Sinyal numarasını yazdır
    printf("Received signal: %d\n", signo);

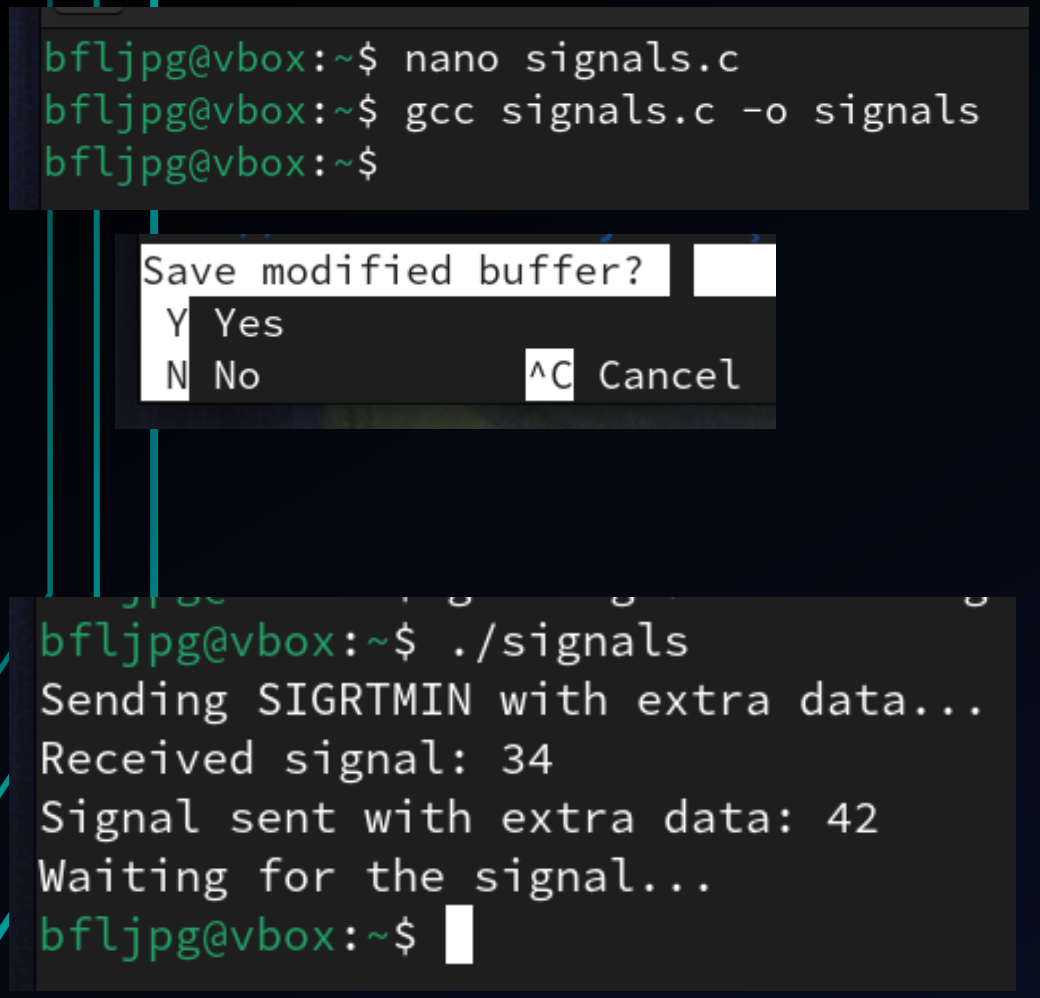
    // Ek veri kontrolü
    if (info->si_code == SI_QUEUE) {
        printf("Signal sent with extra data: %d\n", info->si_value.sival_int);
    } else {
        printf("Signal sent by user\n");
    }
}

int main() {
    struct sigaction sa;

    // Signal handler'ı ayarlıyoruz
    sa.sa_flags = SA_SIGINFO;  // SA_SIGINFO bayrağı kullanarak sa_sigaction kullanacağız
    sa.sa_sigaction = sig_handler;  // Signal handler'ı belirliyoruz

    // SIGRTMIN sinyali için handler'ı ayarla
    if (sigaction(SIGRTMIN, &sa, NULL) == -1) {
        perror("sigaction");
        exit(EXIT_FAILURE);
    }
```

```
31
32      // Bir süre beklemeden önce sinyali gönderelim
33      printf("Sending SIGRTMIN with extra data...\n");
34
35      // Bir işlem numarası (PID) alalım
36      pid_t pid = getpid();
37
38      // sigqueue ile SIGRTMIN sinyalini ve ek veri gönder
39      union sigval value;
40      value.sival_int = 42;  // Ek veri: integer
41
42      if (sigqueue(pid, SIGRTMIN, value) == -1) {
43          perror("sigqueue");
44          exit(EXIT_FAILURE);
45      }
46
47      // Sinyalin alınmasını bekleyelim
48      printf("Waiting for the signal...\n");
49      sleep(2);  // Sinyal alındığında, handler işlevi çalışacak
50
51      return 0;
52  }
```



- İlk resimde görüldüğü gibi signals.c adında dosyamızı oluşturup açarak programı yazmaya başladık. Yazma işlemi bittikten sonra Ctrl+X komutuyla ekrandaki yönergelere göre kaydedip çıktıktan sonra gcc ile derlememizi aldık.
- İkinci resimde derlenen uygulamayı çalıştırdık ve sonuç bu şekilde çıkması gereklidir. Alınan sinyalin 34 olmasının sebebi az önceki slaytlarda da bahsedildiği gibi tanımlanan SIGRTMIN fonksiyonunun sistemlere göre farklı tanımlanabilmesidir. Linux'ta bu değerin 34 olduğunu bu örnekten anlayaiblirsiniz.

Core Dump

Core Dump’ın Amacı:

- Çöken bir programın çalışma zamanındaki durumunu analiz ederek hataları anlamak ve düzeltmek.

Nasıl Çalışır?:

- Bir süreç öldüğünde, işletim sistemi sürecin bellek düzenini (stack, heap, global değişkenler) ve işlemci bağlamını bir dosyada saklar.
- Bu bilgi, programın neden çöktüğünü anlamak için hata ayıklama araçlarıyla incelenir.

Önemli Ayrıntılar:

- Hangi sinyalin core dump oluşturacağı sistem tarafından belirlenmiştir (SIGSEGV, SIGABRT gibi).
- ulimit -c komutuyla core dump dosyasının boyut sınırını kontrol edebilirsiniz.

Pratik Kullanım:

- Core dump, genellikle büyük ölçekli yazılımlarda kritik hataların kökenini anlamak için hayati önem taşır.

Thread Kavramı

- İş parçacığı (thread), bilgisayar biliminde, bir işin eş zamanlı olarak işlenen her bir bölümüdür. İş parçacığı ve işlem arasındaki fark bir işletim sisteminden diğerine değişmekle birlikte genel olarak iş parçacığının oluşturuluşu ve kaynakların paylaşılmasıdır.
- Linux altında thread kullanımı C kütüphanesinin bir parçası olan pthread kütüphanesi ile yapılmaktadır.
- Diğer işletim sistemlerinin aksine, Linux'te thread ile process arasında çok az fark vardır. Bu nedenle thread'ler için aynı zamanda, LightWeightProcess ismi de kullanılabilmektedir.
- Linux versiyon 2.6 öncesindeki thread implementasyonu LinuxThreads şeklinde adlandırılıyordu. Bu implementasyon performans ve senkronizasyon işlemleri açısından önemli limitlere sahipti. Maksimum çalışabilecek thread sayısı 1000'li rakamlarla sınırlıydı.

Native POSIX Thread Library (NPTL)

- 2003 yılında IBM ve RedHat firmasındaki geliştiriciler başta olmak üzere oluşturulan bir ekip, Native POSIX Thread Library (NPTL) projesini kullanılabilir duruma getirmeyi başardı.
- NPTL (Native POSIX Thread Library), Linux sistemlerinde POSIX uyumlu çoklu iş parçacığı (multithreading) desteği sağlamak için kullanılan bir thread kütüphanesidir. 2003 yılında tanıtılan NPTL, eski LinuxThreads kütüphanesinin yerini almıştır. Daha hızlı performans, daha iyi POSIX uyumluluğu ve gelişmiş özellikler sunar.
- NPTL, düşük seviyede işlemciye yakın bir çalışma prensibiyle iş parçacıklarını işletim sistemi süreçleriyle daha iyi entegre eder ve modern uygulamaların ihtiyaç duyduğu verimli paralel işlemeyi destekler.

- İlk olarak enterprise dünyanın Linux üzerinde Java Virtual Machine performansı şikayetlerini çözüme kavuşturmak için RedHat Enterprise versiyon 3'te kullanılmaya başlandı. Ardından GNU C kütüphanesindeki yeni NPTL implementasyonu gelmeye başladı. Günümüzde de her iki implementasyon glibc içerisinde yer almakta, kullanılan çekirdek versiyonu tarafından uygun implementasyon seçilmektedir.
- Her iki implementasyon da thread yönetiminin tamamen kullanıcı kipinde çalışan bir Virtual Machine üzerinde yapıldığı green threads uyarlaması değildir.
- Her program başlatıldığında, bir kernel thread'i de otomatik olarak oluşturulur.
- Çalışan herhangi bir process'in thread spesifik bilgileri /proc/<PID>/task dizini altında yer alır. Single-thread uygulamalar için de bu dizin altında PID ile aynı değere sahip bir task kaydı olduğu görünecektir.
- Bilgisayarınızda çalışan thread implementasyonunu «getconf GNU_LIBPTHREAD_VERSION» komutuyla öğrenebilirsiniz.

Thread Oluşturma

- Yeni bir thread oluşturmak için pthread_create() fonksiyonu kullanılır.
- Thread fonksiyonlarının kullanımı için pthread.h başlık dosyası include edilmelidir.
- Threadler ana program ile aynı adresleme alanını ve aynı file descriptor'ları kullanırlar.
- Pthread kütüphanesi aynı zamanda senkronizasyon işlemleri için gerekli mutex ve conditional işlemleri için gerekli desteği de içermektedir.
- Pthread kütüphanesi fonksiyonları kullanıldığında uygulama pthread kütüphanesi ile de linklenmelidir:

- Fonksiyon gösterimi:
- int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine) (void *), void *arg)
- Başarılı durumda 0 döner. Hata durumunda ise geriye bir hata kodu dönecektir.
- thread parametresi pthread_t türünde olup önceden tanımlanması gerekir. Oluşan thread'e bu referansla her zaman erişilebilecektir.
- attr parametresi thread spesifik olarak pthread_attr_ ile başlayan fonksiyonlarla ayarlanmış, scheduling policy, stack büyüklüğü, detach policy gibi kuralları gösterir.
- start_routine thread tarafından çalıştırılacak olan fonksiyonu gösterir.
- arg ise thread tarafından çalıştırılacak fonksiyona geçirilecek void*'a cast edilmiş genel bir veri yapısını göstermektedir.

Joinable ve Detachable Thread Türleri

- Thread kullanılan bir uygulamada main() fonksiyonundan return edilirse, tüm thread'ler de sonlandırılır ve kullanılan tüm kaynaklar sisteme geri verilir.
- Aynı şekilde herhangi bir thread içerisinden exit() benzeri bir komutla çıkış yapılması halinde gene tüm thread'ler sonlandırılacaktır.
- pthread_join fonksiyonu ile, bir thread'in sonlanmasını bekleyebiliriz. Bu fonksiyonun kullanıldığı thread, sonlanması beklenen thread sonlanana kadar bloklanacaktır.
- Normal (joinable) thread'ler, sonlanmış olsa dahi pthread_join ile join işlemine tabi tutulmazlar ise, CPU tarafından tekrar schedule edilmeseler de sistemden kullandığı kaynaklar geri verilmez.

Detachable Thread

- Bazen pthread_join ile join işlemi yapmanın anlamlı olmadığı, thread'in ne zaman sonlanacağının öngörülemediği durumlar olabilir.
- Bu durumda thread return ettiği noktada tüm kaynakların sisteme otomatik olarak geri verilmesini sağlayabiliriz.
- Bunu sağlamak için ise, ilgili thread'leri, DETACHED durumu ile başlatmamız gerekmektedir.
- Bir thread başlatılırken thread attribute değerleri üzerinden veya pthread_detach fonksiyonu ile DETACH durumu belirtiebilir:

Örnek bir Thread Oluşturalım

- POSIX thread (pthread) kütüphanesini kullanarak çoklu iş parçacıklı bir uygulama örneği sunmaktadır. Programda iki iş parçacığı (thread) oluşturularak her birine farklı bir görev atanır. İş parçacıkları paralel olarak çalışarak, zamanlanmış bir şekilde konsola mesajlar yazdırır. Kod, temel thread oluşturma, fonksiyon atama ve threadlerin bağımsız çalışmasını göstermektedir.
- Amaç: Çoklu iş parçacığı oluşturmayı öğrenmek ve threadlerin bağımsız bir şekilde çalışmasını incelemek.
- pthread_create(), yeni bir thread başlatır ve ona bir işlev ve veri atar.
- usleep(), threadler arasında kısa gecikmeler ekler, böylece çıktılar düzenli gözükür.
- pthread_join, threadlerin kontrollü şekilde bitmesini sağlar.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

// Thread için işlev
void *worker(void *data) {
    char *name = (char*)data;
    for (int i = 0; i < 120; i++) {
        usleep(50000); // 50 ms gecikme
        printf("Hello from thread %s\n", name);
    }
    printf("Thread %s done...\n", name);
    return NULL;
}
```

```c
int main(void) {
    pthread_t th1, th2; // Thread tanımları

    // Thread'leri başlat
    pthread_create(&th1, NULL, worker, "A");
    pthread_create(&th2, NULL, worker, "B");

    // Thread'lerin tamamlanmasını bekle
    pthread_join(th1, NULL);
    pthread_join(th2, NULL);

    // Tüm thread'ler tamamlandıktan sonra mesaj yazdır
    printf("All threads completed. Exiting from main program\n");

    return 0;
}
```

```
bfljpg@vbox:~$ nano thread.c
bfljpg@vbox:~$ gcc thread.c -o thread -lpthread
bfljpg@vbox:~$
```

```
bfljpg@vbox:~$ nano thread.c
bfljpg@vbox:~$ gcc thread.c -o thread -lpthread
bfljpg@vbox:~$ ./thread
Hello from thread A
Hello from thread A
Hello from thread B
Hello from thread A
Hello from thread B
Hello from thread A
Hello from thread B
Hello from thread A
Hello from thread B
Hello from thread A
Hello from thread B
Hello from thread A
Hello from thread B
Hello from thread A
Hello from thread B
Hello from thread A
Hello from thread B
Hello from thread A
Hello from thread B
Hello from thread A
Hello from thread B
Hello from thread A
Hello from thread B
Hello from thread A
```

- Thread ile ilgili uygulamalarda derleme alırken gcc'de «-lpthread» optionını eklemeyi unutmayın. Bu thread kütüphanelerinin etkili olarak kullanılabilmesi için gereken bir seçenek.
- İlk resimde görüldüğü üzere nano ile düzenlememizi yaptık ve gcc ile de derlememizi aldık.
- Uygulamayı çalıştırdığınızda alacağınız sonuç buna benzer olacaktır. Sonsuza kadar devam ediyor gibi çalışması sizi yanıltmasın. Koda daha dikkatli bakarsanız sebebi açık şekilde görünecektir.

Thread Sonlandırma

- pthread_cancel, POSIX thread (pthread) kütüphanesinde bir iş parçacığını (thread) iptal etmek için kullanılan bir fonksiyondur. Ana thread veya başka bir thread, hedef thread'e bir iptal isteği gönderir. İptal isteği, hedef thread'in iptal edilebilir bir durumda olup olmamasına bağlı olarak işlenir.

İptal İsteği Gönderme:

- pthread_cancel ile hedef thread'e bir iptal isteği gönderilir:
- int pthread_cancel (pthread_t thread);

İptal Noktası:

- Bir thread ancak "iptal edilebilir" bir durumdaysa iptal edilir.
- İptal edilebilir fonksiyonlar: pthread_testcancel, sleep, usleep, read, vb.
- Eğer hedef thread, iptal noktası içermiyorsa veya iptal edilebilir değilse, iptal isteği bekletilir.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

// Worker fonksiyonu: Her thread'de çalışan işlev
void *worker(void *data)
{
    char *name = (char*)data;  // Thread'e geçilen veriyi al
    for (int i = 0; i < 120; i++) {
        usleep(50000);  // 50ms gecikme
        printf("Hello from thread %s\n", name);  // Thread'den mesaj yazdır
    }
    printf("Thread %s done...\n", name);  // Thread tamamlandığında mesaj
    return NULL;
}

```

```c
int main(void) {
    pthread_t th1, th2;  // Thread ataması

    // Threadlerin çalıştırılması
    pthread_create(&th1, NULL, worker, "A");
    pthread_create(&th2, NULL, worker, "B");

    sleep(1);  // threadlerin çalışmaya başlaması için gecikme

    printf("## Cancelling thread B\n");
    // Thread B'yi iptal et
    pthread_cancel(th2);

    usleep(100000);  // 100ms bekle
    //(Thread A'nın iptal edilmesinden önce biraz daha çalışmasına izin ver)

    printf("## Cancelling thread A\n");
    // Thread A'yı iptal et
    pthread_cancel(th1);

    printf("Exiting from main program\n");
    return 0;
}
```

```
bfljpg@vbox:~$ nano cancel.c
bfljpg@vbox:~$ gcc cancel.c -o cancel -lpthread
bfljpg@vbox:~$ ./cancel
Hello from thread B
Hello from thread A
Hello from thread B
Hello from thread A
Hello from thread B
Hello from thread B
Hello from thread A
Hello from thread B
## Cancelling thread B
Hello from thread A
Hello from thread A
## Cancelling thread A
Exiting from main program
bfljpg@vbox:~$
```

- Kodumuzu yazdık, derledik ve çalıştırdık.
- Belirttiğimiz süre zarfında threadlerin çalışmasını bekledikten sonra threadlerin sonlandırıldığını ve A threadinin bir süre daha devam ettirilip sonlandığını görebilirsiniz.

Mutex

- Mutex (Mutual Exclusion), çoklu thread'lerin aynı kaynağa aynı anda erişmesini engellemek için kullanılan bir senkronizasyon aracıdır. Bu, veri tutarsızlıklarını veya yarış durumlarını (race conditions) önlemeye yardımcı olur. Bir mutex, bir thread'in belirli bir kaynağa erişimini sağlarken, diğer thread'lerin bu kaynağa erişmesini engeller. Thread bir mutex'i "kilitlediğinde" (lock), diğer thread'ler bu mutex serbest bırakılana kadar beklemek zorundadır. Mutex kullanımı, çoklu thread'lerin paralel çalıştığı programlarda veri bütünlüğünü korumak için kritik öneme sahiptir.

SpinLock & Mutex Karşılaştırması

- SpinLock, bir thread'in bir kaynağa erişim için beklemesi gerektiğinde kullanılan bir tür senkronizasyon primitifidir. SpinLock'ta bir thread, kilidi elde edene kadar aktif olarak döngü içinde bekler ("spin").
- SpinLock'lar, kilidin kısa bir süre içinde serbest bırakılacağının beklendiği durumlarda kullanılır çünkü aktif bekleme, işlemci zamanını boşa harcar. Avantajı, thread'in başka bir duruma geçmeden hızlıca kilidi kontrol etmeye devam etmesidir. Genellikle düşük seviyeli, yüksek performanslı senkronizasyon gereksinimlerinde tercih edilir.

- Linux’un zamanla mutex performansında çok ciddi iyileştirmelere gitmesinden dolayı SpinLock’un geride kalmasını bir kenara bırakacak olursak, Mutex kullanımlarının performans problemi yaratabileceği bazı senaryolar mevcut.
- Örnek olarak A ve B thread'lerinin bir mutex kaynağını kilitleyip, çok hızlı biçimde işlemi gerçekleştirip kilidi kaldırdığını, bu işlemi de saniyede binlerce veya yüzbinlerce defa yaptığını düşünelim.
- Mutex operasyonları context-switch gerektirdiğinden kilitli kalma süresi çok az olsa da bir yerden sonra her bir kilitleme işleminde thread'in Sleep durumuna geçmesi sonra tekrar uyandırılması sürecinin kendisi zaman alıcı bir işleme dönüşür.
- Böyle bir durumda Mutex kullanmak yerine SpinLock kullanımı daha avantajlı olur.

- Ancak Mutex’den farklı olarak SpinLock kullanıcı kipinde busy wait ile gerçekleştirilir. Dolayısıyla ilgili thread Sleep moduna geçmez, context-switch gerçekleşmez ve kullanıcı kipinde kendisine tahsis edilen CPU zamanını sonuna kadar kullanabilmiş olur.
- Ancak eğer ilgili uygulama cpu kullanmaya devam ederek beklediği SpinLock’a çok hızlı bir şekilde erişemez ise bu defa tersi bir etki yaratır ve Sleep moduna geçmesi halinde aynı zaman diliminde CPU ile başka bir işlem yapılabilecekken, bu imkan ortadan kalkar ve CPU yükünün artmasına neden olur.
- Dolayısıyla Mutex ve SpinLock kullanımının oluşacak senaryolara göre değerlendirilmesi ve ona göre kullanılmasını göz önünde bulundurun.

Semafor Kullanımı

- Semaforlar, işaretsiz integer değerler olarak düşünülebilir. Bir semafor üzerinde, artırma ve azaltma yönünde iki temel operasyon gerçekleştirilebilir.
- Çoklu process veya çoklu thread ortamlarında kaynak erişimini senkronize etmek için kullanılırlar. Semaforlar thread-safe'dir ve standartlaştırılmış API'lerle çalışır.
- Bu derste POSIX semaforları üzerinden gideceğiz.
- POSIX Semaforları, POSIX (Portable Operating System Interface) standardına uygun olarak geliştirilmiş semafor mekanizmalarıdır.

Semafor ve Mutex Karşılaştırması

- Semaforların pthread mutex'lerden temel farkı, sadece aynı uygulamanın thread'leri arasında değil, farklı process'ler arasındaki senkronizasyon işlemleri için de kullanılabilmesidir.
- Aynı uygulama içerisinde semafor veya mutex kullanımı açısından performans farkı görülmese de, mutex kullanımı tercih edilmelidir. Mutex kullanımı sayesinde, ancak mutex'i lock etmiş thread'in tekrar unlock yapabilmesini garanti etmiş oluruz.
- Semafor kullanılması durumunda, bir thread tarafından artırılan semafor değeri, başka bir thread tarafından düşürülebilir. Bu nedenle semaforlar, eşzamanlı uygulama geliştirme sürecindeki goto deyimi olarak da nitelendirilir.
- Bununla birlikte, farklı process'ler arasındaki senkronizasyon probleminin çözümü ve async-signal safe yapısı nedeniyle multi-threaded uygulamalarda signal handler içerisinden kullanılabilmesi, semafor kullanımının avantajlı olduğu noktalardır.

Semafor Türleri

POSIX semaforları iki türde çalışabilir:

- Adlandırılmış Semaforlar (Named Semaphores): Sistem genelinde adla tanımlanır ve farklı süreçler arasında paylaşılabilir. Örnek: /sem_open ve /sem_close.
- Adlandırılmamış Semaforlar (Unnamed Semaphores): Genellikle thread'ler arasında kullanılır ve bellekte bir değişken olarak tanımlanır. Örnek: sem_init ve sem_destroy.

İsimlendirilmiş (Named) Semafor Kullanımı

Tipik kullanımı şu şekildedir:

- sem_open() ile semafor oluşturulur veya kullanım için açılır
- sem_post() ile semaforun değeri artırılırken sem_wait() ile semaforun azaltılır
- sem_getvalue() ile o anki değeri öğrenilir
- sem_close() ile semafore erişim iptal edilir
- sem_unlink() ile semafor tamamen silinmek üzere işaretlenir, semaforu kullanan tüm process'ler sem_close() yaptıktan sonra silme gerçekleşir.

sem_open

- Yeni bir isimli semafor oluşturmak veya varolana erişebilmek için kullanılır.
- Eğer yeni bir semafor oluşuyorsa, semafora erişim izni için gerekli bit maskesi ve semaforun ilk değeri de parametre olarak verilir.
- Burada kullanılan bit maskesi, dosya işlemlerinde kullanılanlarla aynıdır.
- Semaforun oluşturulması ve ilk değerinin atanması işlemi atomik olarak gerçekleşir.
- Fonksiyon geri dönüş değeri olarak sem_t * şeklinde bir adres döndürür.
- Hata durumunda bu değer SEM_FAILED'e eşit olur.
- Başarılı durumda /dev/shm dizini altında semaforun referansı oluşur.

sem_close

- Bir süreç semafor oluşturduğunda, çekirdek tarafından ilgili sürecin semafor ilişkisi kurulur.
- sem_close fonksiyonu ile çağrıldığı sürecin ilgili semaforla ilişkisi kesilir.
- İlgili semafor için kullanıldığı süreç sayısı sayacı bir azaltılır.
- Tüm isimlendirilmiş semaforlar, ilgili süreç sonlandığında veya exec çağrısıyla yeni bir uygulama belleğe yüklendiğinde, yukarıdaki işlem çekirdek tarafından otomatik yapılır.

sem_unlink

- Bir semaforu yok etmek için sem_unlink fonksiyonu kullanılır.
- Bu fonksiyonla ilgili semaforun, kullanımda olan tüm süreçler tarafından kapatıldığında otomatik olarak silinmesi istendiğini belirtmiş oluruz.
- İlgili tüm süreçler tarafından semafor kullanımı sonlandırıldığında, semafor yok edilir ve /dev/shm altındaki dosya referansı da silinir.

Adlandırılmamış Semafor Fonksiyonları
sem_wait

- Semaforun değerini 1 azaltmak için kullanılır.
- Eğer ilgili semafor değeri 1'den büyükse azaltma işlemi anında gerçekleştirilir ve fonksiyon geri döner.
- Semafor değeri zaten 0'a eşitse, sem_wait fonksiyonu, semaforun değerinin 1 artmasını bekler. Artış olduğunda hemen tekrar 0'a çeker ve fonksiyon geri döner.
- Yukarıdaki bloklanmış bekleme durumunda bir sinyal yakalanması halinde, sinyale ilişkin SA_RESTART bayrağının ayarlanmış olup olmamasından bağımsız olarak, EINTR hatasıyla fonksiyondan çıkılır.

sem_trywait ve sem_timedwait

- Semafor bekleme operasyonu bir sinyal gelmediği müddetçe işlem gerçekleşene kadar çağıran süreci bloklamaktadır.
- Bloklama yapması istenmediği durumda, sem_trywait kullanılabilir. Bu modelde eğer semafor değeri 1 ise azaltılıp başarılı döner, ancak değer zaten 0 ise bloklama yapılmaz ve EAGAIN hatası ile dönülür.
- Bekleme işlemi için maksimum zaman aralığını seçmek istersek, sem_timedwait fonksiyonunu kullanabiliriz. Bu durumda semafor değeri hemen azaltılamıyorsa, maksimum olarak verilen parametre süresi kadar bekleyecektir. Bu süre zarfında da işlemin gerçekleşmemesi halinde ETIMEDOUT hatası ile döner.

sem_post

- Semaforun değerini 1 artırmak için kullanılır.
- Eğer semaforun değeri zaten 0 ise ve başka bir süreç aynı semaforu beklediği için bloklanmış durumdaysa, ilgili süreç uyandırılır.
- Yukarıdaki durumda birden fazla sürecin aynı semaforu beklerken bloklanması söz konusu olursa, hangi sürecin uyandırılacağı garanti edilemez.

sem_getvalue

- Mevcut bir semaforun o anki değerini almak için kullanılır.

İsimlendirilmiş Semafor ile İlgili Bir Örnek

- sem_open fonksiyonu ile isimli bir semafor (/my_named_semaphore) oluşturuluyor. Bu semafor, belirli sayıda iş parçacığının eş zamanlı olarak bir kaynağa erişmesine izin verir. Semaforun başlangıç değeri, RESOURCE_LIMIT ile belirleniyor (bu örnekte 3). Bu, aynı anda en fazla 3 iş parçacığının kaynağa erişebilmesini sağlar.
- pthread_create fonksiyonu ile 10 iş parçacığı oluşturuluyor. Her iş parçacığı bir kaynak üzerinde çalışmaya başlamadan önce semaforu bekleyecek (sem_wait), kaynak üzerinde 2 saniye çalışacak ve sonra semaforu serbest bırakacak (sem_post).
- Semafor, kaynakları yönetmek için kullanılır. Her iş parçacığı, kaynağa erişim talep ettiğinde semaforun değerini kontrol eder. Eğer semafor değeri 0 ise (yani tüm kaynaklar kullanılıyorsa), iş parçacığı semaforu bekler. Kaynak serbest bırakıldığında (sem_post), semafor değeri artırılır ve bekleyen iş parçacığı kaynağa erişebilir.
- Her iş parçacığı için bir thread_id değeri dinamik olarak ayrılmıştır (malloc ile). Bu dinamik bellek daha sonra iş parçacığı tamamlandığında free ile temizlenir.
- Tüm iş parçacıkları tamamlandığında, sem_close fonksiyonu ile semafor kapatılır ve sem_unlink ile sistemden silinir. Bu adımlar, semaforun doğru şekilde serbest bırakılmasını sağlar.

- Thread kaynak üzerinde çalışmaya başlamadan önce semaforu bekler.
- Kaynak üzerinde 2 saniye süreyle çalıştıktan sonra, semaforu serbest bırakırlar.
- Semaforun değeri, bir iş parçacığı kaynağı serbest bıraktığında artar ve diğer thread kaynağa erişebilir.
- Program aynı anda sadece 3 thread işleyebilecek şekilde kaynak limiti verilerek yazılmıştır.
- Uygulamayı derleyip çalıştırdıktan sonra threadlerin işleyişine dikkat edin.

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <fcntl.h>  // O_CREAT için gerekli

#define THREAD_COUNT 10
#define RESOURCE_LIMIT 3

// İşçi işlevi
void *worker(void *arg) {
    sem_t *resource_semaphore = (sem_t *)arg;  // Semafor pointer'ını alıyoruz
    int thread_id = *((int *)arg);  // Thread ID alıyoruz
    printf("Thread %d: Waiting to access resource...\n", thread_id);

    // Kaynak erişimi için semafor bekle
    sem_wait(resource_semaphore);
    printf("Thread %d: Accessing resource...\n", thread_id);

    // Kaynak üzerinde çalışma (simülasyon)
    sleep(2);

```

```c
    // Kaynağı serbest bırak
    printf("Thread %d: Releasing resource...\n", thread_id);
    sem_post(resource_semaphore);

    free(arg);  // Dinamik olarak ayrılan belleği temizle
    return NULL;
}

int main() {
    pthread_t threads[THREAD_COUNT];

    // İsimli semaforu başlat (kaynak limiti kadar)
    sem_t *resource_semaphore = sem_open("/my_named_semaphore", O_CREAT, 0644, RESOURCE_LIMIT);
    if (resource_semaphore == SEM_FAILED) {
        perror("sem_open failed");
        exit(EXIT_FAILURE);
    }

    for (int i = 0; i < THREAD_COUNT; i++) {
        int *thread_id = malloc(sizeof(int));  // Dinamik bellek ayırıyoruz
        if (thread_id == NULL) {
            perror("malloc");
            exit(EXIT_FAILURE);
        }
        *thread_id = i + 1;
```

```
49
50          // Thread oluştur ve semafor pointer'ını geç
51          pthread_create(&threads[i], NULL, worker, (void *)thread_id);
52      }
53
54      // Tüm thread'lerin tamamlanmasını bekle
55      for (int i = 0; i < THREAD_COUNT; i++) {
56          pthread_join(threads[i], NULL);
57      }
58
59      // Semaforu kapat ve sistemden sil
60      sem_close(resource_semaphore);
61      sem_unlink("/my_named_semaphore");
62
63      printf("All threads completed. Exiting program.\n");
64      return 0;
65  }
66
```

```
bfljpg@vbox:~$ nano named.c
bfljpg@vbox:~$ gcc named.c -o named -lpthread
```

```
bfljpg@vbox:~$ ./named
Thread 1: Waiting to access resource...
Thread 3: Waiting to access resource...
Thread 3: Accessing resource...
Thread 5: Waiting to access resource...
Thread 5: Accessing resource...
Thread 6: Waiting to access resource...
Thread 6: Accessing resource...
Thread 8: Waiting to access resource...
Thread 8: Accessing resource...
Thread 7: Waiting to access resource...
Thread 7: Accessing resource...
Thread 3: Releasing resource...
Thread 7: Releasing resource...
Thread 4: Releasing resource...
Thread 2: Releasing resource...
Thread 10: Releasing resource...
Thread 6: Releasing resource...
Thread 9: Releasing resource...
Thread 8: Releasing resource...
All threads completed. Exiting program.
bfljpg@vbox:~$
```

- İlk resimde görüldüğü gibi nano ile düzenleyip gösterilen gcc komutuyla derleme aldık.
- Uygulamadaki temel amaç semafor oluşturarak ve sonlandırarak kaynak akışını yönetmektir.
- İkinci resimde threadlerin oluşturulması için sıraya girdiğini görebilirsiniz.
- Üçüncü resimde işe sırasıyla kaynağın yok edildiğini görebilirsiniz.
- 10 olan thread sayısını düşürerek tekrar deneyebilir ve programın çalışırken olan gidişatını tekrar gözlemleyebilirsiniz.
- Sayıların yerine göre sıralı olmaması sizi yanıltmasın. Zira çalıştırırken diğer threadleri sıraya almayıp beklettiğini göreceksiniz.

İsimlendirilmemiş Semafor ile İlgili Bir Örnek

- sem_init fonksiyonu ile isimsiz bir semafor oluşturuluyor. İsimsiz semafordaki tek fark oluşturulan semaforun herhangi bir ismi bulunmuyor.
- Bir önceki örneğe kıyasla bu örnekteki fark isimlendirilmemiş semaforun kullanım şekli ve programdaki yeridir.
- Bu durumun haricinde programın çalışma mantığı, semaforun görevi, threadlerin senkronize çalışma durumu gibi şeyler tamamen aynıdır.

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define THREAD_COUNT 10
#define RESOURCE_LIMIT 3

// Isimsiz semafor tanımı
sem_t resource_semaphore;

// İşçi işlevi
void *worker(void *arg) {
    int thread_id = *((int *)arg);
    printf("Thread %d: Waiting to access resource...\n", thread_id);

    // Kaynak erişimi için semafor bekle
    sem_wait(&resource_semaphore);
    printf("Thread %d: Accessing resource...\n", thread_id);

    // Kaynak üzerinde çalışma (simülasyon)
    sleep(2);

```

```c
    // Kaynağı serbest bırak
    printf("Thread %d: Releasing resource...\n", thread_id);
    sem_post(&resource_semaphore);

    free(arg); // Dinamik olarak ayrılan belleği temizle
    return NULL;
}

int main() {
    pthread_t threads[THREAD_COUNT];

    // Isimsiz semaforu başlat (kaynak limiti kadar)
    sem_init(&resource_semaphore, 0, RESOURCE_LIMIT);

    for (int i = 0; i < THREAD_COUNT; i++) {
        int *thread_id = malloc(sizeof(int));
        if (thread_id == NULL) {
            perror("malloc");
            exit(EXIT_FAILURE);
        }
        *thread_id = i + 1;

```

```
47          // Thread oluştur
48          pthread_create(&threads[i], NULL, worker, thread_id);
49      }
50
51      // Tüm thread'lerin tamamlanmasını bekle
52      for (int i = 0; i < THREAD_COUNT; i++) {
53          pthread_join(threads[i], NULL);
54      }
55
56      // Semaforu temizle
57      sem_destroy(&resource_semaphore);
58
59      printf("All threads completed. Exiting program.\n");
60      return 0;
61  }
```

```
bfljpg@vbox:~$ nano unnamed.c
bfljpg@vbox:~$ gcc unnamed.c -o unnamed -lpthread
```

```
bfljpg@vbox:~$ ./unnamed
Thread 1: Waiting to access resource...
Thread 1: Accessing resource...
Thread 2: Waiting to access resource...
Thread 5: Waiting to access resource...
Thread 5: Accessing resource...
Thread 3: Waiting to access resource...
Thread 2: Accessing resource...
Thread 6: Waiting to access resource...
Thread 4: Waiting to access resource...
```

```
Thread 6: Releasing resource...
Thread 8: Accessing resource...
Thread 4: Releasing resource...
Thread 9: Accessing resource...
Thread 7: Releasing resource...
Thread 10: Accessing resource...
Thread 8: Releasing resource...
Thread 9: Releasing resource...
Thread 10: Releasing resource...
All threads completed. Exiting program.
bfljpg@vbox:~$
```

- İlk resimde görüldüğü gibi nano ile kodumuzu yazmaya başladık ve belirtilen şekilde gcc ile derlememizi aldık.
- İkinci resimde threadlerin oluşturulması için sıraya girdiğini görebilirsiniz.
- Üçüncü resimde işe sırasıyla kaynağın yok edildiğini görebilirsiniz.
- 10 olan thread sayısını düşürerek tekrar deneyebilir ve programın çalışırken olan gidişatını tekrar gözlemleyebilirsiniz.
- Sayıların yerine göre sıralı olmaması sizi yanıltmasın. Zira çalıştırırken diğer threadleri sıraya almayıp beklettiğini göreceksiniz.

Shared Memory

- Shared memory (paylaşımlı bellek), birden fazla sürecin (process) aynı bellek alanını kullanmasına olanak tanıyan bir iletişim mekanizmasıdır. Bu mekanizma, birden fazla sürecin hızlı bir şekilde veri paylaşmasını sağlar. Süreçler arasında doğrudan bellek erişimi sağlanarak veri iletimi yapılır, bu da genellikle daha hızlıdır çünkü veri kopyalama işlemleri yapılmaz.
- Semafor kullanımına benzer şekilde, POSIX shared memory objeleri de Linux altında tmpfs dosya sistemi ile /dev/shm dizini altına bağlanmış halde tutulur.
- Bu dizin tmpfs türünde öntanımlı olarak, mevcut sistem belleğinin maksimum yarısını kullanacak şekilde bağlanır ancak mount işleminde parametre vererek bu değeri değiştirmek mümkündür

- Paylaşımlı bellek bölgesi, bir süreç (genellikle işletim sistemi) tarafından oluşturulur. Diğer süreçler bu bölgeyi kendi adres alanlarına haritalayarak (mapping) erişebilirler. Bu sayede, süreçler aynı bellek alanını okuyup yazabilirler.
- Birden fazla süreç aynı bellek alanına erişebileceği için, senkronizasyon önemlidir. Bu, veri tutarlılığını sağlamak için semaforlar, mutex'ler gibi senkronizasyon araçlarının kullanılmasını gerektirir. Eğer senkronizasyon sağlanmazsa, veri çakışmaları ve hatalar meydana gelebilir.
- Paylaşımlı bellek, mesaj kuyruğu ya da boru hatları gibi diğer IPC (inter-process communication - süreçler arası iletişim) yöntemlerine göre daha hızlıdır çünkü verinin bir süreçten diğerine iletilmesi için kopyalanmasına gerek yoktur. Süreçler doğrudan aynı bellek bölgesine erişirler.
- Veritabanları, gerçek zamanlı uygulamalar ve işletim sistemleri gibi büyük miktarda veri paylaşılması gereken sistemlerde sıklıkla kullanılır.

shm_open

- shm_open fonksiyonu, standart kütüphanedeki open fonksiyonuyla aynı arayüze sahiptir.
- Parametre olarak dosya ismi, işleme dair ayarlanan opsiyonlar ve açılacak kaynak üzerindeki erişim yetkilerini alır.
- open fonksiyonundakine benzer şekilde O_CREAT, O_EXCL, O_RDONLY, O_RDRW ve O_TRUNC opsiyonlarının bir veya birkaçı birleştirilerek parametre olarak verilebilir.
- Geriye dönen file descriptor referansı üzerinden fstat fonksiyonuyla yapacağımız kontroller ile, shared memory alanının büyüklüğünü öğrenebiliriz. Yeni açılan shared memory alanlarının boyutu başlangıçta sıfırdır.
- Memory map işlemini yapmadan önce, ftruncate fonksiyonu ile shared memory alanının boyutunu istediğimiz değere ayarlayabiliriz. Bu işlem üretilen alandaki bellek bölgesini '\0' değerleriyle doldurur.

Fonksiyon gösterimi:

- int shm_open(const char *name, int oflag, mode_t mode);

Parametreler:

- name: Paylaşımlı bellek nesnesinin adı (örneğin, "/my_shared_memory").
- oflag: Nesnenin nasıl açılacağını belirleyen bayraklar. O_CREAT (nesne yoksa oluşturulması için) veya O_RDWR (okuma ve yazma için) gibi bayraklar kullanılabilir.
- mode: Nesne oluşturulurken verilmesi gereken izinler (örneğin, S_IRUSR | S_IWUSR).

Açıklama:

- Bu fonksiyon, paylaşımlı belleği açar ya da oluşturur.
- Eğer nesne zaten varsa, belirtilen bayraklar ve izinlerle açılır.
- Nesne, diğer süreçler tarafından da erişilebilir olur.

shm_unlink

- Shared memory alanını kaldırmak istediğimizde kullanılır.
- shm_unlink işlemi sonrası, ilgili alanını memory-map yöntemiyle kullanmakta olan diğer süreçlerin mapping bilgilerini etkilemez. İlgili süreçlerde ayrıca munmap fonksiyonu çağrılmalıdır.
- Shared memory alanını kullanan tüm süreçler shm_unlink yaptıktan veya sonlandıktan sonra, işletim sistemi tarafından ilgili alan tamamen kaldırılır.

Fonksiyon gösterimi:

- int shm_unlink(const char *name);

Parametreler:

- name: Kaldırılacak paylaşımlı bellek nesnesinin adı.

Açıklama:

- Paylaşımlı bellek nesnesinin sistemdeki adını siler.
- Nesnenin içeriği, hâlâ bu nesneye bağlı olan süreçler tarafından erişilebilir.
- shm_unlink kullanıldıktan sonra, başka süreçler bu isme sahip yeni bir paylaşımlı bellek açabilir.

mmap

- mmap (memory map) fonksiyonu, bir dosyayı ya da başka bir kaynağı, bellek adresi alanına yerleştirerek, doğrudan okuma ve yazma işlemlerinin yapılmasını sağlayan bir sistem çağrısıdır. Bu işlem, bellekteki belirli bir alanı (bellek haritası) dosyaya veya başka bir kaynağa bağlar ve süreçlerin bu alanı doğrudan okumasına veya yazmasına olanak tanır.
- mmap, bir dosyayı veya bellek bölgesini, işlemci tarafından erişilebilen bellek adres alanına haritalar. Bu sayede, dosya üzerinde doğrudan okuma/yazma işlemi yapılabilir.
- Geleneksel dosya okuma/yazma işlemlerinde, veriler diske ve bellek arasında kopyalanır. Ancak mmap, dosya içeriğini belleğe haritalayarak bu kopyalama işlemi gereksiz kılar ve veri daha hızlı bir şekilde erişilebilir hale gelir.

- Fonksiyon gösterimi:
- void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
- Parametreler:
- addr: Bellek haritasının başlama adresi. Genellikle NULL verilir ve işletim sistemi uygun bir adres seçer.
- length: Haritalama yapılacak alanın boyutu.
- prot: Bellek bölgesinin erişim izinleri (örneğin PROT_READ, PROT_WRITE).
- flags: Bellek haritalamasının nasıl yapılacağını belirleyen bayraklar. Örneğin, MAP_SHARED (paylaşımlı) veya MAP_PRIVATE (özel) gibi.
- fd: Haritalanacak dosyanın dosya tanımlayıcısı.
- offset: Dosyanın başlangıcından itibaren kaç bayt kadar haritalama yapılacağı.

Açıklama:

- mmap, dosya verilerini doğrudan belleğe yerleştirir ve bu sayede bellekteki veriler üzerinde hızlı okuma/yazma işlemleri yapılabilir.
- Paylaşımlı bellek, dosya veya cihazlar için kullanılabilir.
- Yalnızca belleğe haritalama yapılan dosya verilerine değil, aynı zamanda paylaşımlı bellek alanlarına da erişim sağlar.

mmap ve munmap için Genel Özet:

- mmap, farklı süreçlerin aynı bellek alanına erişmesini sağlayarak paylaşımlı bellek alanları oluşturabilir. Bu, süreçler arasında veri paylaşımını kolaylaştırır.
- munmap fonksiyonu mmap işleminin tersini yaparak ilgili sürecin bellek alanındaki mapping'i kaldırır.
- Bellek üzerinde yapılan operasyonların dosya üzerinde de gerçekleştiğinden emin olmak için, munmap öncesinde msync fonksiyonunu kullanmak gerekir.

POSIX Shared Memory Örneği

- Bu örnekte shm_open(), paylaşımlı bellek alanını oluşturur veya açar. mmap(), paylaşımlı belleği işleme alanına haritalar (belleğe erişim sağlar). shm_unlink(), paylaşımlı bellek nesnesini sistemden kaldırır.
- Göreceğiniz örnek temel olarak shared memory kullanımını göstermektedir.
- Bu programda açtığımız shared memory üzerine 123 değeri yazdırarak çıktısını almayı öğreneceğiz.

```c
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>
#include <stdio.h>
#include <string.h>

int main() {
    const char *name = "/my_shared_memory";
    int shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);  // Paylaşımlı belleği oluştur
    ftruncate(shm_fd, sizeof(int));  // Paylaşımlı belleğin boyutunu ayarla

    int *shared_memory = mmap(0, sizeof(int), PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
    *shared_memory = 123;  // Paylaşımlı belleğe veri yaz

    printf("Paylaşımlı bellek değeri: %d\n", *shared_memory);

    munmap(shared_memory, sizeof(int));  // Paylaşımlı belleği serbest bırak
    shm_unlink(name);  // Paylaşımlı bellek nesnesini sil
    return 0;
}
```

```
bfljpg@vbox:~$ nano shared_memory.c
bfljpg@vbox:~$ gcc shared_memory.c -o sharedmem
bfljpg@vbox:~$ ./sharedmem
Paylaşımlı bellek değeri: 123
bfljpg@vbox:~$ ./sharedmem
Paylaşımlı bellek değeri: 123
bfljpg@vbox:~$
```

- Kodumuzu yazıp derledik ve çalıştırdık.
- Göründüğü üzere programda kodda da verdiğimiz gibi 123 değerini yazdırdı ve programı sonlandırdı

Memory Mapped I/O

- Memory Mapped I/O, read, write fonksiyonlarıyla yapılacak bir IO operasyonunun, ilgili processin adreslediği bellek alanı içerisinde doğrudan bellek erişim yöntemleriyle gerçeklenmesini sağlar.
- Memory Mapped I/O, temelde farklı processlerin belleğe doğrudan erişerek aynı dosyada farklı şekillerde dosya işlenebilmesini sağlar
- Eğer mapping işlemi, MAP_PRIVATE bayrağı ile oluşturulmuş ise, ilgili sürecin yaptığı değişiklikleri diğer süreçler göremezler. Bu şekildeki bir kullanım aynı zamanda, yapılan değişikliklerin dosyaya da tekrar yazılmayacağı anlamına gelmektedir.
- MAP_SHARED bayrağı ile oluşturulmuş mapping'lerde ise, diğer süreçler de yapılan değişiklikleri görebilirler. Bu değişiklikler işletim sistemi tarafından dosyaya da yansıtılır.

Memory Mapped IO Dezavantajları

- Küçük çaplı IO işlemlerinde, memory mapped I/O yönteminin performansı daha düşük olacaktır.
- Bunun en büyük sebebi bellek adresleme yaparak işlem yapmasından dolayı gereksiz performans harcanması ve gereksiz mapping işlemi.
- Bir diğer problem de bellek üzerinde yapılan işlemlerin dosyaya yansıtılmasını garanti etmek için ek işlem yapılma ihtiyacı olarak gösterilebilir.
- Bu nedenlerle basit dosya işlemlerinde kullanımı genellikle önerilmez.

Kaynakça

- https://demirten.gitbooks.io/linux-sistem-programlama/, Murat Demirten – Linux Sistem Programlama
- https://medium.com/@abhilashkrish/i-o-models-81d147ce3bda, Abhilash Krishnan
- http://davmac.org/davpage/linux/rtsignals.html, Davin McCall